

Apprentissage supervisé

Sébastien DA VEIGA
ENSAI - CREST

2023–2024

Résumé

Ce polycopié a été préparé pour les élèves de deuxième année de l'ENSAI qui suivent le cours d'apprentissage supervisé. Les principaux concepts et méthodes abordés sont les suivants :

- Introduction générale, objectifs, notations et définitions
- Concept de risque, validation et sélection de modèles
- Premiers modèles d'apprentissage supervisé : Bayésien naïf, analyse discriminante, plus proches voisins
- Arbres CART et forêts aléatoires
- Comparaison de modèles en classification, courbes ROC
- Apprentissage supervisé pratique (variables catégorielles, grande dimension, sélection / transformation de variables)

Le cours consiste en 6 séances de 3h chacune, avec 3 séances additionnelles de TD/TP de 3h également. Concernant l'évaluation, un TP supplémentaire à faire à la maison comptera pour 1/3 de la note finale, et l'examen terminal pour 2/3.

Table des matières

1	Risque	6
1.1	Définition du risque	6
1.2	Risque empirique	10
1.3	Validation simple, validation croisée, validation Monte-Carlo . . .	14
1.3.1	La validation simple ("hold out")	14
1.3.2	La validation croisée ("cross-validation")	15
1.3.3	La validation croisée Monte-Carlo ou Bootstrap	16
1.4	Quelques remarque supplémentaires sur le découpage d'échantillons	17
1.5	Conclusions et introduction au prochain cours	17
2	Quelques modèles d'apprentissage supervisé	20
2.1	Variations autour du classifieur de Bayes	20
2.1.1	Le classifieur de Bayes naïf	21
2.1.2	L'analyse discriminante linéaire ou quadratique	24
2.2	Méthode des plus proches voisins	29
2.2.1	Principe général	29
2.2.2	Vers le lissage et les méthodes à noyau	36
2.3	Le fléau de la dimension	39
3	Arbres et forêts aléatoires	44
3.1	Arbres CART	44
3.1.1	Introduction aux modèles d'arbres	44
3.1.2	Algorithme CART	46
3.2	Forêts aléatoires	63
4	Compléments sur la comparaison de modèles en classification	71
4.1	Définition d'une matrice de confusion	73
4.2	Coût différencié pour la construction d'un arbre	75
4.3	Fonction de score et courbes ROC	79
4.4	Extension à la classification multi-classes	87
5	Apprentissage supervisé pratique	95
5.1	Gérer les variables explicatives catégorielles	96
5.1.1	Impact sur les modèles vus en cours	96
5.1.2	Stratégie générale : l'encodage	96
5.1.3	Gestion des catégories rares	98
5.2	Jeux de données de grande taille	98
5.3	Gérer la grande dimension des variables explicatives	98

5.3.1	Motivation et Contexte	98
5.3.2	Sélection de variables	99
5.3.3	Projection / Plongement / Embedding	101
5.4	Normalisation et transformations préliminaires de variables	102
5.4.1	Impact sur les modèles vus en cours	103
5.4.2	Transformations	103
5.5	Gérer les données manquantes	104
5.5.1	Généralités	104
5.5.2	Traitement des valeurs manquantes	105
5.6	Optimisation des hyperparamètres d'un modèle	107
5.7	Visualiser, visualiser, visualiser	107
5.8	Bonus 1 : gérer des variables explicatives plus "exotiques"	109
5.9	Bonus 2 : l'apprentissage semi-supervisé	112

Séance 1 : Introduction à l'apprentissage supervisé

28 Août, 2023

- Introduction générale
 - Les différents types d'apprentissage
 - Panorama des concepts et méthodes, liens avec autres cours de l'EN-SAI
- Notations et vocabulaires
 - Variables explicatives et à expliquer
 - Les différents types de variables, distinction régression / classification
- Mon premier exemple de modèle : le classifieur des plus proches voisins
 - Intuition et visualisation
 - Teasing du prochain cours avec ChatGPT

Séance 2 : Notion de risque, classifieur de Bayes, validation / sélection de modèles

5 Septembre, 2023

- Concept de risque
 - Définition et propriétés d'une fonction de perte, exemples
 - Définition du risque théorique
 - Optimalité et classifieur de Bayes
- Risque empirique
 - Définition du risque empirique
 - Compromis biais-variance, sous- et sur-apprentissage
- Ré-échantillonnage par validation simple, croisée et Monte-Carlo

Chapitre 1

Risque

Rappel : on dispose d'un n -échantillon $\mathcal{D}_n = \{\mathbf{x}_i, y_i\}_{i=1, \dots, n}$ d'une loi jointe $\mathbb{P}_{\mathbf{X}Y}$ définie sur $\mathcal{X} \times \mathcal{Y}$. $\mathcal{D}_n$ est un échantillon d'apprentissage. L'objectif est de construire un prédicteur g , noté plutôt f en régression et h en classification, qui à une entrée $\mathbf{x}$ associe une sortie prédite $y = g(\mathbf{x})$.

1.1 Définition du risque

Quand il va s'agir d'essayer de trouver le "meilleur" prédicteur possible, ou quand on veut comparer des prédicteurs entre eux pour choisir le meilleur, il faut obligatoirement se donner un critère qui va mesurer la qualité de prédiction d'un prédicteur g donné.

Pour construire un tel critère global, l'ingrédient de base est ce que l'on appelle une **fonction de perte** qui permet de mesurer une différence entre deux valeurs dans l'espace des sorties $\mathcal{Y}$.

Définition 1 (Fonction de perte). Une fonction mesurable $l : \mathcal{Y} \times \mathcal{Y} \mapsto \mathbb{R}_+$ est une fonction de perte si $l(y, y) = 0$ et $l(y, y') > 0$ pour $y \neq y'$. Une telle fonction est notée l pour "loss" en anglais.

En particulier, si pour une entrée $\mathbf{x}$ donnée la sortie qui lui est réellement associée est y , alors $l(y, g(\mathbf{x}))$ mesure la perte encourue lorsque l'on prédit $g(\mathbf{x})$ pour $\mathbf{x}$.

La définition d'une fonction de perte laisse beaucoup de flexibilité, mais en pratique, certaines pertes sont très (voire quasi-systématiquement) utilisées. Évidemment elles dépendent du contexte, à savoir si on a un problème de régression ou de classification :

- En **régression**, on définit des pertes L^p ($p \geq 1$)

$$l(y, y') = |y - y'|^p.$$

Pour $p = 1$, c'est la perte absolue. Pour $p = 2$, c'est la perte quadratique

$$l(y, y') = (y - y')^2$$

qui est de loin la plus utilisée.

- En **classification**, la perte 0-1 est donnée par

$$l(y, y') = \mathbb{1}_{y \neq y'}.$$

On la note 0-1 car elle ne prend que ces deux valeurs. Dans ce cours, on se focalisera sur cette fonction de perte. Mais sachez qu'il en existe deux autres fameuses définies un peu différemment, voir remarque ci-dessous.

Remarque. Dans le cas où on ne s'intéresse qu'au classifieur final $h(\mathbf{x}) \in \{0, 1\}$ ou $\{-1, +1\}$ en classification **binaire**, la perte encourue avec la fonction de perte 0-1 s'écrit $l(y, h(\mathbf{x})) = \mathbb{1}_{y \neq h(\mathbf{x})}$. Mais comme nous l'avons dit dans le Cours 1, très souvent le classifieur h s'écrit comme la transformation d'une fonction réelle f qui s'appelle la fonction de décision. Cette transformation s'écrit par exemple

- Cas $\{0, 1\}$: $h(\mathbf{x}) = \mathbb{1}_{f(\mathbf{x}) > 0}$
- Cas $\{-1, +1\}$: $h(\mathbf{x}) = \text{sign}(f(\mathbf{x}))$

Par abus de notation, on peut définir alors la perte encourue par le classifieur h via la fonction de décision f en écrivant $l(y, f(\mathbf{x}))$. Cet abus de notation est pratique car il permet de considérer d'autres fonctions de perte qui vont dépendre de f au lieu de h . En particulier (vous verrez l'utilité de cette remarque en troisième année), des fonctions de perte continues ou dérivables par rapport à f . Les deux fameuses sont :

- La perte **hinge** qui s'écrit

$$l_{\text{hinge}}(y, f(\mathbf{x})) = \max(0, 1 - yf(\mathbf{x})) = [1 - yf(\mathbf{x})]_+$$

(cas $\{-1, +1\}$). Elle est utilisée pour les support vector machines (cours de machine learning 3A).

- La perte **entropie-croisée** ("cross-entropy")

$$l_{\text{CE}}(y, f(\mathbf{x})) = -\mathbb{1}_{y=0} \log(1/2 - f(\mathbf{x})) - \mathbb{1}_{y=1} \log(f(\mathbf{x}) - 1/2)$$

(cas $\{0, 1\}$). Vous la reconnaitrez peut-être dans votre cours de GLM en 2A !

Maintenant que l'on dispose d'une fonction de perte, cela nous permet de calculer l'erreur commise par un prédicteur pour **une valeur** de $(\mathbf{x}, y)$ donnée. La qualité finale du prédicteur est alors obtenue en calculant le comportement moyen de la perte, moyenne au sens où $(\mathbf{x}, y)$ va suivre la loi $\mathbb{P}_{\mathbf{X}Y}$ sous-jacente au problème. Ce comportement moyen s'appelle **le risque** ou l'erreur de généralisation.

Définition 2 (Risque). Étant donnée une fonction de perte l , le **risque** d'un prédicteur g est défini par

$$R_{\mathbb{P}_{\mathbf{X}Y}}(g) = \mathbb{E}_{(X,Y) \sim \mathbb{P}_{\mathbf{X}Y}}[l(Y, g(\mathbf{X}))].$$

On l'appellera également le risque théorique.

Evidemment, l'objectif final serait de trouver le prédicteur qui a le risque le plus faible, i.e. on cherche à minimiser le risque. Peut-on le faire? La réponse est oui! En classification binaire, on a le théorème suivant.

Théorème 1 (Classifieur de Bayes). Le classifieur h qui minimise le risque $R_{\mathbb{P}_{\mathbf{X}Y}}(h)$ pour la fonction de perte 0-1 est

$$h^*(\mathbf{x}) = \begin{cases} 1 & \text{si } \eta(\mathbf{x}) > \frac{1}{2} \\ 0 & \text{sinon} \end{cases}$$

où $\eta(\mathbf{x}) = \mathbb{P}(Y = 1 | \mathbf{X} = \mathbf{x})$.

Ce classifieur est le **classifieur de Bayes** et son risque $R_{\mathbb{P}_{\mathbf{X}Y}}(h^*)$ est le **risque de Bayes** $R_{\mathbb{P}_{\mathbf{X}Y}}^*$. Autrement dit, on a

$$\forall h, R_{\mathbb{P}_{\mathbf{X}Y}}^* = R_{\mathbb{P}_{\mathbf{X}Y}}(h^*) \leq R_{\mathbb{P}_{\mathbf{X}Y}}(h)$$

Preuve. On va montrer que $R_{\mathbb{P}_{\mathbf{X}Y}}(h) - R_{\mathbb{P}_{\mathbf{X}Y}}(h^*) \geq 0$ (dans la suite pour alléger la notation on va supprimer l'indexation par $\mathbb{P}_{\mathbf{X}Y}$). On remarque d'abord que

$$\begin{aligned} R(h) &= \mathbb{E}_{\mathbf{X}Y}[\mathbb{1}_{Y \neq h(\mathbf{X})}] \\ &= \mathbb{E}_{\mathbf{X}}[\mathbb{E}[\mathbb{1}_{Y \neq h(\mathbf{X})}]] \\ &= \mathbb{E}_{\mathbf{X}}[\mathbb{P}(Y \neq h(\mathbf{X}) | \mathbf{X})], \end{aligned}$$

ce qui implique par linéarité de l'espérance

$$\begin{aligned} R(h) - R(h^*) &= \mathbb{E}_{\mathbf{X}}[\mathbb{P}(Y \neq h(\mathbf{X}) | \mathbf{X}) - \mathbb{P}(Y \neq h^*(\mathbf{X}) | \mathbf{X})] \\ &= \int [\mathbb{P}(Y \neq h(\mathbf{X}) | \mathbf{X} = \mathbf{x}) - \mathbb{P}(Y \neq h^*(\mathbf{X}) | \mathbf{X} = \mathbf{x})] d\mathbb{P}_{\mathbf{X}}(\mathbf{x}). \end{aligned}$$

Il suffit donc de montrer que

$$\forall \mathbf{x} \in \mathcal{X}, \mathbb{P}(Y \neq h(\mathbf{X}) | \mathbf{X} = \mathbf{x}) - \mathbb{P}(Y \neq h^*(\mathbf{X}) | \mathbf{X} = \mathbf{x}) \geq 0.$$

On écrit ensuite que

$$\begin{aligned} \mathbb{P}(Y \neq h(\mathbf{X}) | \mathbf{X} = \mathbf{x}) &= 1 - \mathbb{P}(Y = h(\mathbf{X}) | \mathbf{X} = \mathbf{x}) \\ &= 1 - \mathbb{P}(Y = 1, h(\mathbf{X}) = 1 | \mathbf{X} = \mathbf{x}) \\ &\quad - \mathbb{P}(Y = 0, h(\mathbf{X}) = 0 | \mathbf{X} = \mathbf{x}) \end{aligned}$$

On va montrer maintenant que

$$\forall \mathbf{x}, \mathbb{P}(Y = 1, h(\mathbf{X}) = 1 | \mathbf{X} = \mathbf{x}) = \mathbb{P}(Y = 1 | \mathbf{X} = \mathbf{x}) \mathbb{P}(h(\mathbf{X}) = 1 | \mathbf{X} = \mathbf{x}).$$

On considère d'abord les $\mathbf{x}$ tels que $h(\mathbf{x}) = 1$. Conditionnellement à $X = \mathbf{x}$, on a $h(\mathbf{X}) = h(\mathbf{x})$, d'où

$$\begin{aligned} \mathbb{P}(Y = 1, h(\mathbf{X}) = 1 | \mathbf{X} = \mathbf{x}) &= \mathbb{P}(Y = 1, h(\mathbf{x}) = 1 | \mathbf{X} = \mathbf{x}) \\ &= \mathbb{P}(Y = 1 | \mathbf{X} = \mathbf{x}) \text{ car } h(\mathbf{x}) = 1 \text{ (hyp.)} \\ &= \mathbb{P}(Y = 1 | \mathbf{X} = \mathbf{x}) \times 1 \\ &= \mathbb{P}(Y = 1 | \mathbf{X} = \mathbf{x}) \mathbb{P}(h(\mathbf{X}) = 1 | \mathbf{X} = \mathbf{x}). \end{aligned}$$

Maintenant, pour les $\mathbf{x}$ tels que $h(\mathbf{x}) = 0$, on a

$$\begin{aligned} \mathbb{P}(Y = 1, h(\mathbf{X}) = 1 | \mathbf{X} = \mathbf{x}) &= \mathbb{P}(Y = 1, h(\mathbf{x}) = 1 | \mathbf{X} = \mathbf{x}) \\ &= 0 \text{ car } h(\mathbf{x}) = 0 \text{ (hyp.)} \\ &= \mathbb{P}(Y = 1 | \mathbf{X} = \mathbf{x}) \times 0 \\ &= \mathbb{P}(Y = 1 | \mathbf{X} = \mathbf{x}) \mathbb{P}(h(\mathbf{X}) = 1 | \mathbf{X} = \mathbf{x}). \end{aligned}$$

L'égalité est donc vraie pour tous les $\mathbf{x}$, et de manière symétrique on montre que

$$\forall \mathbf{x}, \mathbb{P}(Y = 0, h(\mathbf{X}) = 0 | \mathbf{X} = \mathbf{x}) = \mathbb{P}(Y = 0 | \mathbf{X} = \mathbf{x}) \mathbb{P}(h(\mathbf{X}) = 0 | \mathbf{X} = \mathbf{x}).$$

On note aussi que $\mathbb{P}(h(\mathbf{X}) = a | \mathbf{X} = \mathbf{x}) = \mathbb{1}_{h(\mathbf{x})=a}$ pour $a \in \{0, 1\}$, ce qui nous permet d'écrire que

$$\begin{aligned} \mathbb{P}(Y \neq h(\mathbf{X}) | \mathbf{X} = \mathbf{x}) &= 1 - \mathbb{1}_{h(\mathbf{x})=1} \mathbb{P}(Y = 1 | \mathbf{X} = \mathbf{x}) \\ &\quad - \mathbb{1}_{h(\mathbf{x})=0} \mathbb{P}(Y = 0 | \mathbf{X} = \mathbf{x}) \end{aligned}$$

d'où il vient

$$\begin{aligned} &\mathbb{P}(Y \neq h(\mathbf{X}) | \mathbf{X} = \mathbf{x}) - \mathbb{P}(Y \neq h^*(\mathbf{X}) | \mathbf{X} = \mathbf{x}) \\ &= (\mathbb{1}_{h^*(\mathbf{x})=1} - \mathbb{1}_{h(\mathbf{x})=1}) \mathbb{P}(Y = 1 | \mathbf{X} = \mathbf{x}) \\ &\quad + (\mathbb{1}_{h^*(\mathbf{x})=0} - \mathbb{1}_{h(\mathbf{x})=0}) \mathbb{P}(Y = 0 | \mathbf{X} = \mathbf{x}). \end{aligned}$$

On utilise ensuite que $\mathbb{P}(Y = 0 | \mathbf{X} = \mathbf{x}) = 1 - \mathbb{P}(Y = 1 | \mathbf{X} = \mathbf{x})$ et que $\mathbb{1}_{h(\mathbf{x})=0} = 1 - \mathbb{1}_{h(\mathbf{x})=1}$ (idem pour h^*) pour avoir enfin

$$\begin{aligned} &\mathbb{P}(Y \neq h(\mathbf{X}) | \mathbf{X} = \mathbf{x}) - \mathbb{P}(Y \neq h^*(\mathbf{X}) | \mathbf{X} = \mathbf{x}) \\ &= (2\mathbb{P}(Y = 1 | \mathbf{X} = \mathbf{x}) - 1) (\mathbb{1}_{h^*(\mathbf{x})=1} - \mathbb{1}_{h(\mathbf{x})=1}) \\ &= (2\eta(\mathbf{x}) - 1) (\mathbb{1}_{h^*(\mathbf{x})=1} - \mathbb{1}_{h(\mathbf{x})=1}). \end{aligned}$$

On peut alors conclure car

- Si $\eta(\mathbf{x}) > 1/2$, le premier terme est positif et on a $\mathbb{1}_{h^*(\mathbf{x})=1} = 1$. Comme $\mathbb{1}_{h(\mathbf{x})=1} \leq 1$, alors le second terme est positif également.
- Si $\eta(\mathbf{x}) < 1/2$, le premier terme est négatif et on a $\mathbb{1}_{h^*(\mathbf{x})=1} = 0$. Comme $\mathbb{1}_{h(\mathbf{x})=1} \geq 0$, alors le second terme est négatif également.

□

MAIS on voit dans ce théorème que le classifieur de Bayes fait intervenir $\eta(\mathbf{x}) = \mathbb{P}(Y = 1 | \mathbf{X} = \mathbf{x})$:

- Si on connaît la loi jointe $\mathbb{P}_{\mathbf{X}Y}$, on peut faire le calcul : vous vous entraînerez d'ailleurs à le faire en TD.
- Mais dans les situations pratiques, on ne connaît pas $\mathbb{P}_{\mathbf{X}Y}$, on a juste accès à un n -échantillon $\mathcal{D}_n$ distribué selon $\mathbb{P}_{\mathbf{X}Y}$. Donc le classifieur de Bayes n'est pas accessible en pratique, de même que le risque théorique $R_{\mathbb{P}_{\mathbf{X}Y}}(h)$ pour n'importe quel classifieur h !

Comment fait-on donc ? C'est là que le risque empirique entre en jeu.

1.2 Risque empirique

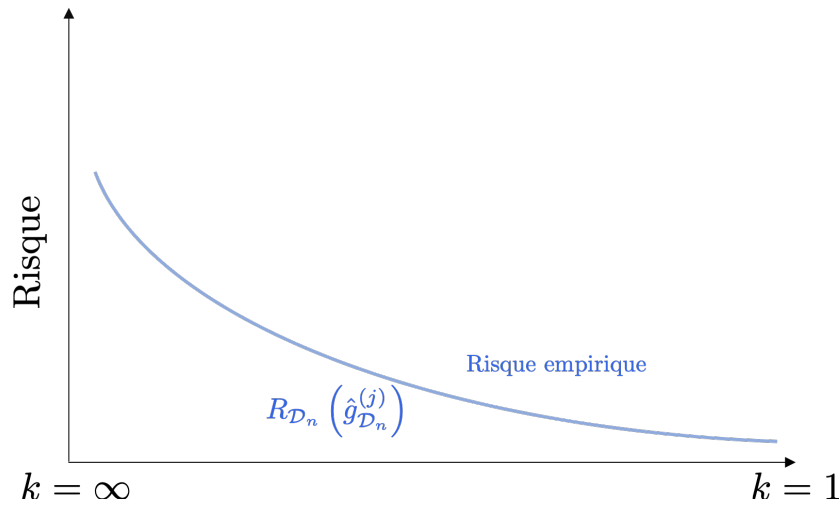
Le risque théorique est donné sous la forme d'une espérance, donc au lieu de le calculer avec $\mathbb{P}_{\mathbf{X}Y}$ inconnue, on va le calculer avec la loi empirique associée à l'échantillon $\mathcal{D}_n$, c'est-à-dire qu'on remplace l'espérance par une moyenne empirique.

Définition 3 (Risque empirique). Étant donné une fonction de perte l et un n -échantillon $\mathcal{D}_n = \{\mathbf{x}_i, y_i\}_{i=1, \dots, n}$, le **risque empirique** d'un prédicteur g est défini par

$$\hat{R}_{\mathcal{D}_n}(g) = \frac{1}{n} \sum_{i=1}^n l(y_i, g(\mathbf{x}_i)).$$

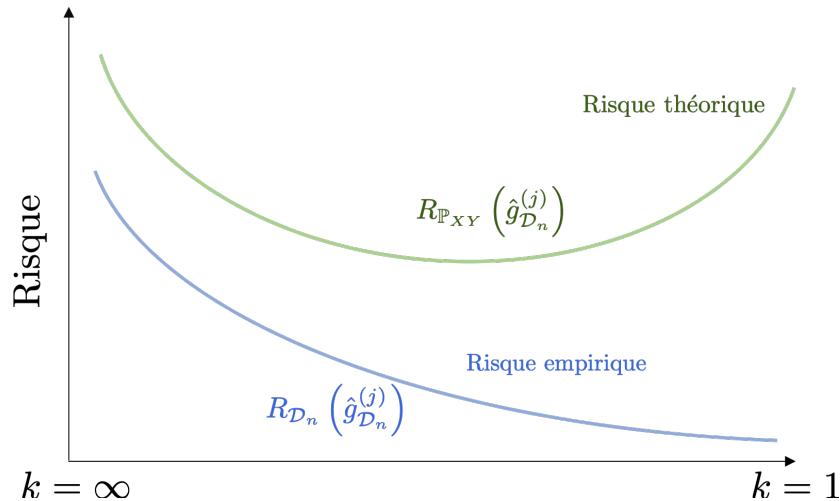
Regardons maintenant ce que le risque empirique fournit comme indication de qualité dans une situation pratique (qui vous arrivera tout le temps !). À partir de $\mathcal{D}_n$, vous construisez plusieurs prédicteurs possibles $\hat{g}_{\mathcal{D}_n}^{(1)}, \dots, \hat{g}_{\mathcal{D}_n}^{(m)}$. Vous voulez sélectionner le meilleur d'entre eux et vous calculez donc leur risque empirique $\hat{R}_{\mathcal{D}_n}(\hat{g}_{\mathcal{D}_n}^{(1)}), \dots, \hat{R}_{\mathcal{D}_n}(\hat{g}_{\mathcal{D}_n}^{(m)})$.

Regardons ce que cela donne quand on prend un prédicteur des k plus proches voisins, quand on change la valeur de k on a différents prédicteurs.



On peut faire un certain nombre de remarques sur ce dessin :

- Si on suit sans réfléchir la recommandation de ce genre de graphique, on choisirait $k = 1$ étant donné que c'est pour cette valeur que le risque empirique est le plus faible. Et vous feriez alors le même choix dans tous les cas pratiques car la courbe serait très similaire !
- Maintenant imaginons qu'en réalité on connaît $\mathbb{P}_{XY}$: on peut calculer le risque théorique $R_{\mathbb{P}_{XY}}(\hat{g}_{\mathcal{D}_n}^{(j)})$ de chaque prédicteur. Dans la figure ci-dessous on a rajouté ce risque en vert.



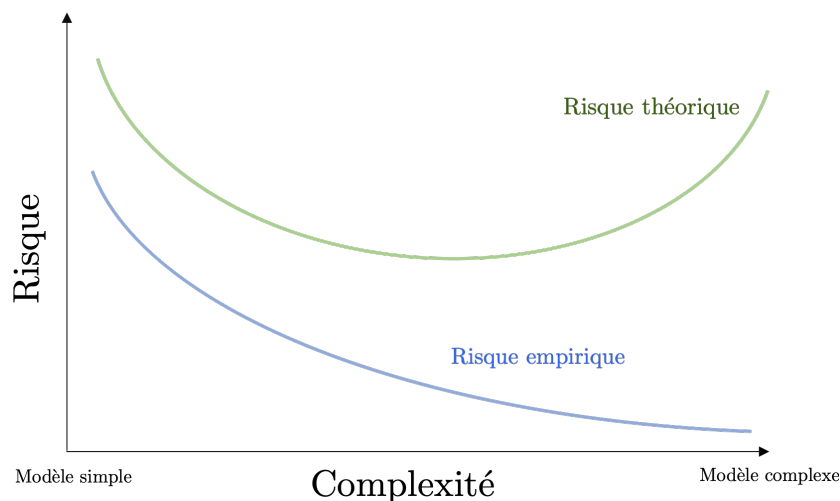
Et là votre choix n'est plus le même. D'où vient le problème ?

On a calculé le risque empirique

$$\hat{R}_{\mathcal{D}_n}(\hat{g}_{\mathcal{D}_n}^{(j)}),$$

c'est-à-dire que l'on a utilisé le **même échantillon pour construire le prédicteur et évaluer sa qualité**. En particulier, si on a par exemple un prédicteur qui est tel que $\hat{g}(x_i) = y_i$ et $\hat{g}(x) =$ n'importe quoi si $x \neq x_i$, alors son risque empirique est égal à zéro ! Évidemment, ça ne veut pas dire qu'un tel prédicteur est parfait, vu qu'il va certainement faire des erreurs gigantesques pour un x quelconque.

Ce que l'on a vu pour les plus proches voisins est en fait général avec l'axe des abscisses qui représente la **complexité** du prédicteur (penser aux degrés de liberté qu'on s'autorise pour apprendre sur les données). Pour les plus proches voisins le modèle va du moins complexe ($k \rightarrow \infty$) au plus complexe ($k = 1$), et de manière similaire pour la régression polynomiale (peu complexe pour degré 1 jusqu'à très complexe quand le degré $\rightarrow \infty$) ou les arbres de décision (peu complexe pour profondeur 1 jusqu'à très complexe quand la profondeur $\rightarrow \infty$).



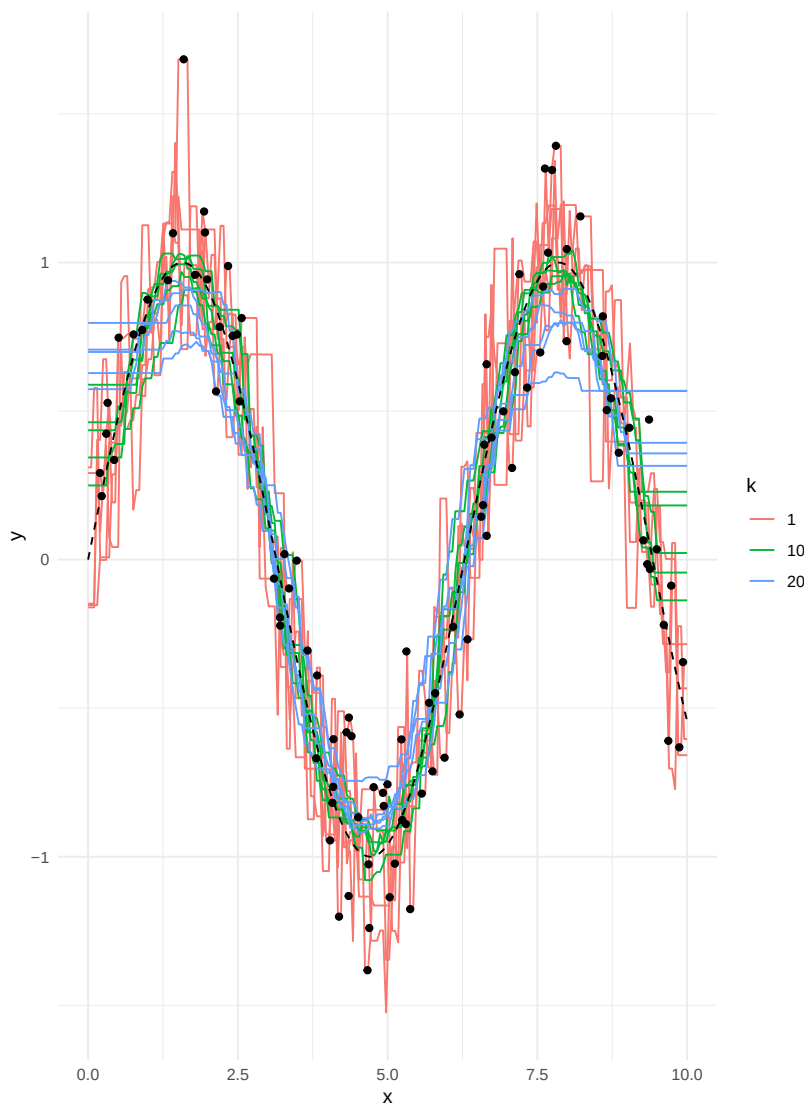
Quand le prédicteur est trop "simple", il va potentiellement mal représenter les données. On parle de **sous-apprentissage**, ou "underfitting" en anglais.

A l'inverse, quand le prédicteur est trop complexe, il va potentiellement trop vouloir bien représenter les données alors qu'elles ne sont qu'une réalisation aléatoire du vrai phénomène. On parle alors de **sur-apprentissage**, ou "overfitting" en anglais.

D'un point de vue inférence statistique, ces phénomènes sont reliés au fameux **compromis biais-variance** que vous avez étudié l'année dernière :

- Un prédicteur peu complexe sera plutôt stable, dans le sens où s'il est entraîné sur des échantillons d'apprentissage différents, il sera toujours à peu près le même (= faible variance). Mais il commet beaucoup d'erreurs et en moyenne il n'est pas un bon estimateur (= grand biais).
- A l'inverse, un prédicteur complexe sera en moyenne très bon (= faible biais), mais fournira des prédictions très variables s'il est entraîné plusieurs fois (= grande variance).

On peut illustrer une fois de plus ces phénomènes sur les plus proches voisins (en pointillé la vraie fonction), où on a répété l'estimation par plus proches voisins 5 fois avec des échantillons d'apprentissage différents (l'un d'entre eux est représenté avec les points noirs). On remarque clairement qu'avec $k = 1$ en rouge, en moyenne le prédicteur est sans biais, mais qu'il exhibe une grande variabilité. À l'inverse pour $k = 20$, le résultat est très stable mais on observe un biais d'estimation systématique. Enfin pour $k = 10$ on semble avoir un bon compromis entre ces deux phénomènes.



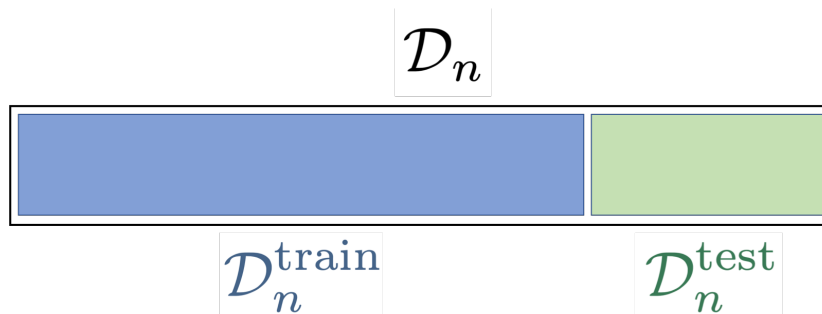
Revenons maintenant au problème du risque empirique. Ce que nous avons observé et commenté implique la règle suivante : **on ne peut pas évaluer la qualité, c'est-à-dire le risque d'un prédicteur sur un échantillon qui a servi à le construire.**

Dans la pratique, on recommande donc de couper ("split" en anglais), l'échan-

tillon initial en deux :

$$\mathcal{D}_n = \mathcal{D}_n^{\text{train}} \cup \mathcal{D}_n^{\text{test}}$$

où $\mathcal{D}_n^{\text{train}}$ est utilisé pour construire le prédicteur, et $\mathcal{D}_n^{\text{test}}$ pour évaluer sa qualité, c'est à dire le risque.



Mais ce n'est pas la fin de l'histoire... Pour construire notre prédicteur, on doit choisir usuellement des hyperparamètres (par exemple le nombre de voisins). Si on se sert de $\mathcal{D}_n^{\text{test}}$ pour choisir les hyperparamètres, cela veut donc dire que $\mathcal{D}_n^{\text{test}}$ a servi à la construction du prédicteur. Donc notre prédicteur construit au final a vu $\mathcal{D}_n^{\text{train}}$ et $\mathcal{D}_n^{\text{test}}$. Comment calculer sa qualité alors ? Il ne nous reste plus d'échantillons indépendants à disposition ! Pour résoudre ce problème, il y a deux voies :

- **Estimation par pénalisation.** C'est la méthode historique en statistique, basée sur l'idée que pour choisir un modèle, on va comparer le risque sur $\mathcal{D}_n^{\text{train}}$ de chacun avec une **pénalité** pour les modèles les plus complexes. On considère alors un critère du type :

$$\hat{R}_{\mathcal{D}_n^{\text{train}}}(\hat{g}_{\mathcal{D}_n^{\text{train}}}) + P(\hat{g}_{\mathcal{D}_n^{\text{train}}})$$

où P est une fonction de pénalisation dont la valeur augmente avec la complexité de $\hat{g}_{\mathcal{D}_n^{\text{train}}}$. Pour choisir le meilleur modèle, on va donc faire un compromis entre le risque empirique et la complexité. Vous verrez des exemples en régression linéaire (pénalisation AIC, BIC, CP de Mallows).

- **Estimation par découpage d'échantillons.** Encore ! C'est la méthode sur laquelle nous allons nous focaliser dans ce cours, qui fait l'objet de la section suivante.

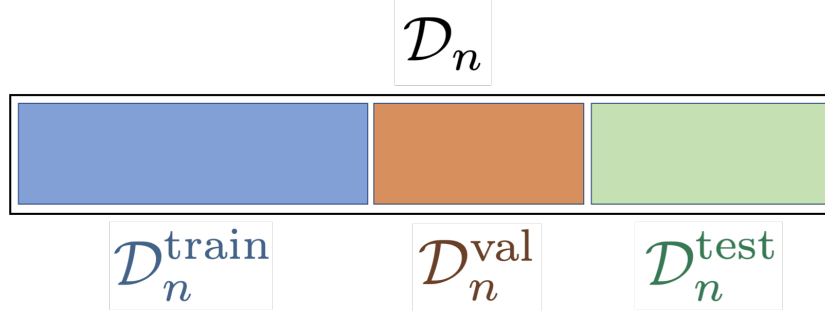
1.3 Validation simple, validation croisée, validation Monte-Carlo

Dans la classe des approches par découpage d'échantillons, il y a principalement trois techniques que l'on décrit dans les sous-sections suivantes.

1.3.1 La validation simple ("hold out")

Le principe est très simple, on recycle notre idée précédente utilisée pour $\mathcal{D}_n^{\text{test}}$, en créant un nouvel échantillon $\mathcal{D}_n^{\text{val}}$ pour "validation" où $\mathcal{D}_n^{\text{val}}$ est l'échantillon

sur lequel on évalue la qualité d'un prédicteur pour chaque valeur des hyperparamètres.



Algorithm 1: Validation simple

Pour une famille de prédicteurs donnée (e.g. knn).

Etape 1 : on entraîne le prédicteur sur $\mathcal{D}_n^{\text{train}}$ pour chaque valeur des hyperparamètres et on obtient la collection

$$\hat{g}_{\mathcal{D}_n^{\text{train}}}^{(1)}, \dots, \hat{g}_{\mathcal{D}_n^{\text{train}}}^{(m)}$$

Etape 2 : on calcule le risque de chacun d'entre eux sur l'échantillon de validation

$$\hat{R}_{\mathcal{D}_n^{\text{val}}}(\hat{g}_{\mathcal{D}_n^{\text{train}}}^{(1)}), \dots, \hat{R}_{\mathcal{D}_n^{\text{val}}}(\hat{g}_{\mathcal{D}_n^{\text{train}}}^{(m)})$$

Etape 3 : on choisit le meilleur d'entre eux

$$j^* = \arg \min_j \hat{R}_{\mathcal{D}_n^{\text{val}}}(\hat{g}_{\mathcal{D}_n^{\text{train}}}^{(j)})$$

que l'on note

$$\hat{g}^{(\text{final})} = \hat{g}_{\mathcal{D}_n^{\text{train}}}^{(j^*)}$$

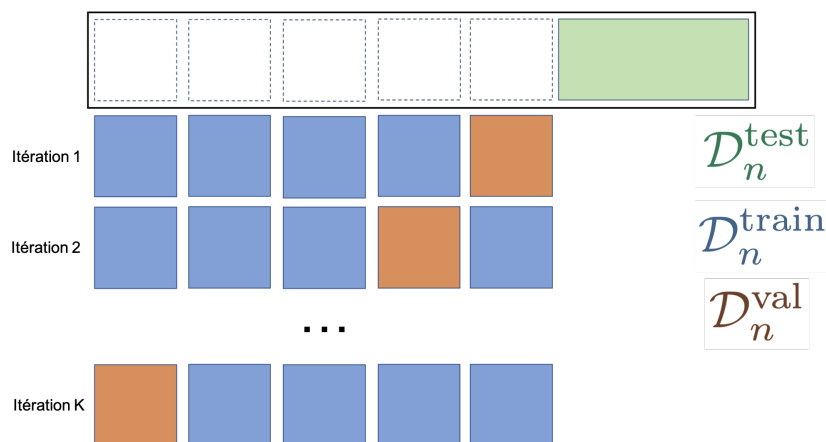
Etape 4 : la qualité du prédicteur final est donnée par son risque sur l'échantillon test

$$\hat{R}_{\mathcal{D}_n^{\text{test}}}(\hat{g}^{(\text{final})})$$

Le choix du meilleur prédicteur final à l'étape 3 peut être assez dépendant du découpage qu'on a fait pour construire $\mathcal{D}_n^{\text{val}}$: la solution la plus communément employée à l'heure actuelle est donc plutôt la validation croisée décrite ci-dessous.

1.3.2 La validation croisée ("cross-validation")

En gardant toujours $\mathcal{D}_n^{\text{test}}$ de côté, en validation-croisée on va répéter le découpage $\mathcal{D}_n^{\text{train}} / \mathcal{D}_n^{\text{val}}$ plusieurs fois. Si on découpe K fois, c'est la **validation-croisée K-blocs** ("K-folds" en anglais).

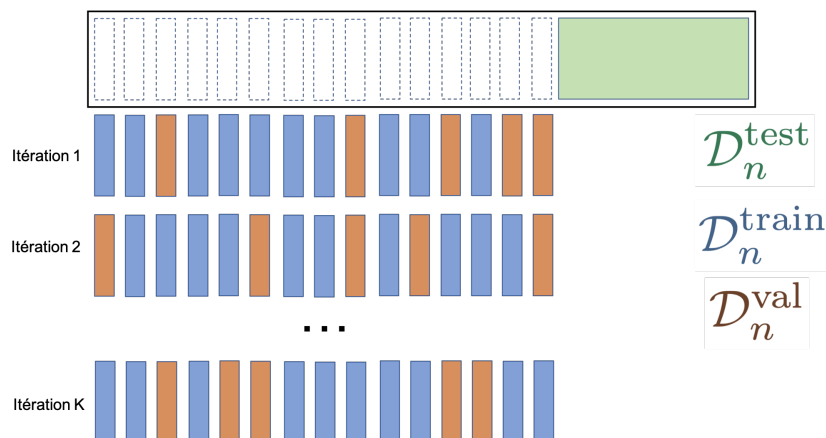


En pratique, dans l'algorithme précédent de la validation simple, on met les étapes 1 et 2 dans une boucle sur les K -blocs et à l'étape 3, on choisit le meilleur prédicteur (plutôt les meilleurs hyperparamètres) par rapport à la moyenne sur les K -blocs des risques sur chacun des $\mathcal{D}_n^{\text{val}}$. Usuellement, on choisit $K = 10$ ou $K = 5$. Le cas extrême, $K = |\mathcal{D}_n^{\text{train}}|$ s'appelle le "leave-one-out".

Avec cette procédure, le prédicteur dont on a identifié les hyperparamètres comme étant les meilleurs n'a été entraîné que sur un sous-ensemble des données, à cause de la procédure basée sur les K -blocs. En pratique on opère donc une dernière étape dite de **re-calibration** : le prédicteur final est obtenu en entraînant sur toutes les données (sauf l'ensemble de test évidemment) le modèle avec les hyperparamètres optimaux identifiés par validation croisée.

1.3.3 La validation croisée Monte-Carlo ou Bootstrap

L'inconvénient des K -blocs est que le même bloc d'observation ne change pas, c'est-à-dire qu'un paquet d'observations est toujours considéré ensemble. L'idée du Monte-Carlo est de "casser" ces paquets, avec un nombre de répétitions K assez grand, de l'ordre de la centaine :



L'approche Monte-Carlo permet d'éviter d'éventuels biais induits par les pa-

quets, mais cela casse l'équilibre de l'approche K-blocs car des données peuvent être utilisées plus pour l'apprentissage que pour la validation. Enfin, la validation croisée Bootstrap est similaire au Monte-Carlo, à la seule différence que les échantillons sont choisis avec remise au lieu de sans remise. Cela implique que certaines observations peuvent se retrouver à la fois dans $\mathcal{D}_n^{\text{train}}$ et $\mathcal{D}_n^{\text{val}}$. Cela induit un biais optimiste qui peut être corrigé, par exemple avec le "leave-one-out Bootstrap 0.632".

Comme pour la validation croisée, le prédicteur final doit être construit avec une étape de re-calibration.

1.4 Quelques remarques supplémentaires sur le découpage d'échantillons

- Pour une famille de modèles donnée (par exemple knn/polynômes/arbres/...), on utilise une des procédures de validation précédente pour choisir les meilleurs hyperparamètres. Pour chaque famille, on a donc un modèle "optimisé" dont on calcule la qualité via le risque sur l'échantillon $\mathcal{D}_n^{\text{test}}$. Votre choix final sera le **meilleur des meilleurs** modèles, en sélectionnant celui qui a la meilleure qualité sur l'échantillon $\mathcal{D}_n^{\text{test}}$.
- Si vous n'avez pas d'hyperparamètres à optimiser, il suffit de faire le seul découpage $\mathcal{D}_n^{\text{train}} / \mathcal{D}_n^{\text{test}}$, pas besoin de validation !
- Toujours dans le cas où il n'y a pas d'hyperparamètres mais que vous avez un jeu de données avec peu d'observations, même faire un seul découpage vous fait perdre potentiellement beaucoup d'informations. Une solution est d'utiliser une procédure de validation-croisée pour remplacer $\mathcal{D}_n^{\text{test}}$.
- Enfin, dans la situation la plus pénalisante où vous avez peu de données et que vous devez optimiser des hyperparamètres, une possibilité est de faire de la validation emboîtée ("nested validation").

Note : vous aborderez les aspects de validation sur le knn lors du TP1.

1.5 Conclusions et introduction au prochain cours

Si l'on revient au début de cette séance avec l'introduction du classifieur optimal de Bayes, **l'objet clef** sur lequel il est basé est la probabilité conditionnelle $\mathbb{P}(Y = 1 | \mathbf{X} = \mathbf{x})$. On a dit précédemment qu'on ne pouvait la calculer que si on connaissait explicitement la loi jointe $\mathbb{P}_{\mathbf{X}Y}$.

De manière similaire, en régression, on peut aussi démontrer que pour la fonction de perte quadratique $l(y, y') = (y - y')^2$, le meilleur prédicteur est donné par le théorème suivant.

Théorème 2. Le prédicteur f qui minimise le risque $R_{\mathbb{P}_{\mathbf{X}Y}}(f)$ pour la fonc-

tion de perte quadratique est

$$f^*(\mathbf{x}) = \mathbb{E}(Y|\mathbf{X} = \mathbf{x}).$$

Preuve. On écrit le risque

$$\begin{aligned} R_{\mathbb{P}_{\mathbf{X}Y}}(f) &= \mathbb{E}_{\mathbf{X}Y}[(Y - f(\mathbf{X}))^2] \\ &= \mathbb{E}_{\mathbf{X}}[\mathbb{E}[(Y - f(\mathbf{X}))^2|\mathbf{X}]] \\ &= \int \mathbb{E}[(Y - f(\mathbf{X}))^2|\mathbf{X} = \mathbf{x}]d\mathbb{P}_{\mathbf{X}}(\mathbf{x}). \end{aligned}$$

Pour minimiser $R_{\mathbb{P}_{\mathbf{X}Y}}(f)$, il suffit de trouver f telle que l'intégrande est minimisée pour chaque $\mathbf{x}$. Pour $\mathbf{x}$ fixé, en notant $a = f(\mathbf{x})$ la valeur que l'on cherche, on doit donc résoudre le problème de minimisation

$$\arg \min_a \mathbb{E}[(Y - a)^2|\mathbf{X} = \mathbf{x}].$$

Une approche consiste à dériver par rapport à a , et trouver la valeur de a qui annule la dérivée. Mais si vous vous souvenez de votre cours de première année, vous connaissez le résultat : $a^* = \mathbb{E}(Y|\mathbf{X} = \mathbf{x})$! $\square$

L'objet clef en régression est donc l'espérance conditionnelle $\mathbb{E}(Y|\mathbf{X} = \mathbf{x})$, qu'on ne peut calculer que si on connaît la loi jointe $\mathbb{P}_{\mathbf{X}Y}$, une fois de plus.

Nous verrons par la suite que beaucoup d'algorithmes d'apprentissage supervisés cherchent en réalité à construire une approximation des objets clefs $\mathbb{P}(Y = 1|\mathbf{X} = \mathbf{x})$ ou $\mathbb{E}(Y|\mathbf{X} = \mathbf{x})$ à partir de l'échantillon. La différence entre ces algorithmes repose essentiellement sur les hypothèses que l'on fait pour construire cette approximation. Discussion lors du prochain cours ...

Séance 3 : Mes premiers modèles d'apprentissage supervisé

19 Septembre, 2023

- Retour sur le classifieur de Bayes et approximation
 - Le classifieur de Bayes naïf
 - L'analyse discriminante linéaire et quadratique
- La méthode des plus proches voisins
 - Formalisation
 - Choix de la distance
 - Extension vers le lissage par noyau

Chapitre 2

Quelques modèles d'apprentissage supervisé

Nous avons vu au chapitre précédent que pour construire un prédicteur, construire une approximation de $\mathbb{P}(Y = 1|\mathbf{X} = \mathbf{x})$ en classification et de $\mathbb{E}(Y|\mathbf{X} = \mathbf{x})$ en régression est une voie prometteuse. Dans cette partie, nous introduisons quelques méthodes d'apprentissage supervisé simples qui suivent ce chemin. Il est cependant important de garder à l'esprit que bien qu'elle soient simples, ces méthodes peuvent avoir de manière surprenantes de bons résultats sur les cas pratiques. Il ne faudra donc pas les négliger, et il est conseillé de toujours les utiliser en premier lieu dans une étude comme **méthodes de référence**.

2.1 Variations autour du classifieur de Bayes

On va s'intéresser en premier lieu aux problèmes de classification binaire, pour lesquels on rappelle ci-dessous le classifieur optimal.

Théorème 3 (Classifieur de Bayes). Le classifieur h qui minimise le risque $R_{\mathbb{P}_{\mathbf{X}Y}}(h)$ pour la fonction de perte 0-1 est

$$h^*(\mathbf{x}) = \begin{cases} 1 & \text{si } \eta(\mathbf{x}) > \frac{1}{2} \\ 0 & \text{sinon} \end{cases}$$

où $\eta(\mathbf{x}) = \mathbb{P}(Y = 1|\mathbf{X} = \mathbf{x})$.

Ce classifieur est le **classifieur de Bayes** et son risque $R_{\mathbb{P}_{\mathbf{X}Y}}(h^*)$ est le **risque de Bayes** $R_{\mathbb{P}_{\mathbf{X}Y}}^*$. Autrement dit, on a

$$\forall h, R_{\mathbb{P}_{\mathbf{X}Y}}^* = R_{\mathbb{P}_{\mathbf{X}Y}}(h^*) \leq R_{\mathbb{P}_{\mathbf{X}Y}}(h)$$

Nous allons voir deux méthodes d'approximation de la probabilité conditionnelle qui intervient dans ce classifieur optimal, basée sur la même intuition initiale. En notant les catégories prises par la variable cible $\{C_1, \dots, C_M\}$, cette

intuition repose sur la formule de Bayes, qui donne pour tout $l = 1, \dots, M$:

$$\mathbb{P}(Y = C_l | \mathbf{X} = \mathbf{x}) = \mathbb{P}(\mathbf{X} = \mathbf{x} | Y = C_l) \mathbb{P}(Y = C_l) / \mathbb{P}(\mathbf{X} = \mathbf{x})$$

dans le cas où toutes les variables explicatives sont des variables aléatoires discrètes, ou

$$\mathbb{P}(Y = C_l | \mathbf{X} = \mathbf{x}) = f_{\mathbf{X}|Y=C_l}(\mathbf{x}) \mathbb{P}(Y = C_l) / f_{\mathbf{X}}(\mathbf{x})$$

si elles sont toutes continues avec $f_{\mathbf{X}}$ leur densité de probabilité. Par souci de concision nous n'écrivons pas la formule générale dans le cas où les types de variables explicatives sont mélangés mais la généralisation est triviale. Le classifieur de Bayes consiste à prédire la classe qui a la plus grande probabilité conditionnelle pour une valeur de $\mathbf{x}$ donnée, ce qui implique que pour construire notre approximation nous pouvons seulement nous concentrer sur le numérateur de la formule de Bayes, puisque le dénominateur est le même pour toutes les catégories pour $\mathbf{x}$ fixé. On écrira donc par exemple

$$\mathbb{P}(Y = C_l | \mathbf{X} = \mathbf{x}) \propto \mathbb{P}(\mathbf{X} = \mathbf{x} | Y = C_l) \mathbb{P}(Y = C_l) \quad (2.1)$$

et

$$\mathbb{P}(Y = C_l | \mathbf{X} = \mathbf{x}) \propto f_{\mathbf{X}|Y=C_l}(\mathbf{x}) \mathbb{P}(Y = C_l). \quad (2.2)$$

La première brique d'approximation commune (dans les deux méthodes d'approximations décrites ci-dessous, et que les variables explicatives soient discrètes ou continues) consiste à remplacer $\mathbb{P}(Y = C_l)$ par

$$\hat{\mathbb{P}}(Y = C_l) := \hat{\pi}_{C_l} = \frac{1}{n} \sum_{i=1}^n \mathbb{1}_{y_i=C_l},$$

c'est-à-dire la fréquence empirique d'apparition de la catégorie C_l dans l'échantillon d'entraînement. Les deux estimateurs que nous décrivons maintenant diffèrent seulement sur les hypothèses qu'ils font pour approcher le premier terme $\mathbb{P}(\mathbf{X} = \mathbf{x} | Y = C_l)$ ou $f_{\mathbf{X}|Y=C_l}(\mathbf{x})$ du produit dans les équations (2.1) ou (2.2).

2.1.1 Le classifieur de Bayes naïf

Le **classifieur de Bayes naïf** part du constat que l'approximation des termes $\mathbb{P}(\mathbf{X} = \mathbf{x} | Y = C_l)$ ou $f_{\mathbf{X}|Y=C_l}(\mathbf{x})$ consiste en l'estimation d'une probabilité en dimension d , le nombre de variables explicatives. Comme ce problème souffre du fléau de la dimension, la simplification principale faite par le Bayésien naïf est que **les variables explicatives sont indépendantes à l'intérieur de chacune des catégories (hypothèse)**. L'approximation obtenue sous cette hypothèse est donc donnée sous la forme d'un produit :

$$\hat{\mathbb{P}}(\mathbf{X} = \mathbf{x} | Y = C_l) = \prod_{j=1}^d \hat{\mathbb{P}}(X^{(j)} = x^{(j)} | Y = C_l)$$

ou

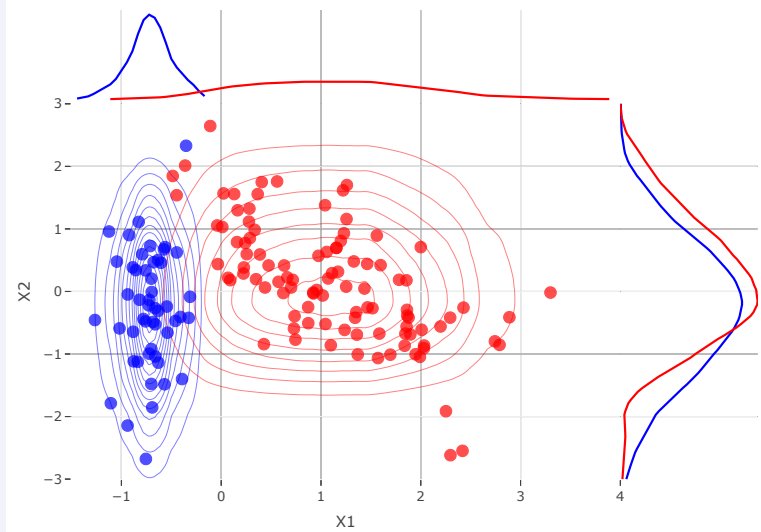
$$\hat{f}_{\mathbf{X}|Y=C_l}(\mathbf{x}) = \prod_{j=1}^d \hat{f}_{X^{(j)}|Y=C_l}(x^{(j)}).$$

Enfin, pour chaque variable explicative, la dernière étape est d'estimer chacune de ces probabilités uni-dimensionnelles :

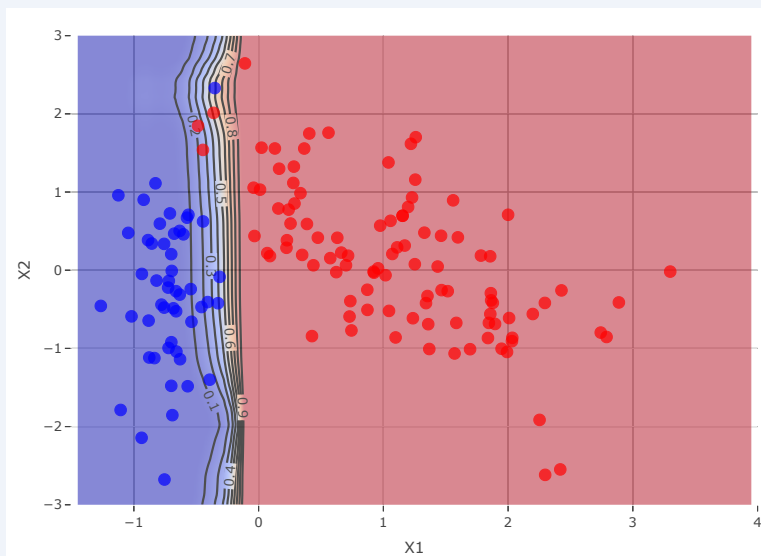
- Pour $\hat{\mathbb{P}}(X^{(j)} = x^{(j)} | Y = C_l)$, on prendra par exemple la fréquence empirique de chaque valeur discrète de la variable $X^{(j)}$ quand on regarde uniquement les individus qui correspondent à $y_i = C_l$. Cependant si l'échantillon complet est de petite taille et/ou certaines des valeurs discrètes prises par $X^{(j)}$ sont rares, il y a un risque qu'une des fréquences empiriques soit nulle, ce qui conduira à une approximation identiquement nulle pour $\hat{\mathbb{P}}(\mathbf{X} = \mathbf{x} | Y = C_l)$ à cause du produit. Il est alors usuel de considérer une variante plus robuste de la fréquence empirique connue sous le terme de *correction de Laplace*, qui consiste simplement à ajouter par défaut un entier positif fixe dans le comptage de chaque valeur discrète avant de calculer la fréquence empirique. La valeur de cet entier, qui devient un hyperparamètre de la méthode, peut être estimé par validation croisée.
- Pour $\hat{f}_{X^{(j)}|Y=C_l}(x^{(j)})$, on commence par assembler l'échantillon $\{x_{C_l}^{(j)}\}$ qui contient toutes les valeurs de la variable explicative $X^{(j)}$ quand on regarde uniquement les individus qui correspondent à $y_i = C_l$. Cet échantillon sert à construire soit une estimation de densité par noyau, soit une estimation paramétrique en supposant que c'est une loi Gaussienne dont la moyenne et la variance sont estimées par maximum de vraisemblance. Si vous souhaitez sélectionner automatiquement la fenêtre du noyau pour l'estimation de densité, ou encore choisir automatiquement entre estimation par noyau ou hypothèse Gaussienne, cela vous place *de facto* dans le cadre de la sélection d'un hyperparamètre, ce qui vous amènera une fois de plus à utiliser une procédure de validation croisée.

Le cas des variables explicatives continues est illustré dans l'exemple ci-dessous.

Exemple (Illustration du classifieur de Bayes naïf). On considère un problème de classification binaire avec deux variables explicatives X_1 et X_2 continues, pour lequel on dispose d'un jeu de données de 150 individus. Pour chaque classe (0 en bleu et 1 en rouge ci-dessous), on estime la densité de chaque variable explicative par noyau. Ces estimations sont représentées sur la figure ci-dessous au-dessus et à droite. Au centre, on représente l'ensemble des individus du jeu de données dans l'espace des variables explicatives, colorés selon leur classe. On y superpose les lignes de niveau des densités jointes des variables explicatives dans chaque classe (qui dans le Bayésien naïf sont obtenues en faisant le produit des densités marginales estimées précédemment, étant donné l'hypothèse d'indépendance).



A l'aide de ces estimations de densité, on en déduit alors l'approximation $\hat{\mathbb{P}}(Y = 1 | \mathbf{X} = \mathbf{x})$ pour toute valeur de $\mathbf{x}$, dont on représente les lignes de niveau.



On remarquera que c'est un cas de classification très facile, car les deux classes sont très bien séparées et le classifieur de Bayes naïf possède une frontière de décision bien nette. Bien entendu cet exemple n'est pas représentatif de la difficulté usuelle de la classification en général, mais il permet de bien représenter l'estimation des densités.

Remarque (Essayer de rendre les variables explicatives indépendantes?). Pour les cas où l'hypothèse d'indépendance semble être fausse sur un jeu de données, vous pourriez penser qu'avant d'appliquer le classifieur de Bayes naïf il serait intéressant d'essayer de rendre les variables explicatives indépendantes, ou au moins décorréliées. Pour ce faire, l'ACP est une candidate naturelle. En pratique, cela vous amènerait donc à faire une ACP préliminaire de vos données, séparément pour chaque classe, et en déduire des nouvelles variables décorréliées pour chaque classe (en effet l'hypothèse d'indépendance n'est pas globale, mais au sein d'une catégorie). Enfin vous estimez les lois marginales de ces nouvelles variables pour chaque classe. Cette procédure pourrait fonctionner de manière efficace sur l'échantillon d'entraînement. Mais le point bloquant apparaît à la phase de prédiction ! Si vous devez prédire un nouvel individu à l'aide de ses variables explicatives, pour calculer la prédiction vous devez transformer les variables explicatives à l'aide d'une projection pour les ramener dans l'espace des nouvelles variables identifiées par l'ACP. Mais vous ne savez pas quelle projection il faut appliquer, car ce choix dépend de la classe de l'individu (puisque vous avez fait une ACP différente par classe), c'est-à-dire de la cible que vous devez prédire. C'est donc une impasse.

2.1.2 L'analyse discriminante linéaire ou quadratique

Au lieu de faire une hypothèse d'indépendance des variables explicatives par catégorie, l'**analyse discriminante linéaire ou quadratique** propose à la place de supposer que, **par catégorie, les variables explicatives suivent une loi normale multivariée**. Cette hypothèse :

- ne peut être faite que pour les problèmes où les variables explicatives sont continues, évidemment,
- n'est ni plus forte ni moins forte que la précédente, seulement de nature différente. Elle permet de modéliser de la dépendance entre les variables explicatives, mais laisse moins de flexibilité sur les distributions marginales de chacune.

Sous cette hypothèse, $f_{\mathbf{X}|Y=C_l}(\mathbf{x})$ est alors estimée par

$$\hat{f}_{\mathbf{X}|Y=C_l}(\mathbf{x}) = \mathcal{N}_{\mathbf{x}}\left(\hat{\mu}_k, \hat{\Sigma}_k\right)$$

où $\hat{\mu}_k$ et $\hat{\Sigma}_k$ sont le vecteur moyenne et la matrice de covariance estimés des variables explicatives pour la catégorie C_l . L'estimateur $\hat{\mu}_k$ est obtenu directement en calculant la moyenne empirique sur l'échantillon $\{\mathbf{x}_{C_l}\}$ qui contient toutes les valeurs des variables explicatives quand on regarde uniquement les individus qui correspondent à $y_i = C_l$. Pour la matrice de covariance, deux chemins sont possibles :

1. L'**analyse discriminante linéaire** (*Linear Discriminant Analysis* en anglais, ou **LDA**) fait l'hypothèse supplémentaire que la matrice de covariance est la même pour toutes les catégories, i.e. $\forall k = 1, \dots, M, \hat{\Sigma}_k = \hat{\Sigma}$.

Pour construire l'estimateur $\hat{\Sigma}$, il suffit de calculer la covariance empirique à partir des variables explicatives de tous les individus **après les avoir toutes recentrées sur leur moyenne empirique par catégorie** (c'est-à-dire $\hat{\mu}_k$). Ceci est équivalent à faire une combinaison linéaire pondérée des covariances empiriques dans chaque catégorie, où les poids sont donnés par la proportion de chaque catégorie.

2. L'**analyse discriminante quadratique** (*Quadratic Discriminant Analysis* en anglais, ou **QDA**) ne fait pas cette hypothèse, et considère des estimateurs $\hat{\Sigma}_k$ égaux à la covariance empirique des variables explicatives dans chaque catégorie.

Une question légitime que l'on peut se poser est naturellement : pourquoi dans le premier cas parle-t-on d'analyse discriminante *linéaire*, et dans l'autre *quadratique* ? La réponse est simple, mais nécessite de poursuivre jusqu'à la fin la construction du classifieur après avoir fait ces hypothèses.

Plaçons nous dans le cadre de la classification binaire où $Y \in \{0, 1\}$, l'analyse discriminante linéaire fournit alors les approximations suivantes des probabilités conditionnelles de la sortie :

$$\begin{aligned}\hat{\mathbb{P}}(Y = 0 | \mathbf{X} = \mathbf{x}) &\propto \mathcal{N}_{\mathbf{x}}(\hat{\mu}_0, \hat{\Sigma}) \hat{\pi}_0 \\ \hat{\mathbb{P}}(Y = 1 | \mathbf{X} = \mathbf{x}) &\propto \mathcal{N}_{\mathbf{x}}(\hat{\mu}_1, \hat{\Sigma}) \hat{\pi}_1.\end{aligned}$$

La frontière de décision du classifieur final correspond aux valeurs de $\mathbf{x}$ telles que $\hat{\mathbb{P}}(Y = 1 | \mathbf{X} = \mathbf{x}) = 1/2$, où de manière équivalente $\hat{\mathbb{P}}(Y = 1 | \mathbf{X} = \mathbf{x}) = \hat{\mathbb{P}}(Y = 0 | \mathbf{X} = \mathbf{x})$, soit :

$$\begin{aligned}\mathcal{N}_{\mathbf{x}}(\hat{\mu}_1, \hat{\Sigma}) \hat{\pi}_1 &= \mathcal{N}_{\mathbf{x}}(\hat{\mu}_0, \hat{\Sigma}) \hat{\pi}_0 \\ \Leftrightarrow -\frac{1}{2}(\mathbf{x} - \hat{\mu}_1)^T \hat{\Sigma}^{-1}(\mathbf{x} - \hat{\mu}_1) + \log \hat{\pi}_1 &= -\frac{1}{2}(\mathbf{x} - \hat{\mu}_0)^T \hat{\Sigma}^{-1}(\mathbf{x} - \hat{\mu}_0) + \log \hat{\pi}_0 \\ \Leftrightarrow \mathbf{x}^T \hat{\Sigma}^{-1}(\hat{\mu}_1 - \hat{\mu}_0) - \frac{1}{2} \hat{\mu}_1^T \hat{\Sigma}^{-1} \hat{\mu}_1 + \frac{1}{2} \hat{\mu}_0^T \hat{\Sigma}^{-1} \hat{\mu}_0 + \log \frac{\hat{\pi}_1}{\hat{\pi}_0} &= 0.\end{aligned}$$

La dernière équation est de la forme $\mathbf{x}^T \mathbf{w} + b = 0$, qui est l'équation d'un hyperplan : la frontière de décision est donc linéaire, d'où la dénomination de la méthode. Ce résultat se généralise à la classification multi-classes, où la frontière de décision est définie par l'intersection de plusieurs hyperplans.

De manière similaire, on peut écrire l'équation satisfaite par la frontière de décision pour l'analyse discriminante quadratique :

$$\begin{aligned}\mathcal{N}_{\mathbf{x}}(\hat{\mu}_1, \hat{\Sigma}_1) \hat{\pi}_1 &= \mathcal{N}_{\mathbf{x}}(\hat{\mu}_0, \hat{\Sigma}_0) \hat{\pi}_0 \\ \Leftrightarrow -\frac{1}{2}(\mathbf{x} - \hat{\mu}_1)^T \hat{\Sigma}_1^{-1}(\mathbf{x} - \hat{\mu}_1) + \log \hat{\pi}_1 &= -\frac{1}{2}(\mathbf{x} - \hat{\mu}_0)^T \hat{\Sigma}_0^{-1}(\mathbf{x} - \hat{\mu}_0) + \log \hat{\pi}_0 \\ \Leftrightarrow -\frac{1}{2} \mathbf{x}^T (\hat{\Sigma}_1^{-1} - \hat{\Sigma}_0^{-1}) \mathbf{x} + \mathbf{x}^T (\hat{\Sigma}_1^{-1} \hat{\mu}_1 - \hat{\Sigma}_0^{-1} \hat{\mu}_0) & \\ -\frac{1}{2} \hat{\mu}_1^T \hat{\Sigma}_1^{-1} \hat{\mu}_1 + \frac{1}{2} \hat{\mu}_0^T \hat{\Sigma}_0^{-1} \hat{\mu}_0 + \log \frac{\hat{\pi}_1}{\hat{\pi}_0} &= 0\end{aligned}$$

qui est cette fois-ci de la forme $\mathbf{x}^T A \mathbf{x} + \mathbf{x}^T \mathbf{w} + b = 0$, c'est-à-dire l'équation d'une surface quadratique. Dans la dernière égalité, il apparaît clairement que si

les matrices de covariance sont supposées égales pour les différentes catégories, le terme quadratique disparaît et l'on retrouve l'analyse discriminante linéaire.

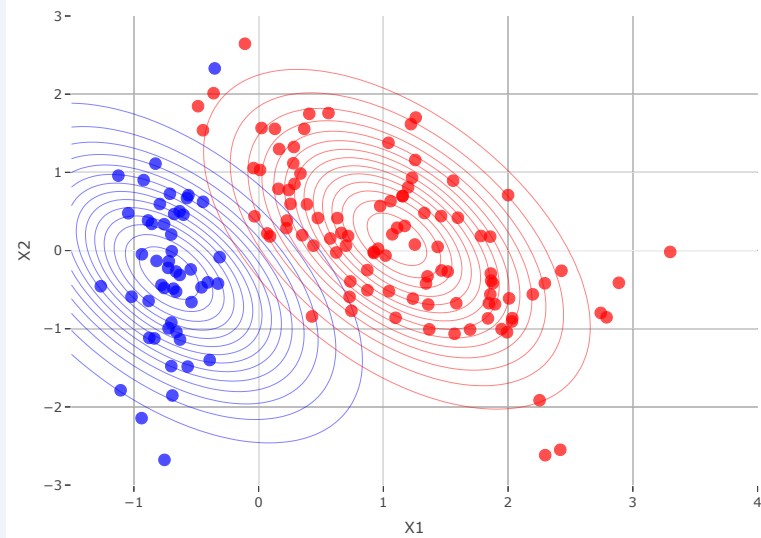
Remarque (Ecole anglo-saxonne vs française). Pour introduire l'analyse discriminante ici, nous avons adopté la présentation répandue plutôt dans l'école anglo-saxonne, qui correspond en théorie à l'analyse discriminante dite *Bayésienne*. Ce point de vue permet de faire un parallèle pédagogique entre LDA et classifieur de Bayes naïf, qui ont exactement la même forme mais diffèrent seulement sur l'hypothèse faite pour approcher les lois des variables explicatives dans chaque catégorie.

Dans l'école française, et dans les livres de référence que vous pourrez trouver à la bibliothèque, la présentation de la LDA suit plutôt la proposition originelle de Fisher, proche des idées de l'analyse en composante principale : l'idée est de chercher un axe factoriel dit discriminant, qui explique le mieux l'éloignement des classes dans l'espace des variables explicatives, cet éloignement étant mesuré par la matrice de covariance des barycentres de chaque classe. L'hypothèse de Gaussianité n'est donc pas faite explicitement dans ce cadre, mais les équations finales du classifieur sont exactement les mêmes.

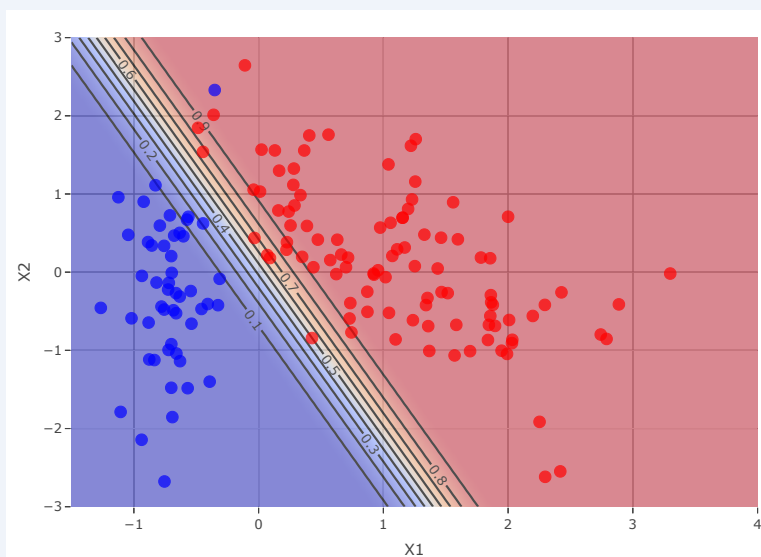
Remarque (Lien additionnel entre QDA et classifieur de Bayes naïf). Dans un cas particulier de QDA où l'on ferait l'hypothèse supplémentaire que les matrices de covariance pour chaque catégorie sont **diagonales**, le classifieur QDA serait alors strictement le même que le classifieur de Bayes naïf avec des lois marginales supposées être Gaussiennes (cette correspondance n'a pas lieu si elles sont estimées par noyau).

Nous illustrons maintenant la LDA et la QDA sur le même exemple que précédemment.

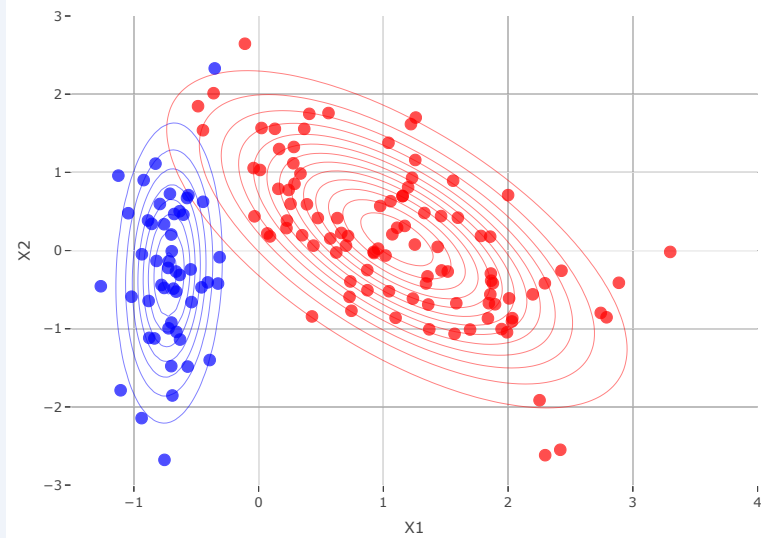
Exemple (Illustration de la LDA et QDA). On commence par représenter l'estimation des lois jointes pour chaque catégorie produites par la LDA.



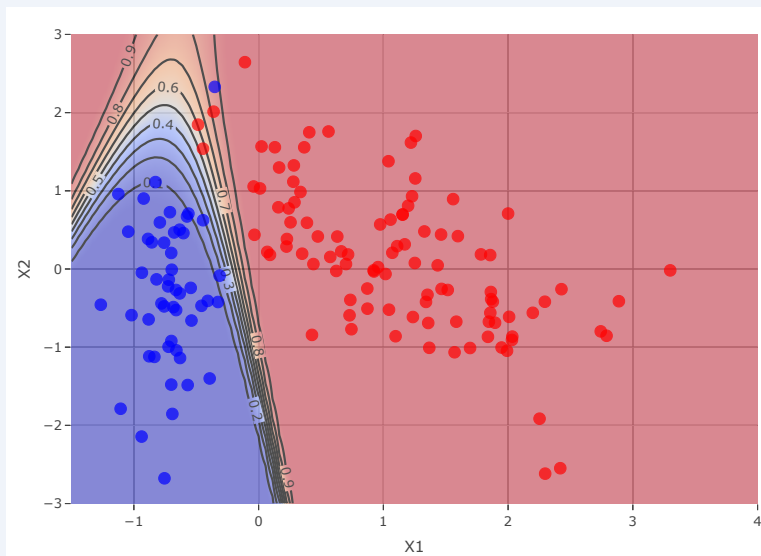
Comme attendu, vous reconnaissez les lignes de niveau d'une loi Gaussienne (ellipses), et au centrage près les deux lois sont les mêmes car la LDA suppose que les covariances sont identiques. A l'aide de ces estimations de densité, on en déduit alors l'approximation $\hat{\mathbb{P}}(Y = 1 | \mathbf{X} = \mathbf{x})$ pour toute valeur de $\mathbf{x}$, dont on représente les lignes de niveau.



On retrouve bien que les lignes de niveau sont des droites. On fait de même pour la QDA.



Cette fois-ci les Gaussiennes n'ont pas la même orientation car leur matrice de covariance est estimée séparément dans chaque classe par la QDA. À l'aide de ces estimations de densité, on en déduit alors l'approximation $\hat{\mathbb{P}}(Y = 1 | \mathbf{X} = \mathbf{x})$ pour toute valeur de $\mathbf{x}$, dont on représente les lignes de niveau.



Maintenant le comportement est différent, à cause des lignes de niveau qui sont quadratiques.

2.2 Méthode des plus proches voisins

Nous avons déjà introduit brièvement le prédicteur des plus proches voisins (*k-nearest neighbors* en anglais, ou **KNN**) lors du premier cours, et vous avez pu le manipuler en travaux pratiques pour voir l'effet de son unique hyperparamètre, le nombre de plus proches voisins. Ici pour aller plus loin, nous le formalisons mathématiquement et discutons de ses possibles interprétations.

2.2.1 Principe général

Nous commençons par donner la définition du prédicteur des plus proches voisins.

Définition 4 (KNN en classification). Pour une distance d donnée dans l'espace des variables explicatives $\mathcal{X}$ et un entier fixé $k > 0$, le **classifieur des plus proches voisins** au point $\mathbf{x}$ est donné par

$$\hat{h}_{\text{knn}}(\mathbf{x}) = C_{l^*}$$

où

$$l^* = \arg \max_{l=1,\dots,M} \frac{1}{k} \sum_{i \in N_k(\mathbf{x})} \mathbb{1}_{y_i = C_l}$$

et $N_k(\mathbf{x})$ est l'ensemble des indices des individus qui sont parmi les k plus proches voisins de $\mathbf{x}$ au sens de la distance d . Autrement dit, il prédit pour $\mathbf{x}$ la catégorie dont la fréquence empirique est la plus élevée parmi les k plus proches voisins de $\mathbf{x}$ (on parle parfois de *vote à la majorité*).

Définition 5 (KNN en régression). Pour une distance d donnée dans l'espace des variables explicatives $\mathcal{X}$ et un entier fixé $k > 0$, le **régresseur des plus proches voisins** au point $\mathbf{x}$ est donné par

$$\hat{f}_{\text{knn}}(\mathbf{x}) = \frac{1}{k} \sum_{i \in N_k(\mathbf{x})} y_i$$

où $N_k(\mathbf{x})$ est l'ensemble des indices des individus qui sont parmi les k plus proches voisins de $\mathbf{x}$ au sens de la distance d . Autrement dit, il prédit pour $\mathbf{x}$ la moyenne des sorties associées aux k plus proches voisins de $\mathbf{x}$.

Remarque (La distance d). Au delà du choix du nombre de plus proches voisins k que vous pouvez sélectionner automatiquement par validation croisée, le prédicteur KNN est particulièrement dépendant du choix de la distance d . Nous reviendrons sur ce point plus tard dans cette section.

Intuitivement, le principe de prendre en compte les plus proches voisins pour faire une prédiction correspond à l'idée sous-jacente qu'il existe une certaine ré-

gularité pour la relation entre $\mathbf{x}$ et y : quand des valeurs de $\mathbf{x}$ sont proches au sens de la distance d , alors les sorties correspondantes doivent être proches. C'est le point de vue que nous avons évoqué lors du premier cours. Mais on peut également l'interpréter comme un estimateur un peu naïf (en apparence seulement) de $\mathbb{P}(Y = 1 | \mathbf{X} = \mathbf{x})$ en classification et de $\mathbb{E}(Y | \mathbf{X} = \mathbf{x})$ en régression.

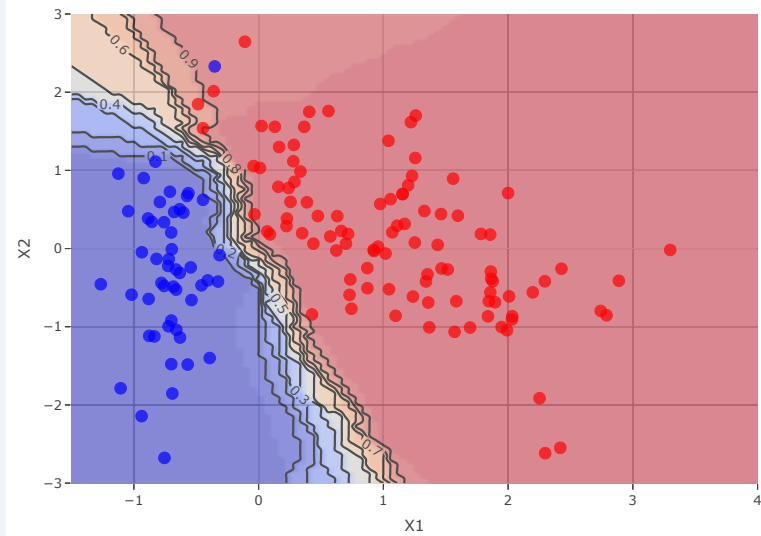
Imaginons la situation idéale (complètement illusoire en pratique) où l'on disposerait, pour un $\mathbf{x}$ que l'on doit prédire, de m réalisations $\{y_i^{\mathbf{x}}\}_{i=1,\dots,m}$ de la variable aléatoire $Y | \mathbf{X} = \mathbf{x}$. Il est évident que l'on pourrait alors utiliser cet échantillon pour construire des estimateurs empiriques de Monte-Carlo des quantités qui nous intéressent en prédiction :

$$\begin{aligned}\hat{\mathbb{P}}(Y = 1 | \mathbf{X} = \mathbf{x}) &= \frac{1}{m} \sum_{i=1}^m \mathbb{1}_{y_i^{\mathbf{x}}=1}, \\ \hat{\mathbb{E}}(Y | \mathbf{X} = \mathbf{x}) &= \frac{1}{m} \sum_{i=1}^m y_i^{\mathbf{x}}.\end{aligned}$$

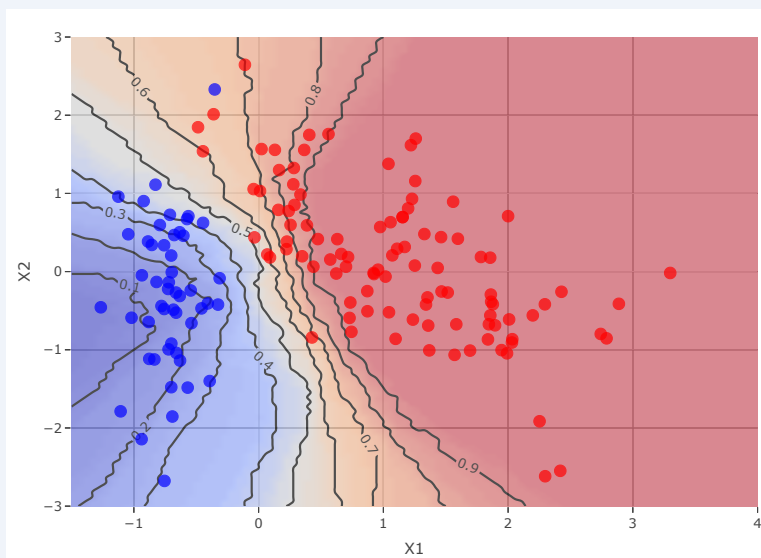
Mais bien évidemment en pratique un tel échantillon n'est pas disponible (si c'était le cas vous n'auriez même pas besoin de l'échantillon d'apprentissage $\mathcal{D}_n$!). Le prédicteur KNN peut dans ce cas être interprété comme une méthode qui suppose que la distribution de l'échantillon $\{y_i\}_{i \in N_k(\mathbf{x})}$ est proche de celle de $\{y_i^{\mathbf{x}}\}_{i=1,\dots,m}$ (si $\mathbf{x}_i$ est proche de $\mathbf{x}$, alors la valeur y_i aurait pu être générée par la loi de $Y | \mathbf{X} = \mathbf{x}$), et que l'on peut donc remplacer les estimateurs empiriques ci-dessus en sommant sur les plus proches voisins de $\mathbf{x}$ à la place.

Illustrons maintenant les plus proches voisins sur l'exemple précédent de classification.

Exemple (Illustration du KNN en classification). Sur l'exemple précédent, on obtient les lignes de niveaux suivantes pour le prédicteur KNN avec $k = 10$:



et pour $k = 50$:

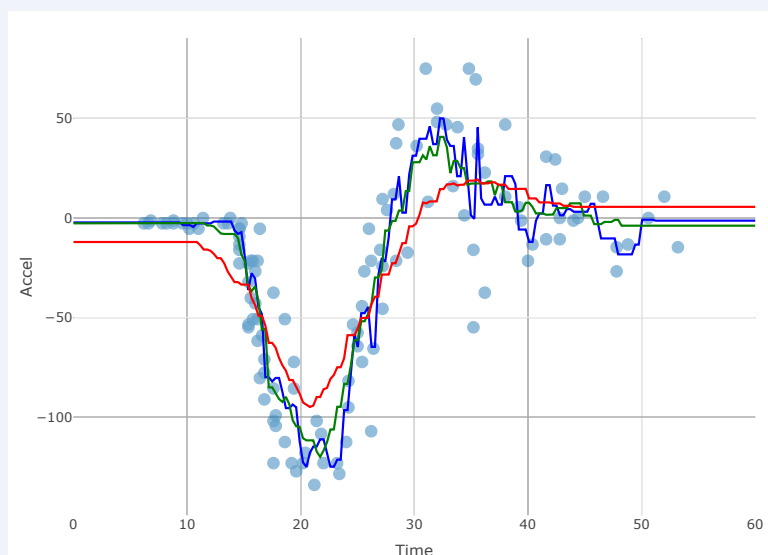


Si ici la frontière de décision qui correspond au niveau 0.5 est similaire dans les deux cas, la confiance qu'accorde le KNN à ses prédictions évolue selon k .

On peut également l'illustrer sur un problème de régression.

Exemple (Illustration du KNN en régression). On utilise le jeu de données `mcycle` du package **MASS** en R, qui consiste en un échantillon de taille 133 d'observations de l'accélération de la tête en g (la cible y à prédire)

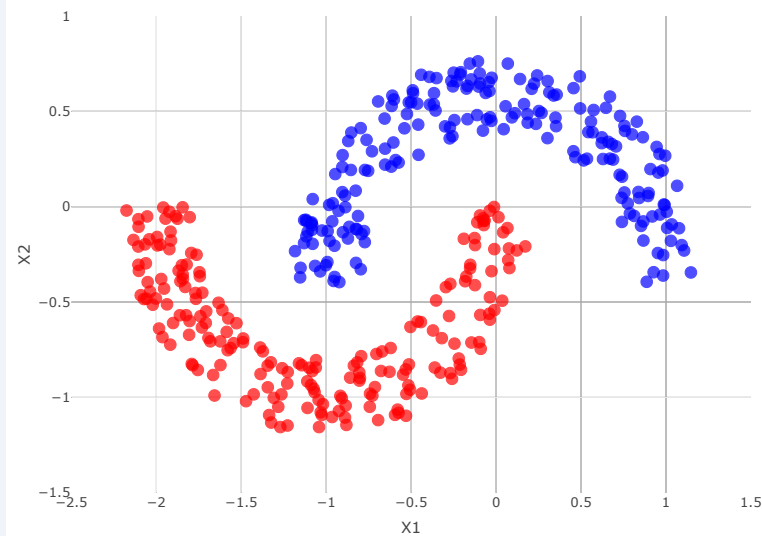
en fonction du temps en ms (la variable explicative x) lors de mesures effectuées sur un motard lorsqu'il roule. Ici on ne considère que les individus avec des temps entre 5 and 55, ce qui amène au final à un échantillon de taille $n = 124$. On représente les prédictions du KNN pour $k = 3$ (bleu), $k = 10$ (vert) et $k = 30$ (rouge).



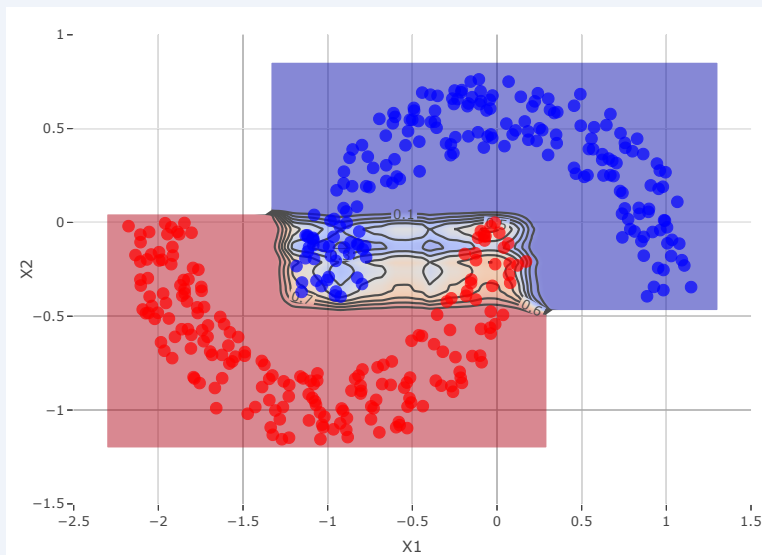
On observe un comportement erratique pour $k = 3$, avec beaucoup de sauts mais avec une prédiction moyenne qui semble bonne. Inversement pour $k = 30$ le prédicteur est plus lisse mais semble être biaisé. Le choix $k = 10$ apparaît comme un bon compromis entre ces deux comportements.

Enfin, il est intéressant d'observer son comportement sur un problème de classification où le classifieur de Bayes naïf et l'analyse discriminante sont peu précis (car les hypothèses qu'ils font ne sont pas vérifiées sur le jeu de données).

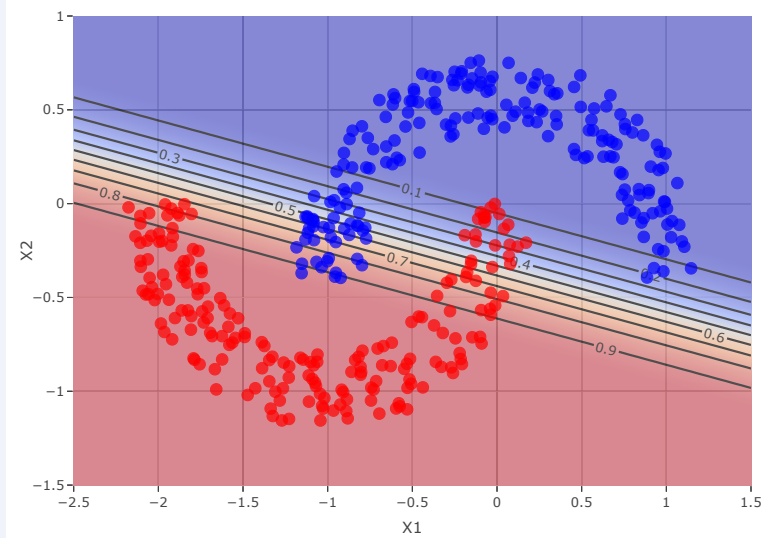
Exemple (Classification des données "Deux Lunes"). Le jeu de données "Deux Lunes" (*Two Moons* en anglais) est fameux en apprentissage statistique, et vous l'avez peut-être déjà croisé en première année lors du cours de clustering. C'est un jeu de données que l'on peut générer par simulation aléatoire, et ici nous en avons construit un particulier avec $n = 200$ individus, qui est représenté ci-dessous avec la convention habituelle de la classe 0 en bleu et de la classe 1 en rouge.



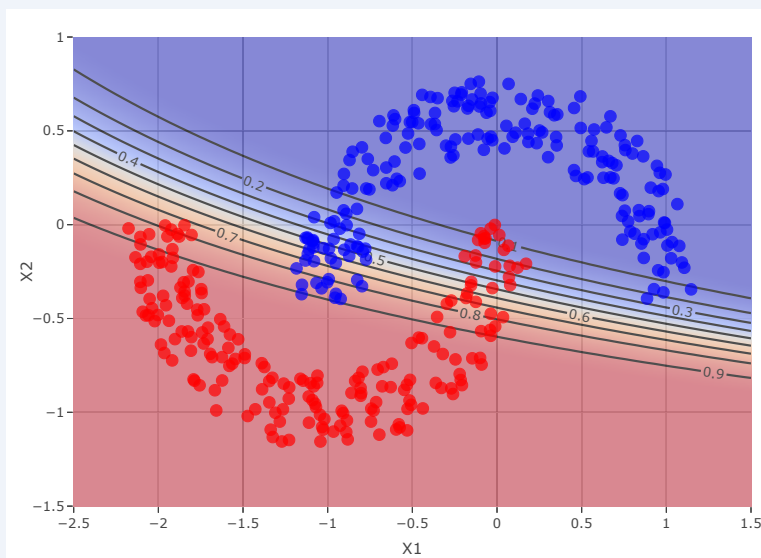
Avant même de comparer les différents classifieurs étudiés jusqu'à présent, il apparaît clairement que, dans chaque classe, les individus ne sont ni distribués de manière indépendante (contredisant l'hypothèse faite par le Bayésien naïf), ni distribués selon une Gaussienne (invalidant l'hypothèse de la LDA et de la QDA). Par conséquent, ces prédicteurs ne seront pas très compétitifs. Voici par exemple les lignes de niveau pour le Bayésien naïf (les coins sont vides car les probabilités estimées des deux classes sont nulles - le classifieur dégénère - dans ces zones) :



Les individus au centre, peu importe leur classe, sont en majorité mal classés. Le phénomène est le même avec la LDA :

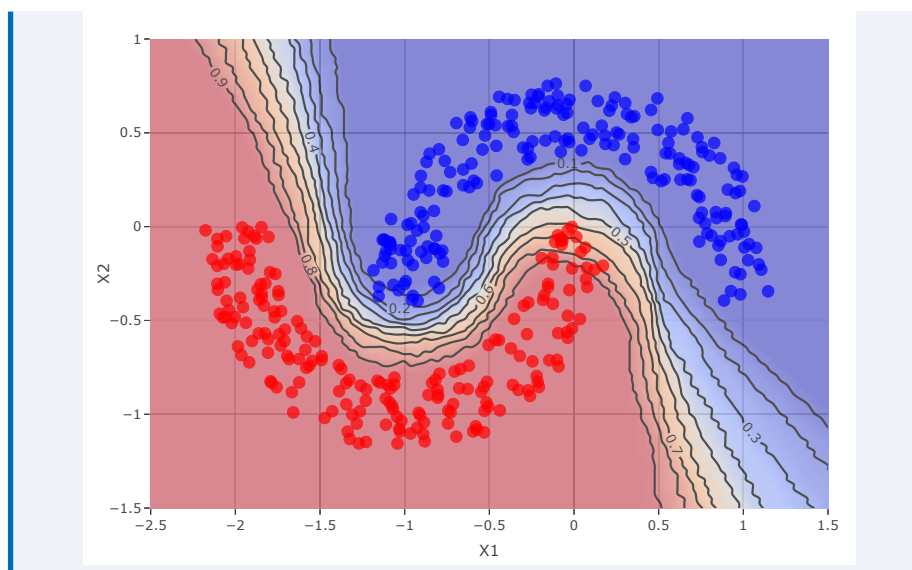


ainsi qu'avec la QDA :



Vous pouvez remarquer que sur ce cas la LDA et la QDA donnent des résultats extrêmement similaires : ce n'est pas surprenant, car les deux classes sont équilibrées (autant d'individus dans chacune) et ont une distribution très semblable à une symétrie horizontale près, donc des matrices de covariance très proches.

Enfin, et à l'inverse des prédicteurs précédents, le classifieur des plus proches voisins avec $k = 50$ par exemple se comporte très bien :



Le dernier point crucial à aborder concerne le choix de la distance d utilisée pour déterminer les plus proches voisins dans l'espace des variables explicatives $\mathcal{X}$:

- Si $\mathcal{X} \subset \mathbb{R}^d$, c'est-à-dire que toutes les variables explicatives sont continues, il est usuel d'utiliser la distance euclidienne $d(\mathbf{x}, \mathbf{x}') = \|\mathbf{x} - \mathbf{x}'\|_2$. Mais il est également possible d'utiliser la distance de Manhattan (*taxicab* en anglais aussi) $d(\mathbf{x}, \mathbf{x}') = \|\mathbf{x} - \mathbf{x}'\|_1$ qui utilise la norme L^1 ou encore la distance maximum $d(\mathbf{x}, \mathbf{x}') = \|\mathbf{x} - \mathbf{x}'\|_\infty$ avec la norme L^∞ .
- Si $\mathcal{X}$ est un espace discret non ordonné, c'est-à-dire que toutes les variables sont catégorielles, le problème est plus épineux (de loin). En effet vous savez que dans un tel cas il est impossible de définir un ordre entre des valeurs, et donc encore moins une distance qui a du sens. En toute généralité et sans connaissance a priori sur le problème, par défaut on peut utiliser la distance de dirac $d(\mathbf{x}, \mathbf{x}') = \sum_{j=1}^d \mathbf{1}_{x^{(j)} \neq x'^{(j)}}$, ou une variante normalisée par la proportion de chaque valeur discrète. Dans certains problèmes pratiques, il arrive que d'un point de vue opérationnel les spécialistes du domaine de l'étude puissent définir une distance "métier", que vous pourrez utiliser.
- Si les variables explicatives sont de type mixte, une distance connue est la *distance de Gower* définie par $d(\mathbf{x}, \mathbf{x}') = \sqrt{1 - S(\mathbf{x}, \mathbf{x}')}$ où S une fonction de similarité (comportement inverse d'une distance). La fonction de similarité S est construite en combinant la distance de dirac ci-dessus avec une version normalisée de la distance de Manhattan. Plus précisément, on a

$$S(\mathbf{x}, \mathbf{x}') = \frac{1}{d} \sum_{j=1}^d s(x^{(j)}, x'^{(j)})$$

où $s(x^{(j)}, x'^{(j)}) = \mathbf{1}_{x^{(j)} = x'^{(j)}}$ si $X^{(j)}$ est catégorielle, et $s(x^{(j)}, x'^{(j)}) = 1 - |x^{(j)} - x'^{(j)}| / (\max x^{(j)} - \min x^{(j)})$ si $X^{(j)}$ est continue. Elle a également été étendue au cas où le jeu de données contient des données manquantes.

- Nous en discuterons également dans le dernier chapitre de ce cours, mais en apprentissage statistique moderne il est également très fréquent de transformer les variables catégorielles au préalable, c'est l'*encodage* (*encoding* en anglais). Après transformation, les variables deviennent continues, et on se ramène alors au premier cas. Mais il est important de noter qu'il n'y a pas de magie, en tout cas pour les encodages les plus simples. Par exemple, l'encodage naïf le plus simple connu sous le nom de *one-hot encoding*, s'il est utilisé en conjonction avec la distance euclidienne sur les variables transformées, donnera à une constante près la distance de dirac que l'on aurait calculée sur les valeurs catégorielles initiales !

Enfin, nous insistons une fois de plus sur le fait que le choix de la distance a un impact très fort sur les performances que le prédicteur KNN pourra avoir au final. Si elle n'est pas informative, le KNN sera très peu précis. Mais à l'inverse si elle est bien choisie, le KNN peut être un prédicteur particulièrement impressionnant, même par rapport à des alternatives modernes (et que l'on pense souvent comme imbattables, à tort assez régulièrement) telles que les réseaux de neurones profonds. Un exemple récent est particulièrement éclairant à ce sujet : <https://aclanthology.org/2023.findings-acl.426/>. N'oubliez donc pas de systématiquement utiliser un prédicteur KNN dans vos études, qui vous servira de méthode de référence au même titre que le classifieur de Bayes naïf ou la LDA pour les problèmes de classification.

2.2.2 Vers le lissage et les méthodes à noyau

Nous terminons ce chapitre en mentionnant brièvement quelques extensions naturelles et intuitives des plus proches voisins, que vous pourrez approfondir dans un cours spécifique au second semestre.

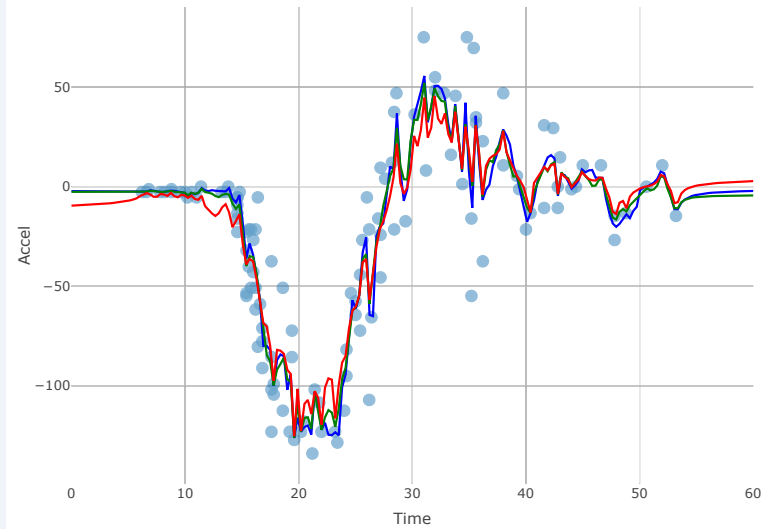
La première idée intuitive d'extension est la suivante : quand on moyenne l'information des plus proches voisins pour construire le KNN, tous les voisins ont le même poids, qu'ils soient proches (le plus proche voisin) ou plus éloignés (le k -ième plus proche voisin). Or il est naturel de penser que selon leur distance au point $\mathbf{x}$ de prédiction, ils sont plus ou moins informatifs. On peut alors simplement proposer de faire une moyenne pondérée par les distances, par exemple en régression

$$\hat{f}_{\text{knn}}(\mathbf{x}) = \alpha \sum_{i \in N_k(\mathbf{x})} d(\mathbf{x}, \mathbf{x}_i)^{-1} y_i$$

où $\alpha^{-1} = \sum_{i \in N_k(\mathbf{x})} d(\mathbf{x}, \mathbf{x}_i)^{-1}$ est un terme de normalisation pour que les poids somment à 1. Cette modification est parfois utilisée en pratique, elle est illustrée sur l'exemple précédent ci-dessous.

Exemple (Illustration du KNN avec pondération selon l'inverse de la distance).

On reprend le jeu de données `mcycle` du package **MASS** en R, et on représente les prédictions du KNN où on utilise une pondération avec l'inverse des distances aux plus proches voisins pour $k = 3$ (bleu), $k = 10$ (vert) et $k = 30$ (rouge).



Le résultat obtenu est assez typique de ce qu'on obtient généralement avec cette pondération : l'influence de k diminue sur le comportement du prédicteur, mais on observe un certain nombre de "pics" car la fonction inverse n'est pas régulière en fonction de la distance.

La seconde idée, qui rejoindra la première au final, découle de l'observation du comportement erratique du prédicteur KNN particulièrement prononcé quand k est petit (voir les figures précédentes). Pour analyser ce comportement, on peut ré-écrire le prédicteur KNN en régression comme

$$\hat{f}_{\text{knn}}(\mathbf{x}) = \sum_{i=1}^n w_i(\mathbf{x}) y_i$$

avec des poids $w_i(\mathbf{x})$ donnés par

$$w_i(\mathbf{x}) = \begin{cases} 1/k & \text{si } \mathbf{x}_i \text{ est un des } k \text{ plus proches voisins de } \mathbf{x} \\ 0 & \text{sinon.} \end{cases}$$

Puisque les poids $w_i(\mathbf{x})$ font des sauts en passant d'un ensemble des plus proches voisins à un autre, c'est aussi le cas pour $\hat{f}_{\text{knn}}(\mathbf{x})$. En partant de ce constat et de l'expression du prédicteur comme une moyenne pondérée, le *lissage par noyau* propose de remplacer la fonction de poids discontinue par une fonction de poids plus lisse $K : \mathbb{R} \mapsto \mathbb{R}$ connue sous le nom de *noyau*. Pour garantir de bonnes propriétés à l'estimateur résultant, on impose que $\int K(t)dt = 1$, $\int tK(t)dt = 0$ et $\int t^2 K(t)dt < \infty$ (vous verrez pourquoi dans le cours de spécialisation). En résumé, au lieu d'attribuer un poids égal à $1/k$ aux voisins les plus proches et 0 aux autres individus de l'échantillon, on utilise le noyau pour définir les poids en fonction de la distance entre $\mathbf{x}$ et les individus $\mathbf{x}_i$, avec un poids décroissant en fonction de la distance. Des exemples de noyaux courants sont le noyau Gaussien

$$K(t) = \frac{1}{\sqrt{2\pi}} \exp(-t^2/2)$$

ou le noyau Epanechnikov

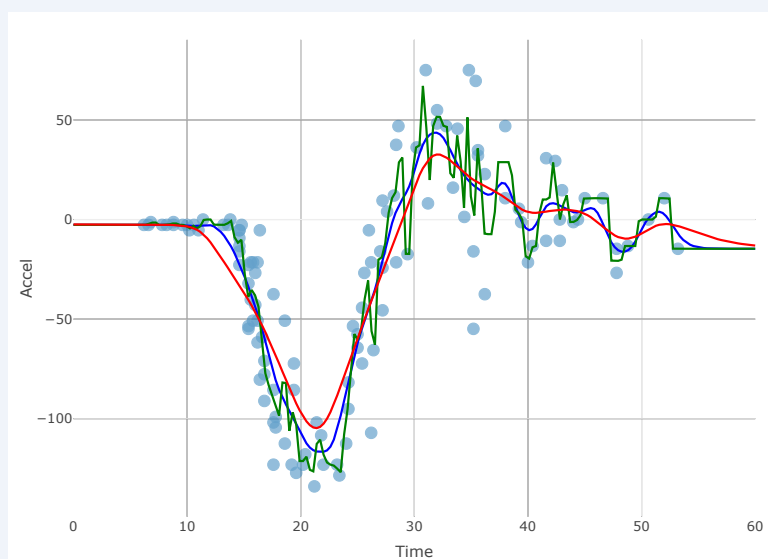
$$K(t) = \frac{3}{4}(1 - t^2)\mathbf{1}(|t| \leq 1).$$

Remarquez que dans ces deux cas, $K(t)$ diminue effectivement avec t . Les poids sont finalement donnés par

$$w_i(\mathbf{x}) = \frac{K(\|\mathbf{x} - \mathbf{x}_i\|_2/h)}{\sum_{j=1}^n K(\|\mathbf{x} - \mathbf{x}_j\|_2/h)}$$

où h est un hyperparamètre contrôlant la vitesse de décroissance des poids par rapport à la distance, qui s'appelle la *fenêtre du noyau* (*bandwidth* en anglais). La fenêtre a exactement le même effet que le nombre de plus proches voisins : quand elle est petite, on lisse peu et on prend en compte seulement les individus très proches, et à l'inverse quand elle est grande on lisse beaucoup. En pratique elle pourra être estimée automatiquement par validation croisée. Le prédicteur obtenu au final est le fameux estimateur de **Nadaraya-Watson**, que l'on illustre ci-dessous sur l'exemple précédent.

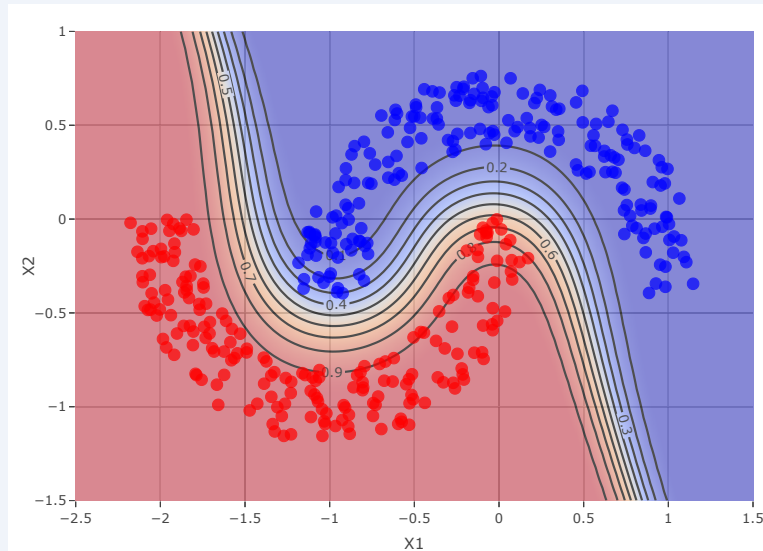
Exemple (Illustration du lissage par noyau). Sur le jeu de données *mcycle*, on représente les prédictions de l'estimateur de Nadaraya-Watson avec un noyau Gaussien et une fenêtre h optimisée par validation croisée (bleu), h trop petite (vert) et h trop grande (rouge).



On observe clairement l'effet de lissage du noyau par rapport au prédicteur KNN.

On peut également remarquer l'effet de lissage par rapport au prédicteur KNN sur le problème de classification vu précédemment.

Exemple (Classification des données "Deux Lunes" - suite). Sur le jeu de données "Deux Lunes" étudié précédemment, voici les lignes de niveau de la fonction de décision pour le classifieur utilisant le lissage par noyau (avec un noyau Gaussien et une fenêtre fixée $h = 0.3$).



2.3 Le fléau de la dimension

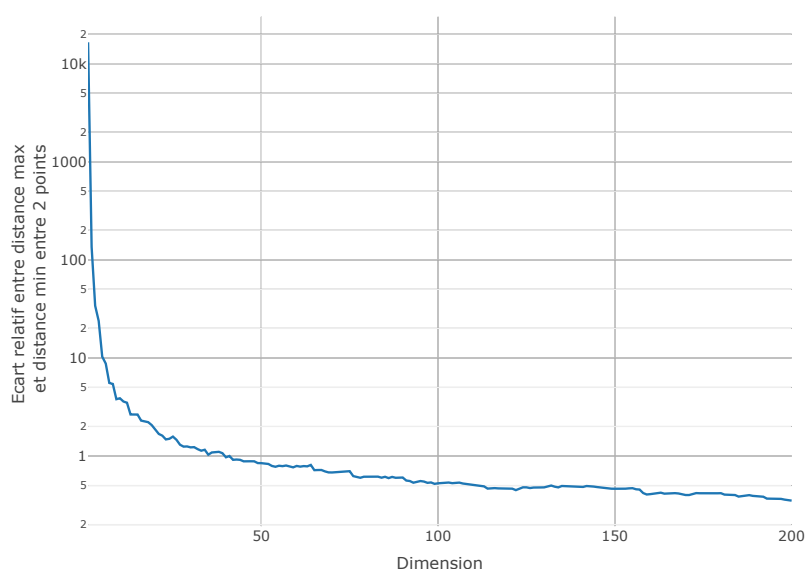
Le fléau ou la malédiction de la dimension (*Curse of dimensionality* en anglais) en apprentissage supervisé se réfère au défi croissant que pose l'augmentation du nombre de variables explicatives par rapport au nombre d'observations dans un jeu de données. L'argument traditionnel est que cette situation conduit à une augmentation exponentielle de la complexité et de la taille de l'espace des caractéristiques, rendant la tâche d'apprentissage plus difficile. Les modèles ont tendance à surajuster les données, perdent en généralité et nécessitent des ensembles de données massifs pour être efficaces.

Si l'on se focalise sur les prédicteurs que nous avons vu dans ce chapitre, on peut faire les commentaires suivants :

- **Classifieur de Bayes naïf.** Etant donné qu'il est basé sur une hypothèse d'indépendance des variables explicatives, par nature ce classifieur ne souffre pas d'une augmentation exponentielle de la complexité.
- **LDA / QDA.** A première vue, comme ces classifieurs font une hypothèse forte sur la loi des variables explicatives, il semble qu'ils soient également peu sensibles au fléau de la dimension. Cependant, ce n'est pas le cas (le diable se cache dans les détails ...) : en effet ils reposent tous les deux sur une estimation de la matrice de covariance des variables explicatives, dont le nombre de coefficients est de l'ordre de d^2 , qui croît donc très vite avec la dimension d . L'estimation de matrices de covariance en grande

dimension est un domaine de recherche encore très actif à l'heure actuelle et un certain nombre d'approches intéressantes pour lutter contre le fléau de la dimension existent, mais en général elles ne seront pas disponibles dans les librairies usuelles en R ou en Python.

- **KNN.** La brique essentielle de ce prédicteur est la distance définie dans l'espace des variables explicatives. Or quand la dimension augmente, un phénomène de *concentration* se produit : si les variables sont indépendantes, les distances maximales et minimales observées entre des points deviennent de plus en plus indiscernables, rendant quelque peu caduque l'intuition de proximité dans l'espace (et donc le concept de plus proches voisins). La figure ci-dessous illustre ce phénomène.



Pour cette raison, il est fréquemment recommandé (dans les livres de référence ou dans les cours) d'essayer de réduire la dimension avant d'utiliser un prédicteur KNN. Dans les approches que vous connaissez à l'heure actuelle, l'ACP est une candidate potentielle, mais nous verrons dans la dernière partie du cours d'autres méthodes de réduction de dimension qui sont non-linéaires et pourront être plus performantes selon les cas. La procédure complète consistant en une réduction de dimension préliminaire qui fournit un nombre restreint de nouvelles variables, qui sont ensuite utilisées pour calculer des distances dans le prédicteur KNN est illustrée dans un exemple de classification en grande dimension ci-dessous.

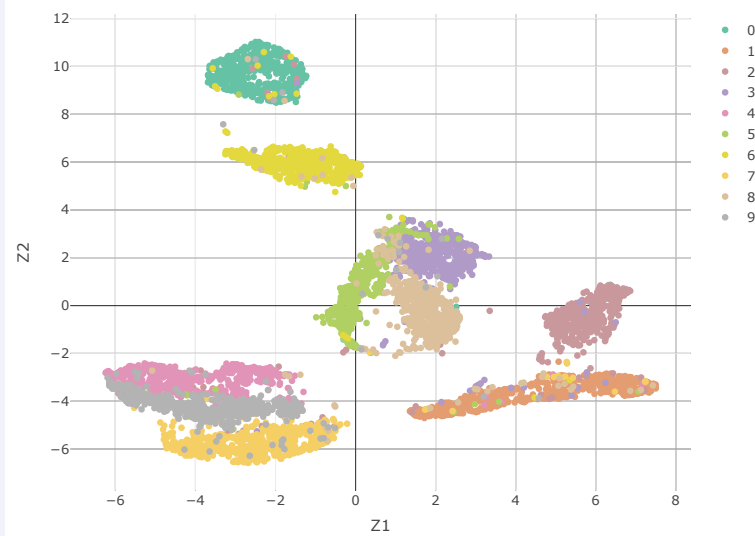
En particulier, cet exemple montre que le fléau de la dimension, bien qu'il existe, n'est pas aussi général et universel qu'on l'entend parfois et qu'il convient de nuancer certains propos usuels sur la "malédiction". Par exemple, le phénomène de concentration mentionné précédemment est mathématiquement justifié uniquement si les variables explicatives sont indépendantes entre elles, ce qui n'est pas le cas usuel en apprentissage statistique. Vous pouvez lire la discussion sur le fléau de la dimension ici :

https://en.wikipedia.org/wiki/Curse_of_dimensionality.

Exemple (Classification sur le jeu de données MNIST). Le jeu de données MNIST est très fameux en apprentissage supervisé et est utilisé dans de nombreuses comparaisons de prédicteurs. C'est un problème de classification : on dispose d'images de chiffres (de 0 à 9) dessinés par des humains et on cherche à prédire le chiffre correspondant. Chaque image est un ensemble de $28 \times 28 = 784$ pixels en niveaux de gris, impliquant que chaque individu du jeu de données est représenté par des variables explicatives en dimension $d = 784$ (une valeur par pixel). Le jeu de données original a déjà été séparé en un échantillon d'entraînement de taille 60000, et un échantillon de test de taille 10000. Ici nous avons travaillé sur des échantillons de taille plus petite, respectivement 6000 et 1000 pour accélérer les calculs. Comme méthode de réduction de dimension préliminaire, nous évaluons l'ACP, qui sélectionne 86 nouvelles variables à la place des 784 initiales (90% de variance expliquée), et la méthode UMAP (que nous aborderons lors du dernier cours) que nous avons utilisé pour trouver seulement 2 nouvelles variables. Pour chaque méthode évaluée (KNN dans l'espace original, KNN après ACP et KNN après UMAP), nous calculons τ_{erreur} le taux de mauvaise classification sur l'ensemble de test (i.e. le risque empirique avec la fonction de perte 0-1) :

	KNN	ACP + KNN	UMAP + KNN
τ_{erreur}	9.26%	9.46%	8.17%

De manière surprenante à première vue par rapport à notre discussion sur le fléau de la dimension, l'étape d'ACP préliminaire, bien qu'elle permette de réduire la dimension du problème de 90%, ne semble pas aider à améliorer les prédictions du KNN. Cela justifie la nuance que nous avons apportée au-dessus. Enfin, les résultats obtenus avec la méthode UMAP sont légèrement meilleurs, mais il convient de garder en tête que nous n'avons gardé que 2 nouvelles variables explicatives ici : un tel résultat est alors assez remarquable. Pour illustration, nous traçons ci-dessous la représentation de l'échantillon d'entraînement dans ce nouvel espace de dimension 2.



Vous observerez que sans aucune information de supervision, la transformation non-linéaire identifiée par UMAP permet d'isoler les classes dans l'espace des nouvelles variables explicatives de manière extrêmement propre, et il n'est donc pas surprenant qu'une prédiction par plus proches voisins fonctionne bien dans ce nouvel espace.

Séance 4 : Arbres et forêts aléatoires

3 Octobre, 2023

- Introduction aux modèles d'arbres
 - Généralités, exemples
 - Description de l'algorithme CART
- Forêts aléatoires

Chapitre 3

Arbres et forêts aléatoires

Au chapitre précédent nous avons vu des modèles d'apprentissage supervisé dont la construction naturelle est intuitive quand ils sont vus comme des approximations explicites de $\eta^*(\mathbf{x}) = \mathbb{P}(Y = 1|\mathbf{X} = \mathbf{x})$ en classification et de $f^*(\mathbf{x}) = \mathbb{E}(Y|\mathbf{X} = \mathbf{x})$ en régression. Dans ce chapitre, nous changeons de paradigme, pour adopter un point de vue plus proche de celui que vous avez étudié en régression linéaire, à savoir celui de l'approximation d'une fonction. Dans ce cadre, η^* et f^* sont vus comme des fonctions inconnues que vous devez estimer, et comme point de départ vous allez supposer que vous allez chercher une approximation qui a une forme simple. En régression linéaire vous approchez f^* par une fonction linéaire $\hat{f}$, et vous verrez dans le cours de modèles linéaires généralisés (*Generalized Linear Models* ou GLM) que vous chercherez à approcher η^* par $\hat{\eta}$ une transformation simple d'une fonction linéaire. Les modèles à base d'arbres, de leur côté, vont également chercher une approximation qui a une forme simple : une fonction constante par morceaux !

A partir des modèles d'arbres, des propositions plus récentes basées sur l'idée de combiner ensemble plusieurs arbres ont émergé, et sont à l'heure actuelle considérées comme parmi les plus puissantes en apprentissage supervisé. Nous nous concentrerons dans ce cours sur les forêts aléatoires, et vous verrez en troisième année une autre variante appelée "xgboost".

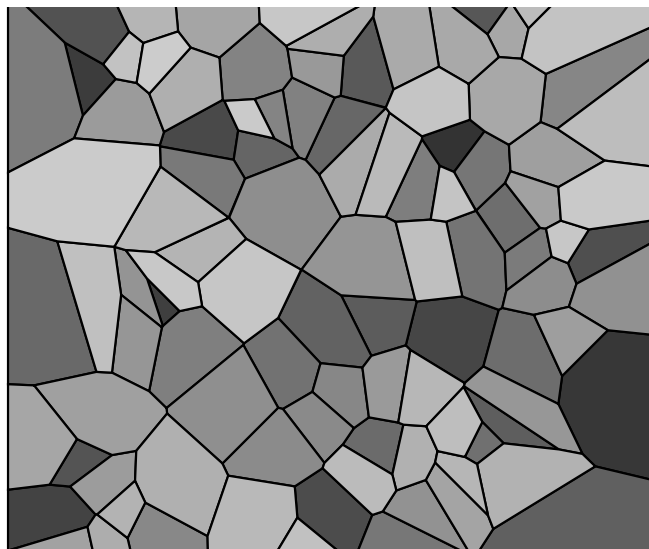
3.1 Arbres CART

Avant de détailler les spécificités des arbres CART, qui sont l'objet de ce chapitre, nous introduisons d'abord de manière générale le principe des modèles de régression ou de classification à base d'arbres.

3.1.1 Introduction aux modèles d'arbres

Les arbres de décision sont une technique puissante en apprentissage statistique utilisée pour prendre des décisions en séparant progressivement un ensemble de données en sous-groupes basés sur des caractéristiques spécifiques. Le principe général derrière les arbres de décision est de diviser récursivement l'ensemble de données en fonction des variables explicatives, ce qui aboutit à un pavage de l'espace des variables explicatives. On en donne un exemple (purent illustratif)

ci-dessous.



Une fois que l'espace des variables explicatives est découpé en régions disjointes (R_j) chacune contenant n_j individus de l'échantillon d'entraînement, i.e. $\mathcal{X} = \bigcup_{j=1}^J R_j$, les prédictions d'un modèle d'arbre sont très intuitives :

- En classification, on identifie la région $j_{\mathbf{x}}$ dans laquelle tombe les caractéristiques de l'individu à prédire $\mathbf{x}$, et on utilise un vote à la majorité $\hat{h}_{\text{arbre}}(\mathbf{x}) = C_{l^*}$ où l^* est le numéro de la catégorie majoritaire des individus de l'échantillon d'apprentissage qui sont dans la région $j_{\mathbf{x}}$:

$$l^* = \arg \max_{l=1, \dots, M} \frac{1}{n_{j_{\mathbf{x}}}} \sum_{i: \mathbf{x}_i \in R_{j_{\mathbf{x}}}} \mathbb{1}_{y_i = C_l}.$$

- En régression, on utilise simplement la moyenne des individus de l'échantillon d'apprentissage qui sont dans la région $j_{\mathbf{x}}$:

$$\hat{f}_{\text{arbre}}(\mathbf{x}) = \frac{1}{n_{j_{\mathbf{x}}}} \sum_{i: \mathbf{x}_i \in R_{j_{\mathbf{x}}}} y_i.$$

Remarque (Variante de la règle de prédiction en classification). Nous reviendrons à la fin de cette section sur le choix du classifieur final une fois les régions identifiées, car dans certaines situations pratiques il est recommandé d'adapter le choix de la classe à prédire.

On peut démontrer facilement qu'avec des régions **fixées**, ce prédicteur est le minimiseur du risque empirique avec la fonction de perte 0-1 en classification et la fonction de perte quadratique en régression. Au final, toutes les prédictions pour des individus $\mathbf{x}$ qui tombent dans la même région sont identiques, le

prédicteur est donc une fonction constante par morceaux. Nous verrons également dans le cas particulier des arbres CART pourquoi avec un tel modèle on parle d'*arbre*.

Le cœur du problème est par conséquent l'identification d'un pavage de l'espace qui permettra au final d'avoir un bon prédicteur. Ce problème est malheureusement extrêmement complexe : l'ensemble des découpages possibles est infini, il est également très difficile de trouver une paramétrisation générale des régions qui permettrait de les faire bouger à l'aide de quelques paramètres, et quand bien même on aurait de tels paramètres, leur optimisation pour paver l'espace au mieux serait particulièrement ardue. C'est la raison pour laquelle toutes les méthodes d'arbres :

1. Font des hypothèses simplificatrices sur la forme des découpages qu'elles vont considérer, pour rendre leur identification plus aisée.
2. Utilisent une optimisation simplifiée des régions, en les optimisant de manière réursive l'une après l'autre ; c'est ce qu'on appelle une optimisation *gloutonne* (*greedy* en anglais).

3.1.2 Algorithme CART

Principe général

Les arbres CART (Classification And Regression Trees) sont une variante populaire des arbres de décision, qui sont en pratique les plus utilisés en apprentissage supervisé. Leur particularité principale est qu'ils découpent l'espace, de manière **itérative**, en **régions rectangulaires**. Pour arriver à ces régions rectangulaires, la procédure procède par optimisation gloutonne comme dit précédemment, avec un principe à retenir : ce sont des **règles de construction binaires, sur une seule variable explicative à la fois, et parallèles aux axes**. Il est alors évident que si l'on suit de telles règles, alors les régions obtenues au final seront bien des rectangles. En pratique :

- A chaque étape de la construction, on va donc avoir une liste de régions candidates, et l'algorithme choisira parmi elles selon un **critère** d'efficacité. Nous discuterons de quels critères sont utilisés en pratique un peu plus tard.
- Au début de la construction, l'espace contient une seule et unique région : l'espace $\mathcal{X} \in \mathbb{R}^d$. La première itération et proposition de région est simple, l'algorithme va chercher à couper l'espace en deux pour en faire deux régions distinctes, en coupant par rapport à une variable explicative, de manière parallèle aux axes. Par exemple pour une variable explicative continue, un découpage potentiel sera donné par $R_1 = \{\mathbf{x} \in \mathbb{R}^d : x^{(1)} \leq 4.35\}$ et $R_2 = \{\mathbf{x} \in \mathbb{R}^d : x^{(1)} > 4.35\}$. Pour une variable explicative discrète qui prend les modalités {rouge, vert, bleu, jaune} un découpage potentiel s'écrit $R_1 = \{\mathbf{x} \in \mathbb{R}^d : x^{(2)} \in \{\text{rouge, bleu}\}\}$ et $R_2 = \{\mathbf{x} \in \mathbb{R}^d : x^{(2)} \in \{\text{vert, jaune}\}\}$.

L'algorithme devra donc à la fois choisir selon quelle variable explicative il devrait couper, et aussi le seuil (pour les variables continues) ou la séparation de classes (pour les variables catégorielles). Il va tester toutes

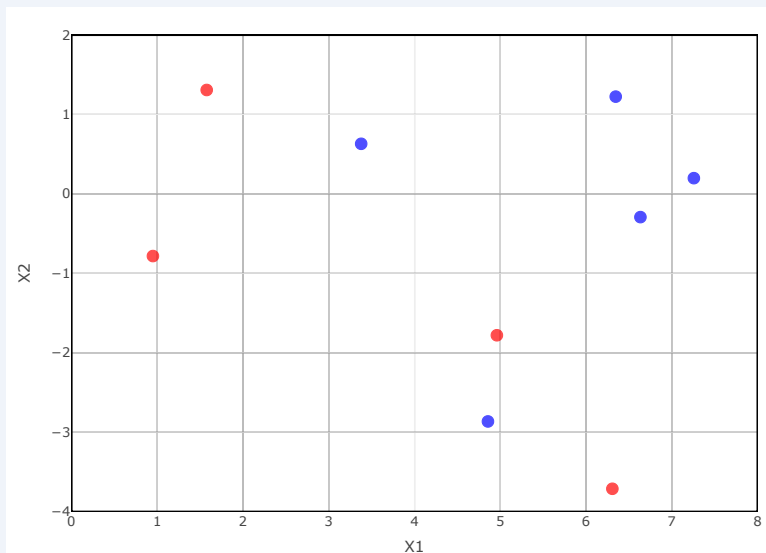
les possibilités et choisir le meilleur découpage selon le critère d'efficacité défini au départ.

- Après cette première itération, les suivantes sont très similaires. La liste des régions candidates est obtenue de manière assez simple : on part des régions de l'itération précédente, et on va proposer d'en couper une en deux, avec la même procédure décrite au point précédent. On va répéter ce test de découpe en deux pour toutes les régions, et choisir pour quelle région, sur quelle variable, et selon quel seuil le critère est le meilleur.
- Cet algorithme itératif se poursuit jusqu'à atteindre un **critère d'arrêt**. Nous en discuterons également plus tard, mais typiquement en pratique on va imposer qu'on ne peut plus découper une région en deux quand il ne reste qu'un petit nombre d'individus de l'échantillon d'entraînement dedans.
- A la fin de l'algorithme, on obtient alors un découpage de l'espace en régions, et dans chacune d'entre elles se trouve des individus de l'ensemble d'entraînement. Cela permet de construire le prédicteur défini au début du chapitre (vote à la majorité en classification ou moyenne en régression).
- La séquence de choix qui ont été faits par l'algorithme peut se représenter sous la forme d'un **arbre de décision**, voir l'exemple ci-dessous.

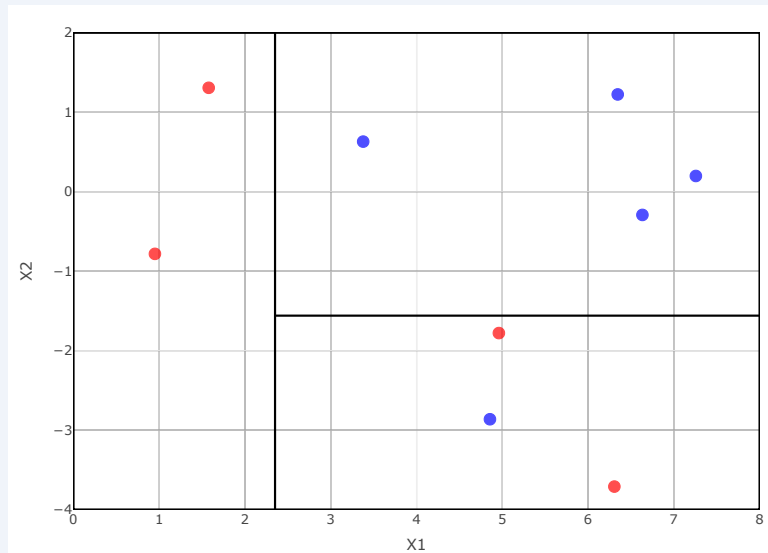
Pour clarifier les choses, on illustre maintenant cet algorithme sur un cas jouet.

Exemple (Construction d'un arbre CART pour un problème de classification).

On considère un problème de classification binaire avec deux variables explicatives, et un échantillon d'entraînement de taille $n = 9$. On représente d'abord le jeu de données (classe 0 en bleu et classe 1 en rouge).



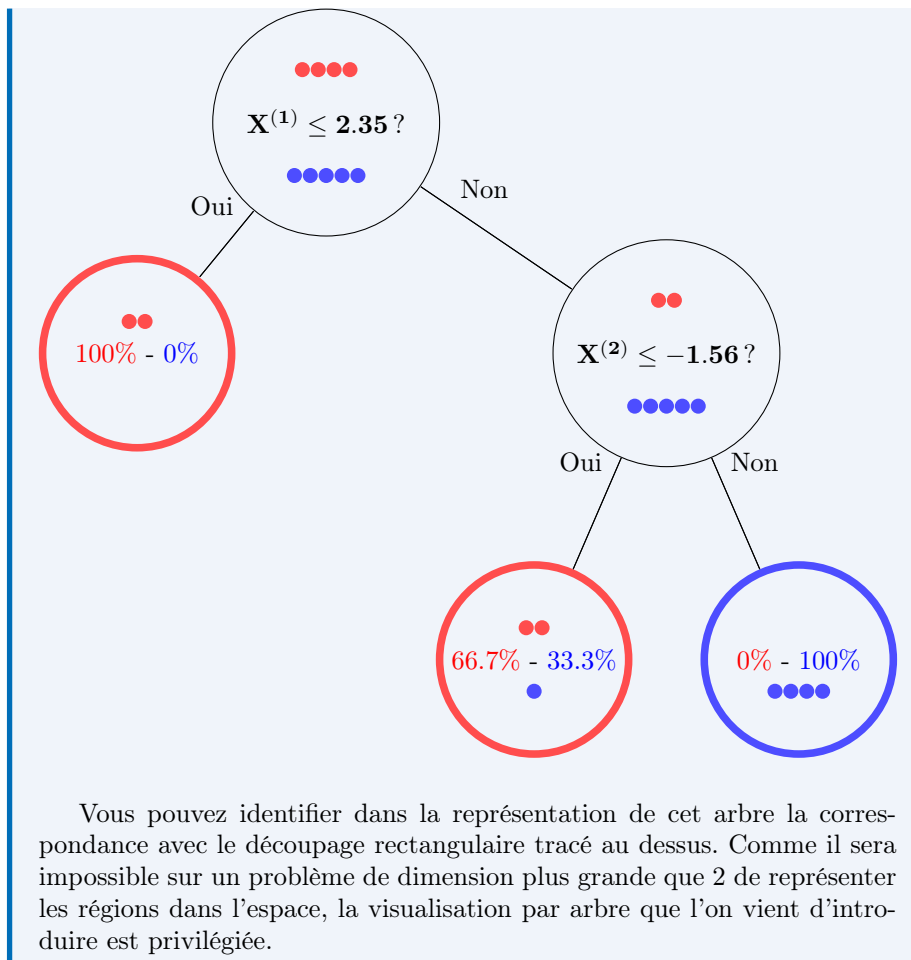
On utilise l'algorithme CART, et on obtient le découpage en région suivant. L'algorithme a d'abord découpé l'espace complet (le grand rectangle qui définit le domaine) selon la variable $X^{(1)}$ avec un seuil égal à 2.35, puis il a découpé la région de droite selon la variable $X^{(2)}$ avec un seuil égal à -1.56 , et s'est ensuite arrêté. On visualise ci-dessous à quoi ressemblent les régions obtenues au final.



On peut donc maintenant représenter la suite de décisions de divisions sous une forme arborescente, représentée sur notre cas ci-dessous :

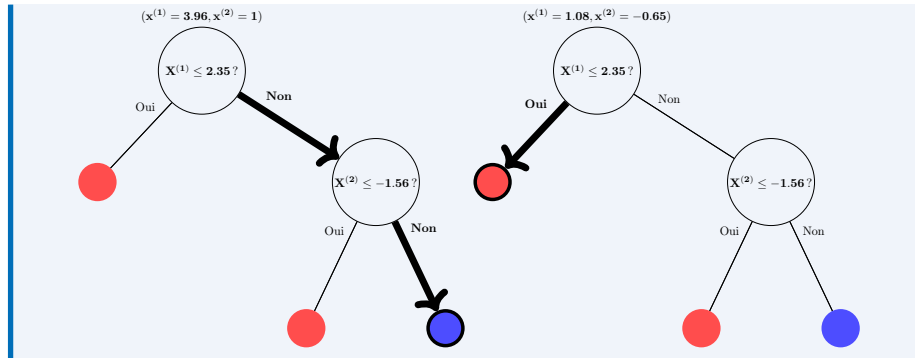
- Au départ l'ensemble des individus de l'ensemble d'entraînement n'est pas divisé, ils sont tous à l'intérieur d'un même **nœud**. Ce nœud s'appelle la **racine**.
- Un nœud peut ensuite :
 - soit être re-découpé, il sera alors un nœud intermédiaire, qui sera appelé alors **nœud-parent**, et dont la division générera deux **nœuds-enfants**.
 - soit ne plus être divisé, ce sera alors un **nœud-terminal**, ou encore une **feuille**.
- Au centre d'un nœud on donne la règle de découpage (ou de **division**, ou de **scission**) qui a été identifiée comme la meilleure par l'algorithme.

Pour illustration ici, à l'intérieur d'un nœud on représente aussi les individus de l'ensemble d'apprentissage qui sont dedans. Pour les nœuds terminaux, on indique la proportion à l'intérieur du nœud de chaque classe calculée sur les individus qui sont dans ce nœud, car comme dit précédemment le prédicteur final utilise cette information.



Une fois l'arbre construit, on peut alors l'utiliser en prédiction. On notera en pratique que la représentation graphique usuelle des arbres ne contiendra pas les points rouge et bleu dessinés dans l'exemple précédent, en général vous aurez le pourcentage de chaque classe dans le nœud affichée à la place.

Exemple (Prédiction dans un arbre CART). Dans l'exemple précédent, imaginons que l'on souhaite prédire la classe pour deux nouveaux individus non observés $\mathbf{x} = (x^{(1)} = 3.96, x^{(2)} = 1)$ et $\mathbf{x} = (x^{(1)} = 1.08, x^{(2)} = -0.65)$. Pour ce faire, la procédure est simple : on fait *descendre* l'individu dans l'arborescence, en choisissant à chaque nœud intermédiaire de quel côté il va continuer son chemin, jusqu'à ce qu'il atteigne un nœud terminal, dont on utilise les proportions de classe (en classification) ou la moyenne (en régression) pour faire la prédiction. Ce principe est illustré graphiquement ci-dessous.



Critères de division

Dans la description précédente de l'algorithme CART, nous avons simplement dit que le choix de la meilleure division dépendait d'un critère d'efficacité. Nous allons maintenant discuter des critères communément utilisés.

Pour commencer, intuitivement, il semble logique d'adopter un principe général basé sur la **pureté** ou l'**homogénéité** d'un nœud. Par exemple en classification, si à l'intérieur d'un nœud tous les individus ont la même classe, il n'y a aucun intérêt à continuer de le diviser (puisque toute division supplémentaire aboutira à des nœuds avec une seule classe). On dira alors qu'un tel nœud est *pur* ou *homogène*. A l'inverse, si à l'intérieur d'un nœud les individus se répartissent uniformément dans toutes les classes, le nœud est *impur* ou *hétérogène*, et dans ce cas il est probable qu'une division puisse permettre d'aboutir à deux nouveaux nœuds-enfants plus homogènes. En résumé, si un nœud est pur, on ne le divisera pas, et si un nœud est impur, on essaiera de le diviser pour aboutir à des nœuds-enfants plus purs. S'il est pas possible de trouver une division améliorant la pureté, alors l'algorithme s'arrêtera.

La question est donc désormais de pouvoir mesurer quantitativement le niveau de pureté d'un nœud donné. Comme vous en avez l'habitude, la discussion va différer selon si l'on a un problème de régression ou de classification. Nous introduisons d'abord quelques notations qui nous seront utiles par la suite.

Notations. On note t le nœud d'un arbre et t_G et t_D les deux nœuds-enfants issus d'une possible division de t (G pour gauche et D pour droite par convention). On commence par donner les notations des quantités théoriques du problème :

- $\mathbb{P}(\mathbf{X} \in t)$ est la probabilité que les valeurs des variables explicatives $\mathbf{X}$ tombent dans la région de l'espace associée à un nœud t .
- En **classification**, pour toutes les classes C_l , $l = 1, \dots, M$ dans lesquelles la sortie peut se retrouver, on écrira

$$\mathbb{P}(t|C_l) := \mathbb{P}(\mathbf{X} \in t|Y = C_l)$$

la probabilité que les valeurs de $\mathbf{X}$ tombent dans le nœud t sachant

que Y a pour classe C_l , et réciproquement

$$\mathbb{P}(C_l|t) := \mathbb{P}(Y = C_l | \mathbf{X} \in t)$$

la probabilité que Y ait pour classe C_l sachant que les valeurs de $\mathbf{X}$ tombent dans le nœud t . Enfin, on notera

$$\pi_l := \mathbb{P}(Y = C_l)$$

la probabilité que Y ait pour classe C_l .

- En **régression**,

$$\mu(Y|t) := \mathbb{E}(Y | \mathbf{X} \in t)$$

et

$$\text{Var}(Y|t) := \text{Var}(Y | \mathbf{X} \in t)$$

sont l'espérance et la variance conditionnelle de la sortie Y sachant que les valeurs de $\mathbf{X}$ tombent dans le nœud t .

A partir de l'échantillon d'apprentissage, on peut en déduire des estimateurs de certaines de ces quantités :

- En notant $n(t)$ le nombre d'individus de l'ensemble d'apprentissage dans le nœud t , on a

$$\hat{\mathbb{P}}(\mathbf{X} \in t) = \frac{n(t)}{n}$$

- En **classification**, si on note n_l le nombre d'individus de classe C_l dans l'échantillon et $n_l(t)$ le nombre d'individus de classe C_l dans le nœud t , on a :

$$\begin{aligned} \hat{\mathbb{P}}(t|C_l) &= \frac{n_l(t)}{n_l} \\ \hat{\pi}_l &= \frac{n_l}{n}. \end{aligned}$$

- En **régression**,

$$\begin{aligned} \hat{\mu}(Y|t) &= \bar{y}(t) \\ \widehat{\text{Var}}(Y|t) &= s^2(t) \end{aligned}$$

où $\bar{y}(t)$ et $s^2(t)$ sont la moyenne et la variance empirique de la sortie pour les individus dont les valeurs $\mathbf{x}_i$ sont dans le nœud t .

A l'aide de ces notations, nous pouvons maintenant donner la définition d'une *mesure d'impureté* (empirique ici).

Définition 6 (Mesure d'impureté). Une **mesure d'impureté** $i(t)$ d'un nœud t est une fonction positive, nulle quand le nœud est pur (que des individus avec la même valeur dans le nœud) et dont la valeur augmente quand le

noeud est impur. La **réduction d'impureté** induite par la division d'un noeud t , notée $\Delta i(t)$, est donnée par

$$\Delta i(t) = i(t) - \frac{n(t_G)}{n(t)} i(t_G) - \frac{n(t_D)}{n(t)} i(t_D).$$

Puisque que l'on va chercher à réduire l'impureté lors de la construction de l'arbre, on voit par exemple que la réduction maximale pour un noeud donné est obtenue quand $i(t_G) = i(t_D) = 0$, c'est-à-dire que l'on a réussi à créer deux noeuds-enfants complètement purs. Dans un cas idéal où l'on pourrait obtenir une telle réduction maximale pour tous les noeuds, on déciderait alors de couper le noeud dont l'impureté est la plus grande (puisque dans un tel cas on aurait une réduction d'impureté $\Delta i(t) = i(t)$ pour tous les noeuds t).

Nous commençons par introduire la mesure d'impureté en régression, car elle est très simple, intuitive et fait appel à une notion que vous connaissez bien, la variance. En effet vous savez que c'est une mesure de dispersion, et que par exemple la variance empirique calculée sur un échantillon est nulle si tous les échantillons sont égaux, de même qu'elle augmente quand les valeurs des individus sont très différentes. C'est donc par définition même une mesure d'impureté! On définit alors la réduction d'impureté en régression à l'aide de la variance empirique.

Définition 7 (Réduction d'impureté en régression). A partir de la mesure d'impureté égale à la variance empirique, la réduction d'impureté en régression est donnée par

$$\Delta i_V(t) = s^2(t) - \frac{n(t_G)}{n(t)} s^2(t_G) - \frac{n(t_D)}{n(t)} s^2(t_D).$$

Remarque (Décomposition de la réduction d'impureté en régression). Il est intéressant de remarquer que l'on peut ré-écrire cette réduction d'impureté sous la forme

$$\begin{aligned} & s^2(t) - \frac{n(t_G)}{n(t)} s^2(t_G) - \frac{n(t_D)}{n(t)} s^2(t_D) \\ &= \bar{y}^2(t) - \bar{y}(t)^2 - \frac{n(t_G)}{n(t)} (\bar{y}^2(t_G) - \bar{y}(t_G)^2) - \frac{n(t_D)}{n(t)} (\bar{y}^2(t_D) - \bar{y}(t_D)^2) \\ &= \frac{n(t_G)}{n(t)} \bar{y}^2(t_G) + \frac{n(t_D)}{n(t)} \bar{y}^2(t_D) - \bar{y}(t)^2 \\ &\quad - \frac{n(t_G)}{n(t)} (\bar{y}^2(t_G) - \bar{y}(t_G)^2) - \frac{n(t_D)}{n(t)} (\bar{y}^2(t_D) - \bar{y}(t_D)^2) \\ &= \frac{n(t_G)}{n(t)} \bar{y}(t_G)^2 + \frac{n(t_D)}{n(t)} \bar{y}(t_D)^2 - \bar{y}(t)^2 \end{aligned}$$

où l'on a utilisé l'identité évidente $\bar{y}^2(t) = \frac{n(t_G)}{n(t)}\bar{y}^2(t_G) + \frac{n(t_D)}{n(t)}\bar{y}^2(t_D)$. De plus comme $n(t) = n(t_G) + n(t_D)$, on a

$$\begin{aligned}
& s^2(t) - \frac{n(t_G)}{n(t)}s^2(t_G) - \frac{n(t_D)}{n(t)}s^2(t_D) \\
&= \frac{n(t_G)}{n(t)}[\bar{y}(t_G)^2 + \bar{y}(t)^2] + \frac{n(t_D)}{n(t)}[\bar{y}(t_D)^2 + \bar{y}(t)^2] - 2\bar{y}(t)^2 \\
&= \frac{n(t_G)}{n(t)}[\bar{y}(t_G)^2 + \bar{y}(t)^2] + \frac{n(t_D)}{n(t)}[\bar{y}(t_D)^2 + \bar{y}(t)^2] \\
&\quad - 2\left(\frac{n(t_G)}{n(t)}\bar{y}(t_G) + \frac{n(t_D)}{n(t)}\bar{y}(t_D)\right)\bar{y}(t) \\
&= \frac{n(t_G)}{n(t)}[\bar{y}(t_G)^2 + \bar{y}(t)^2] - 2\frac{n(t_G)}{n(t)}\bar{y}(t_G)\bar{y}(t) \\
&\quad + \frac{n(t_D)}{n(t)}[\bar{y}(t_D)^2 + \bar{y}(t)^2] - 2\frac{n(t_D)}{n(t)}\bar{y}(t_D)\bar{y}(t) \\
&= \frac{n(t_G)}{n(t)}[\bar{y}(t_G) - \bar{y}(t)]^2 + \frac{n(t_D)}{n(t)}[\bar{y}(t_D) - \bar{y}(t)]^2,
\end{aligned}$$

qui est la **variance inter-nœuds**. Autrement dit, chercher à réduire l'impureté est équivalent à essayer de trouver des nœuds-enfants les plus différents possibles.

En classification, il nous faut définir une mesure d'impureté similaire à une variance, qui permet d'évaluer si la répartition des classes à l'intérieur d'un nœud est plus ou moins homogène. Si l'on voit les individus dans un nœud comme un échantillon d'une loi de probabilité discrète définie sur les classes, cela revient à chercher une quantité qui mesure à quel point une loi de probabilité discrète est *diffuse*. Il se trouve que le concept d'*entropie* répond parfaitement à cette question.

Définition 8 (Entropie d'une loi de probabilité discrète). L'**entropie** de Shannon d'une variable aléatoire discrète Y à valeurs dans $\{C_1, \dots, C_M\}$ est définie par

$$H(Y) = -\sum_{l=1}^M p_l \log(p_l)$$

où $p_l = \mathbb{P}(Y = C_l)$. De plus, elle possède les propriétés suivantes :

- $H(Y) \geq 0$
- $H(Y)$ est maximale si $\forall l = 1, \dots, M, p_l = 1/M$
- $H(Y) = 0$ si et seulement si $\exists l, p_l = 1$ et $\forall l' \neq l, p_{l'} = 0$.

L'estimateur empirique $\hat{H}(Y)$ de l'entropie de Shannon de Y obtenu à

partir d'un échantillon $\{y_i\}_{i=1}^n$ est donné par

$$\hat{H}(Y) = - \sum_{l=1}^M \hat{p}_l \log(\hat{p}_l)$$

où $\hat{p}_l = \frac{1}{n} \sum_{i=1}^n \mathbb{1}_{y_i=C_l}$ est la fréquence empirique de la classe C_l dans l'échantillon.

Par abus de notation, si on note $\hat{H}(t)$ l'entropie de Shannon empirique calculée sur les individus qui sont dans un nœud t , on voit alors qu'elle possède les propriétés attendues pour une mesure d'impureté : elle sera nulle si le nœud est pur (une seule classe présente), et maximale si le nœud est complètement impur (toutes les classes présentes avec la même nombre d'individus).

Définition 9 (Réduction d'impureté en classification avec entropie de Shannon). A partir de la mesure d'impureté égale à l'entropie de Shannon empirique, la réduction d'impureté en classification est donnée par

$$\Delta i_H(t) = \hat{H}(t) - \frac{n(t_G)}{n(t)} \hat{H}(t_G) - \frac{n(t_D)}{n(t)} \hat{H}(t_D).$$

Une autre mesure est également populaire pour quantifier la diversité d'une loi de probabilité discrète, c'est l'*indice de diversité de Gini*.

Définition 10 (Indice de diversité de Gini d'une loi de probabilité discrète). L'**indice de diversité de Gini**, ou indice de Gini, d'une variable aléatoire discrète Y à valeurs dans $\{C_1, \dots, C_M\}$ est définie par

$$I_G(Y) = \sum_{l=1}^M \sum_{k \neq l} p_l p_k = 1 - \sum_{l=1}^M p_l^2$$

où $p_l = \mathbb{P}(Y = C_l)$. De plus, il possède les propriétés suivantes :

- $I_G(Y) \geq 0$
- $I_G(Y)$ est maximal si $\forall l = 1, \dots, M, p_l = 1/M$
- $I_G(Y) = 0$ si et seulement si $\exists l, p_l = 1$ et $\forall l' \neq l, p_{l'} = 0$.

L'estimateur empirique $\hat{I}_G(Y)$ de l'indice de Gini de Y obtenu à partir d'un échantillon $\{y_i\}_{i=1}^n$ est donné par

$$\hat{I}_G(Y) = 1 - \sum_{l=1}^M \hat{p}_l^2$$

où $\hat{p}_l = \frac{1}{n} \sum_{i=1}^n \mathbb{1}_{y_i=C_l}$ est la fréquence empirique de la classe C_l dans l'échantillon.

Comme pour l'entropie, on notera $\hat{I}_G(t)$ l'indice de Gini empirique calculé sur les individus qui sont dans un nœud t , et il correspond bien à une mesure d'impureté grâce à ses propriétés.

Définition 11 (Réduction d'impureté en classification avec indice de Gini).

A partir de la mesure d'impureté égale à l'indice de Gini empirique, la réduction d'impureté en classification est donnée par

$$\Delta i_G(t) = \hat{I}_G(t) - \frac{n(t_G)}{n(t)} \hat{I}_G(t_G) - \frac{n(t_D)}{n(t)} \hat{I}_G(t_D).$$

Remarque (Lien entre Shannon et Gini : entropie de Tsallis). L'entropie de Shannon est historiquement la première à avoir été définie par Shannon en 1948. Mais des généralisations ont depuis été proposées, en particulier l'entropie de Tsallis, définie par

$$H_q(Y) = \frac{1}{q-1} \left(1 - \sum_{l=1}^M p_l^q \right)$$

pour $q \in \mathbb{R}$. On voit immédiatement que l'indice de Gini correspond à l'entropie de Tsallis pour $q = 2$, et l'entropie de Shannon est obtenue pour la limite $q \rightarrow 1$.

Après avoir introduit les deux mesures d'impureté les plus utilisées en classification, le dernier point à aborder dans cette section concerne leur évaluation en pratique. Toutes les deux font apparaître l'estimation de la probabilité de chaque classe dans un nœud, autrement dit $\hat{\mathbb{P}}(C_l|t)$ en suivant les notations introduites précédemment. Deux voies sont possibles :

- La vision fréquentiste, la plus courante (car similaire à ce qui se fait en régression), qui fait simplement appel à une fréquence empirique

$$\hat{\mathbb{P}}(C_l|t) = \frac{n_l(t)}{n(t)}.$$

- La vision Bayésienne, qui utilise la formule de Bayes

$$\begin{aligned} \hat{\mathbb{P}}(C_l|t) &= \frac{\hat{\mathbb{P}}(t|C_l) \hat{\pi}_l}{\hat{\mathbb{P}}(t)} \\ &= \frac{\frac{n_l(t)}{n_l} \hat{\pi}_l}{\sum_{k=1}^M \frac{n_k(t)}{n_l} \hat{\pi}_k}. \end{aligned}$$

Ici $\hat{\pi}_l$ est interprété comme une probabilité a priori d'être dans la classe C_l . Dans le cas où l'on choisit $\hat{\pi}_l = \frac{n_l}{n}$, on retrouve exactement la même formule que pour le cas fréquentiste. Mais il est possible d'utiliser d'autres formes d'a priori, telles que $\hat{\pi}_l = 1/M$ ou encore des $\hat{\pi}_l$ fixés par des connaissances d'experts.

Dans tous les cas, une fois la mesure d'impureté choisie, le critère de division de l'algorithme CART consiste à chercher, parmi toutes les divisions binaires d'une seule variable parallèles aux axes, celle qui amène à la plus grande réduction d'impureté.

Définition 12 (Critère de coupure CART). Pour une mesure d'impureté donnée $i(t)$ d'un nœud t , le **critère de division CART** est défini par le problème d'optimisation

$$\arg \max_t \max_{\text{div} \in \mathcal{Div}(t)} \Delta i(t)$$

où $\mathcal{Div}(t)$ est l'ensemble des divisions possibles du nœud t . Autrement dit, on identifie la meilleure division pour chaque nœud, et on divise le nœud qui amène à la meilleure réduction d'impureté.

Règles d'arrêt et élagage

Maintenant que nous avons vu comment l'algorithme CART procède par divisions successives, il convient de s'intéresser à la méthode employée pour arrêter le processus de division. Tout d'abord, si un nœud t est pur, alors $i(t) = 0$ et toute division supplémentaire n'amènera à aucune réduction d'impureté. Cela veut donc dire qu'à chaque itération, les nœuds candidats à une coupure sont seulement ceux qui ont une impureté $i(t) > 0$.

Mais laisser le processus se poursuivre jusqu'à ce que tous les nœuds soient purs amène généralement à une situation de grave sur-apprentissage, où tous les nœuds ne contiennent qu'un seul individu. En pratique on impose alors des contraintes supplémentaires pour limiter le nombre de divisions :

1. Un nœud n'est considéré comme candidat à la division que s'il contient un minimum d'individus (*minimum split* en anglais), typiquement 10 ou 20. Si aucun des nœuds ne satisfait cette condition, le processus s'arrête.
2. Un nœud terminal doit contenir un minimum d'individus (*minimum bucket* en anglais), typiquement une fraction du minimum split (1/3 par exemple).
3. On ne considère que les divisions qui amènent à une réduction d'impureté suffisamment grande, par exemple au moins 0.01. Si aucune division ne permet de réduire l'impureté d'au moins cette valeur, le processus s'arrête.
4. On limite la **profondeur** de l'arbre à une valeur maximum (*maximum depth* en anglais), par exemple 20 ou 30, où le nœud racine correspond à une profondeur égale à 0.

En pratique, toutes ces règles d'arrêt sont activées par défaut, et permette de construire ce que l'on appelle l'**arbre maximal**. Si elles permettent de contrôler la croissance de l'arbre pour qu'il atteigne une taille raisonnable, elles ne sont malheureusement pas suffisantes pour éviter complètement le sur-apprentissage. Il faut donc rajouter une étape supplémentaire dite d'*élagage*.

Définition 13 (Elagage d'un arbre CART). L'**élagage** d'un arbre CART (*pruning* en anglais) consiste à couper les branches les moins informatives et les plus longues. En pratique, elle s'effectue en deux étapes :

- Construction d'une séquence de sous-arbres emboîtés T_j , $j \geq 1$ de l'arbre maximal, qui ont un *coût-complexité* minimum.
Le coût-complexité d'un sous-arbre T est défini par

$$C_\alpha(T) = \hat{C}(T) + \alpha|\tilde{T}|$$

où $\alpha > 0$ est un paramètre de complexité, $\hat{C}(T)$ est la perte associée au sous-arbre (par exemple calcul du risque par validation-croisée si l'on utilise comme prédicteur le sous-arbre T) et $|\tilde{T}|$ est le nombre de nœuds terminaux de T .

- Parmi cette séquence, sélection du sous-arbre qui a la perte la plus faible, calculée par validation-croisée.

En pratique donc, vous utiliserez donc systématiquement une procédure de validation-croisée lors de la construction d'un arbre CART, à la fois pour potentiellement régler les hyperparamètres des règles d'arrêt, mais aussi obligatoirement pour mettre en œuvre l'élagage de l'arbre maximal.

Choisir des divisions légèrement moins bonnes ?

Précédemment nous avons vu dans la Définition 12 que l'algorithme CART va, à chaque itération de sa construction, tester toutes les divisions possibles et sélectionner la meilleure d'entre elles. En particulier, nous pouvons sauvegarder la réduction d'impureté de toutes les divisions possibles, et celles qui vont nous intéresser ici sont celles qui font intervenir des variables explicatives différentes de celle qui apparaît dans la meilleure division. Pourquoi s'intéresser à celles-ci ? Tout simplement car il y a des situations pratiques où choisir la meilleure division n'est pas forcément possible ou est plus coûteux :

- **Valeurs manquantes et divisions équidivisantes** (*surrogate* en anglais). Il arrive que dans le jeu de données certaines valeurs des variables explicatives soient manquantes. Si dans un tel cas il est toujours possible de calculer les mesures d'impureté (en ne prenant pas en compte ces individus dans le calcul des fréquences ou variances empiriques), et donc d'identifier la meilleure coupure, il faut ensuite affecter les individus du nœud qu'il faut diviser aux nœuds-enfants pour poursuivre la construction de l'arbre. Or si la variable explicative impliquée dans cette division est manquante pour un des individus du nœud cette affectation n'est alors pas possible. Pour poursuivre la construction de l'arbre malgré tout, une possibilité est d'identifier parmi toutes les divisions possibles celles qui auraient amené à une répartition dans les nœuds-enfants la plus similaire à la meilleure, et qui font intervenir une autre variable explicative dont la valeur n'est pas manquante pour l'individu d'intérêt. Ces divisions sont dites équidivisantes, ou de substitution. Elles sont utilisées pour affecter un individu lors de la construction de l'arbre, mais sont aussi particuliè-

rement pratiques lors de la phase de prédiction dans le cas où l'on doit prédire un nouvel individu avec des valeurs manquantes.

- **Variables explicatives coûteuses à mesurer et divisions équiréductrices** (*competing* en anglais). Une autre situation que vous pourrez rencontrer en pratique est celle où les valeurs de certaines variables explicatives sont difficiles ou coûteuses à obtenir. Dans le domaine médical par exemple, si certaines variables ne peuvent être mesurées que par des procédures invasives sur un patient, ou à l'aide d'instruments de mesure très chers. Dans le monde industriel également, il arrive fréquemment que certaines variables ne puissent être mesurées que par des dispositifs d'instrumentation coûteux. Dans de tels cas, si certaines des divisions identifiées comme optimales lors de la construction font intervenir une variable explicative coûteuse, cela implique qu'en phase de prédiction il faudra nécessairement enregistrer la valeur de ces variables explicatives pour tout nouvel individu à prédire. Mais il est tout à fait possible qu'à chaque itération de l'arbre, la deuxième (ou la troisième, etc) meilleure division ait impliqué une autre variable explicative peu coûteuse, avec une réduction d'impureté moins bonne mais proche de l'optimale. Il peut donc y avoir un intérêt à utiliser une telle division beaucoup moins coûteuse pour la construction : c'est ce qu'on appelle une division équiréductrice.

Affectation des nœuds terminaux - Prédicteur final

Après construction de l'arbre CART, les nœuds terminaux, ou feuilles, correspondent au découpage final de l'espace $\mathcal{X}$ en régions disjointes. La dernière étape pour construire le prédicteur final est d'affecter une valeur à chacune des feuilles, qui seront utilisées pour prédire les individus qui arriveront dans les feuilles lors de la phase de prédiction.

- En **régression**, l'affectation optimale consiste à calculer la moyenne empirique $\bar{y}(t)$ de la sortie des individus de l'ensemble d'apprentissage qui sont dans une feuille t , comme indiqué au début de cette section.
- En **classification**, nous avons également mentionné que l'affectation optimale est obtenue avec la classe qui a la plus grande fréquence empirique dans une feuille. Cependant en pratique, à la place de la fréquence empirique dans un nœud terminal t , on utilise les estimateurs $\hat{\mathbb{P}}(C_l|t)$. Dans le cas fréquentiste cela est équivalent, mais pas forcément dans le cas Bayésien.
- En **classification** toujours, vous pourrez rencontrer des situations où il peut être plus dangereux ou coûteux de mal prédire certaines classes. Imaginez par exemple un cas où vous devez prédire si un champignon est mortel ou non (ici il vaut mieux se tromper en prédisant par erreur qu'un champignon est mortel alors qu'il ne l'est pas réellement, plutôt que l'inverse!), ou des cas où les prédictions concernent des diagnostics de maladies graves. On peut alors introduire dans l'affectation finale un **coût différentiel de mauvais classement** via un coût d'erreur de classement $c(\cdot|\cdot)$ défini a priori :

- $c(l|k)$ est le coût de l'affectation d'un individu à la classe C_l alors qu'il est de classe C_k .
- $c(l|l) = 0$.
- Un cas particulier est donné par $c(l|k) = 1, \forall l \neq k$ et $\forall k = 1, \dots, M$.

A l'aide de ce coût d'erreur $c(\cdot|\cdot)$, pour chaque feuille t on peut calculer le **coût d'affectation**

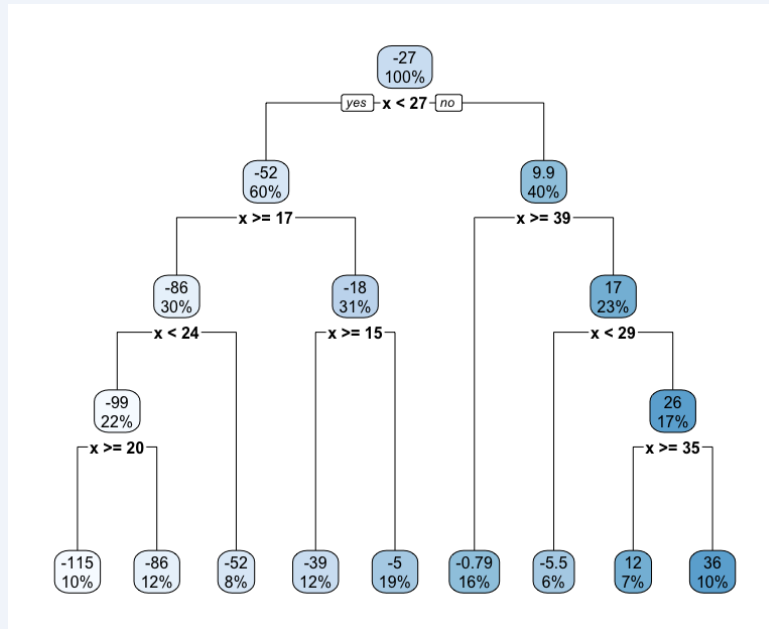
$$CM_l(t) = \sum_{k=1}^M c(l|k) \hat{\mathbb{P}}(C_k|t)$$

et on affecte au final à t la classe qui a le coût d'affectation le plus faible (au lieu de $\hat{\mathbb{P}}(C_k|t)$ la plus grande). Si l'on utilise des valeurs de coût unitaires comme dans le cas particulier décrit précédemment, on retombe sur l'affectation usuelle.

Exemples

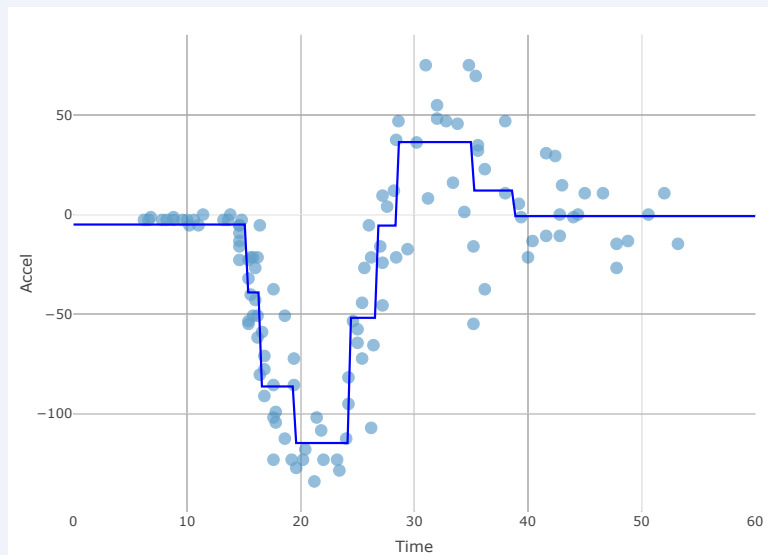
Maintenant que nous avons détaillé toutes les étapes de l'algorithme CART, nous pouvons finalement étudier les exemples du chapitre précédent pour illustrer son comportement.

Exemple (CART en régression). On reprend le jeu de données `mcycle` du package `MASS` en R, et on représente l'arbre CART obtenu avec les réglages par défaut du package `rpart`, tracé à l'aide du package `rpart.plot`.



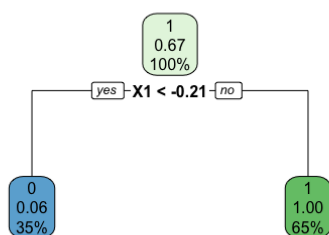
Dans cette représentation, vous pouvez visualiser toutes les divisions, et dans chaque nœud sont données la moyenne empirique des individus qui le

composent, et la proportion de l'échantillon d'apprentissage qu'il contient. Avec cet arbre, on peut alors faire des prédictions, qui sont représentées ci-dessous.

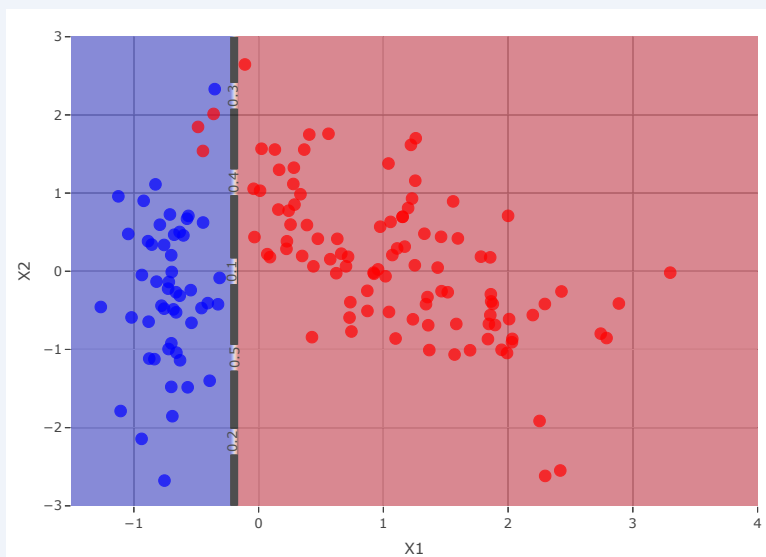


Vous pourrez vérifier qu'on obtient bien une approximation constante par morceaux.

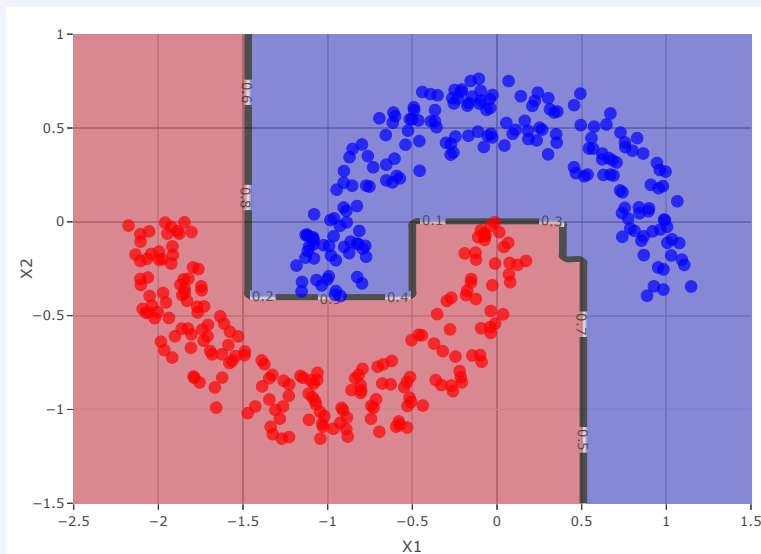
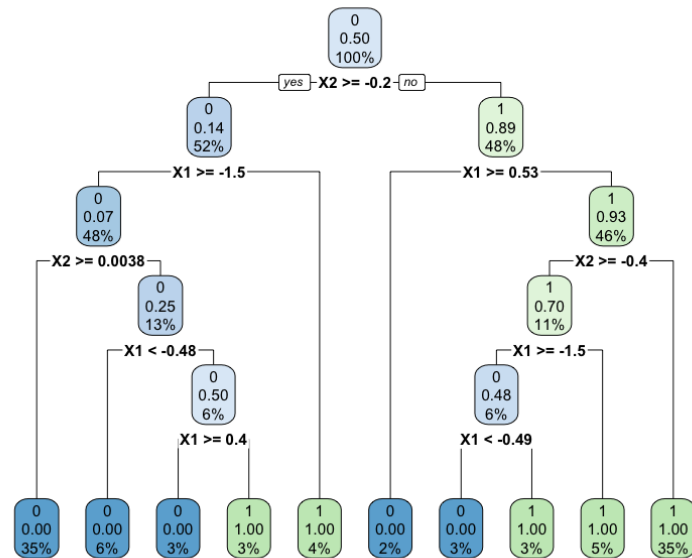
Exemple (CART en classification). On commence par visualiser l'arbre CART construit sur le premier exemple du chapitre précédent, où la classification est facile.



L'arbre est extrêmement simple, avec une seule division. Pour la classification, le package **rpart.plot** affiche la classe affectée pour chaque nœud, la probabilité estimée de la classe 1, et la proportion de l'échantillon d'apprentissage qu'il contient. On trace ensuite les lignes de niveau de la fonction de décision, et comme il n'y a qu'une seule coupure l'espace n'est divisé qu'en deux régions.



Sur le jeu de données "Deux Lunes", qui est plus difficile, voici le résultat.



Avantage et inconvénients des arbres CART

Pour terminer cette section, nous listons de manière succincte les avantages et inconvénients des arbres CART.

- **Avantages :**

1. Résultats et prédictions explicites (les conditions sont formulées sur les variables d'origine, lecture graphique simple)
2. Modèle non-paramétrique, sans hypothèse sur les lois suivies par les variables explicatives

3. Tous les types de variables sont traités de manière naturelle
4. Traitement facile des valeurs manquantes
5. L'arbre n'est pas modifié par transformation monotone des variables explicatives

• **Inconvénients :**

1. Optimisation gloutonne, qui risque de trouver un optimum local et non global
2. Optimisation gloutonne, qui entraîne un manque de robustesse (en perturbant légèrement le jeu de données, on peut aboutir à un arbre très différent - vous illustrerez ce phénomène en travaux pratiques - on dit qu'ils sont instables)
3. Limitation à des régions rectangulaires

3.2 Forêts aléatoires

Dans le but de limiter les inconvénients des arbres CART tout en préservant leurs avantages (à l'exception du premier), Leo Breiman, l'inventeur de CART, a aussi proposé l'un des modèles d'apprentissage supervisé les plus connus à l'heure actuelle, les forêts aléatoires (*random forests* en anglais, abrégé en RF dans la suite). Comme le nom l'indique, une forêt aléatoire est tout simplement un ensemble d'arbres (construits avec une pincée d'aléa additionnelle).

Motivation pour les méthodes d'ensemble

Les forêts aléatoires sont une illustration de ce que l'on appelle une *méthode d'ensemble*, terme qui désigne toutes les méthodes d'apprentissage statistique qui utilisent plusieurs estimations pour construire un nouveau modèle qui agrège toutes les prédictions. L'espoir est qu'en combinant de nombreux modèles, le modèle final pourra atteindre une meilleure précision. Cette proposition s'appuie sur l'idée générale de la "Sagesse des foules" : la connaissance collective d'un groupe diversifié et indépendant de personnes dépasse généralement la connaissance de n'importe quel individu seul. Dans notre contexte, un individu est un modèle d'apprentissage statistique, la foule est un ensemble de modèles d'apprentissage que nous voulons diversifiés et indépendants. Nous verrons ci-dessous comment les forêts aléatoires en particulier atteignent la diversité et l'indépendance, ou du moins visent à les atteindre. Mais analysons d'abord comment le concept général de sagesse des foules se traduit de manière mathématique.

Désignons $g_1(\mathbf{x}), \dots, g_B(\mathbf{x})$ un ensemble de B prédicteurs de base. Pour la **régression** d'abord, définissons un nouveau prédicteur d'ensemble tout simplement en prenant la moyenne de tous les prédicteurs :

$$\bar{g}_B(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B g_b(\mathbf{x}).$$

Nous pouvons étudier les performances de $\bar{g}_B$ en examinant son biais et sa variance, en faisant une **supposition peu réaliste**, mais qui sert d'illustration :

chaque g_b est une variable aléatoire (car il est entraîné sur un échantillon aléatoire = l'ensemble d'entraînement) et nous supposons que tous les $g_1, \dots, g_B$ sont indépendants et identiquement distribués. Dans ce cadre, nous avons que

- Biais. $\mathbb{E}(\bar{g}_B(\mathbf{x})) = \mathbb{E}(g_b(\mathbf{x}))$
- Variance. $\text{Var}(\bar{g}_B(\mathbf{x})) = \frac{1}{B} \text{Var}(g_b(\mathbf{x}))$

En d'autres termes, le biais du nouveau prédicteur reste inchangé, mais sa variance est beaucoup plus faible par rapport aux prédicteurs initiaux.

De même, pour la **classification** binaire, le classifieur d'ensemble s'écrit

$$\bar{g}_B(\mathbf{x}) = \mathbb{1} \left(\frac{1}{B} \sum_{b=1}^B g_b(\mathbf{x}) > 0.5 \right).$$

Supposons que chaque classifieur initial a une erreur indépendante $\varepsilon < 0.5$ pour prédire la décision correcte $Y = 1$, c'est-à-dire $\mathbb{P}(g_b(\mathbf{x}) \neq 1) = \varepsilon < 0.5$. Considérons maintenant l'inégalité de Hoeffding quand elle est appliquée à des variables de Bernoulli B_i , $i = 1, \dots, n$:

$$\mathbb{P} \left(\mathbb{E}(B_i) - \frac{1}{n} \sum_{i=1}^n B_i > t \right) < \exp(-2nt^2).$$

On peut alors montrer que

$$\begin{aligned} \mathbb{P}(\bar{g}_B(\mathbf{x}) \neq 1) &= \mathbb{P} \left(\frac{1}{B} \sum_{b=1}^B g_b(\mathbf{x}) < 0.5 \right) \\ &= \mathbb{P} \left(1 - \varepsilon - \frac{1}{B} \sum_{b=1}^B g_b(\mathbf{x}) > 0.5 - \varepsilon \right) \\ &= \mathbb{P} \left(\mathbb{E}(g_b(\mathbf{x})) - \frac{1}{B} \sum_{b=1}^B g_b(\mathbf{x}) > 0.5 - \varepsilon \right) \\ &< \exp \left(-\frac{1}{2} B (1 - 2\varepsilon)^2 \right), \end{aligned}$$

qui est inférieur à ε pour B suffisamment grand, signifiant que le classifieur d'ensemble a une erreur plus faible que les classifieurs initiaux.

Bien sûr, nous avons dû utiliser une supposition irréaliste pour obtenir ces résultats, car tous les prédicteurs initiaux sont construits à partir du même échantillon d'entraînement. Cependant, cela illustre le genre d'avantages que nous espérons obtenir en utilisant les méthodes d'ensemble. Maintenant, en pratique, il faut essayer de construire des prédicteurs aussi diversifiés et indépendants que possible.

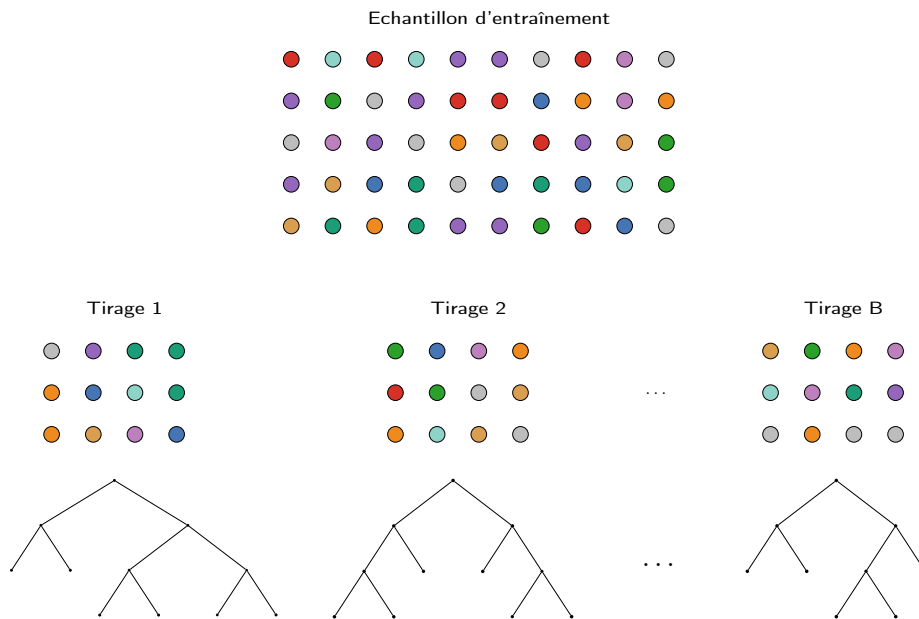
Principe général des forêts aléatoires

Les forêts aléatoires sont basées sur 3 ingrédients fondamentaux, qui visent à atteindre la diversité et l'indépendance mentionnées précédemment :

1. Les prédicteurs combinés sont des **arbres CART**, **typiquement non élagués**.

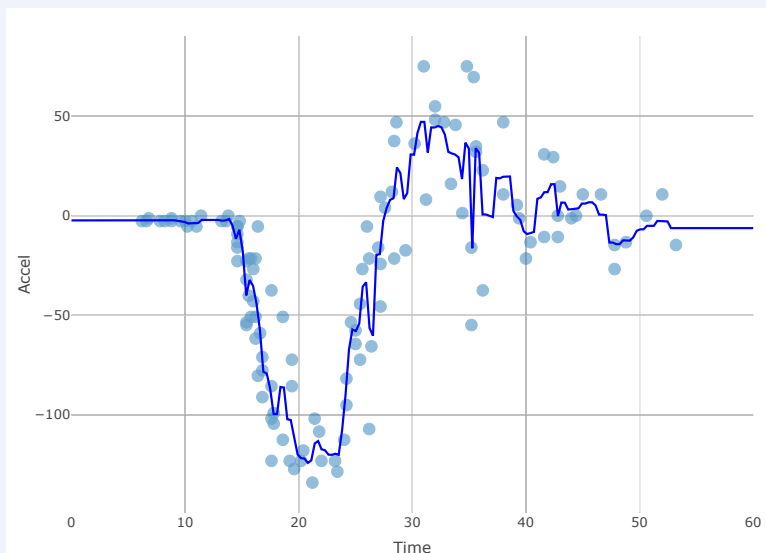
2. Chaque arbre est entraîné sur un **jeu de données différent**, obtenu par tirage aléatoire avec remise de l'ensemble d'entraînement (c'est le *bootstrap*).
3. Lors de la construction de chaque arbre, et à chaque itération, le critère CART n'est pas optimisé pour toutes les coupures possibles, mais seulement sur celles faisant intervenir des **variables explicatives tirées au hasard**.

Les ingrédients 1. et 2., quand on les combine, profitent en fait de l'instabilité des arbres mentionnée ci-dessus pour atteindre plus de diversité et d'indépendance. Si l'on se limite à ces deux ingrédients seulement, on obtient ce que l'on appelle du *bagging d'arbres* (bagging étant une contraction de *bootstrap aggregating*). Les forêts aléatoires introduisent en plus l'ingrédient 3., qui permet d'augmenter encore plus l'indépendance. Le principe général est illustré dans la figure ci-dessous.

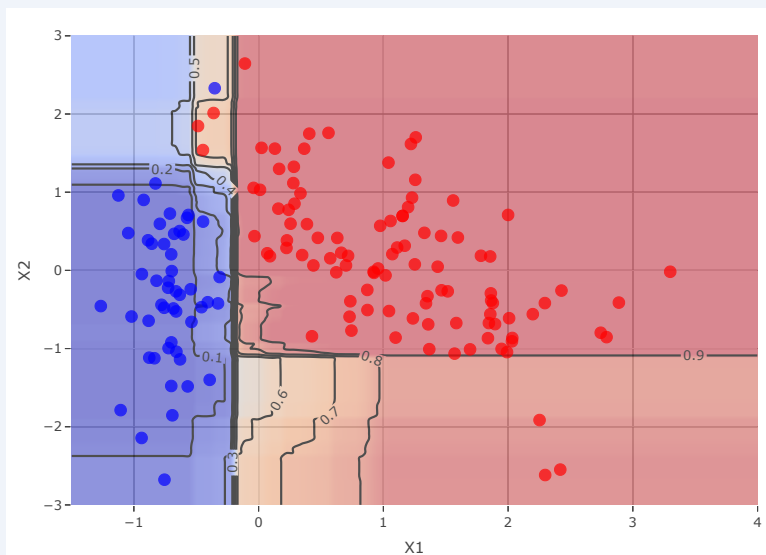


D'un point de vue pratique, le nombre d'arbres B (i.e. de tirages) ainsi que le nombre de variables explicatives choisies aléatoirement sont des hyperparamètres, que l'on peut estimer par validation croisée. Leurs valeurs par défaut usuelles sont cependant assez robustes, par exemple $B = 500$ ou $B = 1000$, et un nombre de variables explicatives (mtry dans les librairies) $mtry = \sqrt{d}$ en classification et $mtry = d/3$ en régression. Avec ces hyperparamètres par défaut, nous illustrons le comportement d'une forêt aléatoire sur les exemples habituels.

Exemple (RF en régression). On reprend le jeu de données `mcycle` du package **MASS** en R, et on entraîne une forêt aléatoire avec le package **ranger**. Les prédictions sont représentées ci-dessous.

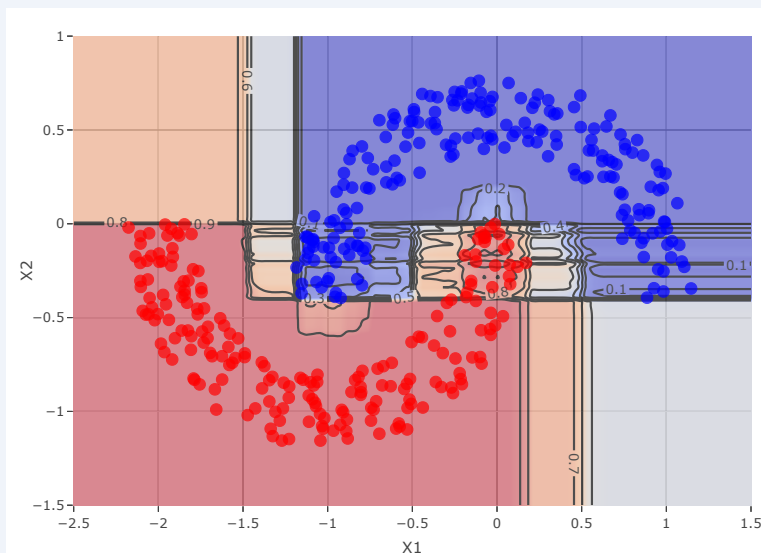


Exemple (RF en classification). On commence par visualiser les lignes de niveau de la fonction de décision de la forêt aléatoire sur le premier exemple du chapitre précédent, où la classification est facile.



Sur le jeu de données "Deux Lunes", qui est plus difficile, voici le ré-

sultat.



Estimation de l'erreur - Les échantillons Out-Of-Bag (OOB)

Une particularité intéressante des forêts aléatoires est que la procédure de tirage des échantillons par bootstrap implique que les individus du jeu d'entraînement ne sont pas utilisés dans la construction de tous les arbres, mais seulement dans certains d'entre eux. On parle d'échantillons *Out-Of-Bag* (OOB). Cela veut donc dire qu'il est possible de construire une estimation du risque empirique calculé sur un échantillon test, sans avoir d'échantillon test ! En effet, pour rappel, à l'aide d'un échantillon test $\mathcal{D}_n^{\text{test}} = \{\mathbf{x}_i^*, y_i^*\}_{i=1, \dots, n^{\text{test}}}$, l'erreur de généralisation du prédicteur $\bar{g}_B$ d'une forêt aléatoire est donnée par

$$\hat{R}_{\mathcal{D}_n^{\text{test}}}(\bar{g}_B) = \frac{1}{n^{\text{test}}} \sum_{i=1}^{n^{\text{test}}} l \left(y_i^*, \frac{1}{B} \sum_{b=1}^B g_b(\mathbf{x}_i^*) \right).$$

En se servant des échantillons OOB, on peut construire une erreur similaire à l'aide de l'échantillon d'entraînement $\mathcal{D}_n^{\text{train}} = \{\mathbf{x}_i, y_i\}_{i=1, \dots, n^{\text{train}}}$ uniquement :

$$\hat{R}_{\text{OOB}}(\bar{g}_B) = \frac{1}{n^{\text{train}}} \sum_{i=1}^{n^{\text{train}}} l \left(y_i, \frac{1}{|B_{-i}|} \sum_{b \in B_{-i}} g_b(\mathbf{x}_i) \right)$$

où $B_{-i} \subset \{1, \dots, B\}$ désigne le sous-ensemble des arbres qui n'ont pas été entraînés avec le i -ième individu de l'ensemble d'apprentissage.

Vous pourrez trouver un certain nombre d'études où l'erreur OOB est utilisée pour valider la qualité finale d'une forêt aléatoire, cependant il est important de garder en tête qu'il arrive assez souvent qu'elle sous-estime l'erreur de généralisation : il est donc conseillé de plutôt toujours conserver un échantillon de test séparé pour la calculer comme vous avez l'habitude de le faire. En revanche, une utilisation alternative intéressante (d'un point de vue temps de

calcul) est de se servir de cette erreur OOB pour estimer les hyperparamètres de la forêt aléatoire, au lieu d'utiliser la validation croisée. En effet dans ce cas la sous-estimation de l'erreur est moins préjudiciable, puisqu'ici le réglage des hyperparamètres nécessite juste d'avoir une métrique de comparaison de leurs valeurs de façon relative et non absolue.

Importance de variables

Un dernier aspect des forêts aléatoires, que vous rencontrerez très certainement si vous les utilisez en stage ou en projet, est relié à ce que l'on appelle l'importance de variables. Derrière ce terme générique se cache un thème de recherche extrêmement vaste et diversifié des statistiques, qui s'intéresse à mesurer si une variable explicative est importante pour prédire la variable cible. Il existe toute une zoologie de mesures de ce type, avec une myriade d'avantages, d'inconvénients, et de cadres d'application pour chacune d'entre elles.

Quand on parle d'importance de variables pour les forêts aléatoires, on fait référence à deux mesures quantitatives de l'effet d'une variable explicative sur la sortie :

1. **Mean Decrease in Impurity (MDI).** C'est la mesure la plus répandue et souvent celle sélectionnée par défaut dans les bibliothèques de forêts aléatoires (en Python et R). Elle se base sur une intuition assez simple : si une variable explicative est importante pour expliquer / prédire la sortie, alors quand elle est sélectionnée pour une coupure dans un des arbres elle devrait faire beaucoup décroître l'impureté. On calcule alors pour chaque variable explicative la réduction d'impureté qu'elle a engendrée (*Decrease in Impurity*), en moyenne sur tous les nœuds et tous les arbres où elle a été impliquée (*Mean*). Plus formellement, la version non-normalisée de la MDI pour la variable explicative $X^{(j)}$ s'écrit

$$\text{MDI}(X^{(j)}) = \frac{1}{B} \sum_{b=1}^B \sum_{\text{nœud } k \in T_b | \text{coupure}(k)=X^{(j)}} p_{T_b}(k) \Delta i_{T_b}(k)$$

où T_b désigne le b -ième arbre, $p_{T_b}(k)$ est la proportion de l'échantillon dans le nœud k de l'arbre T_b et $\Delta i_{T_b}(k)$ est la réduction d'impureté observée lors de la coupure du nœud k de l'arbre T_b . En pratique, après avoir calculé la MDI pour toutes les variables explicatives, elles sont normalisées pour avoir une somme égale à 1.

2. **Mean Decrease Accuracy (MDA).** L'idée principale de cette mesure repose sur le fait que si une variable explicative n'est pas importante pour prédire la cible, qu'on la prenne en compte ou non dans la construction de la forêt ne devrait pas changer l'erreur de généralisation. On pourrait donc calculer la décroissance de l'erreur de généralisation due à la suppression d'une des variables explicatives (*Decrease Accuracy*) moyennée sur un échantillon de test (*Mean*) :

$$\text{MDA}(X^{(j)}) = \frac{1}{n^{\text{test}}} \sum_{i=1}^{n^{\text{test}}} l \left(y_i^*, \frac{1}{B} \sum_{b=1}^B g_b^{(-j)}(\mathbf{x}_i^*) \right) - l \left(y_i^*, \frac{1}{B} \sum_{b=1}^B g_b(\mathbf{x}_i^*) \right)$$

où $g_b^{(-j)}$ désigne la forêt aléatoire construite sans la j -ième variable explicative. Mais en pratique ce calcul implique de construire plusieurs fois la forêt aléatoire, en supprimant tour à tour chacune des variables explicatives. Pour des raisons de temps de calcul, une astuce est alors très souvent utilisée à la place : la permutation aléatoire. On entraîne une seule et unique forêt, et dans la phase de prédiction des individus de l'échantillon de test on calcule (1) leur prédiction usuelle, (2) leur prédiction quand on effectue une permutation aléatoire des valeurs d'une variable explicative et (3) la décroissance de l'erreur de généralisation entre ces deux prédictions. Une telle permutation aléatoire des observations "casse" la dépendance statistique qui aurait pu exister entre la variable explicative et la sortie, et mime donc ce qu'on aurait obtenu en supprimant dès le départ la variable explicative. Vous verrez donc également la dénomination **Permutation Importance** pour cette méthode. Enfin, dans l'immense majorité des librairies, le calcul ne se fait pas sur un échantillon de test séparé, mais à l'aide des échantillons OOB décrits précédemment.

Enfin, il est important de noter que ces deux méthodes d'importance de variables **souffrent de très forts biais** et sont donc à analyser avec recul et prudence. Par exemple, elles sont toutes les deux extrêmement sensibles aux variables explicatives catégorielles avec beaucoup de modalités (i.e. elles vont souvent ressortir comme étant beaucoup plus influentes qu'elles ne le sont en réalité). Elles sont également fortement biaisées quand les variables explicatives sont dépendantes entre elles, en particulier quand la relation avec la variable cible fait intervenir des interactions.

Séance 5 : Compléments sur la comparaison de modèles en classification

10 Octobre, 2023

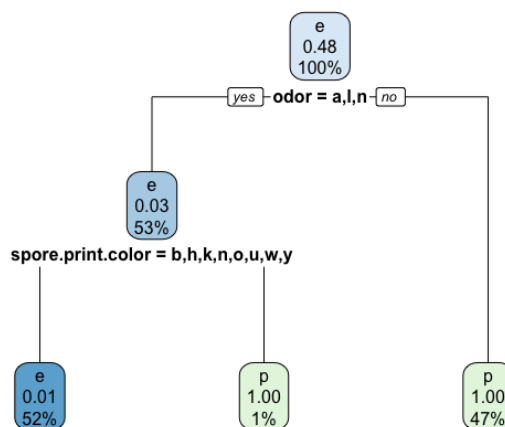
- Matrice de confusion et erreurs asymétriques
- Prise en compte de l'asymétrie dans les arbres
- Approche générale pour les méthodes à base de score, courbes ROC

Chapitre 4

Compléments sur la comparaison de modèles en classification

Dans ce chapitre, nous nous focalisons sur les problèmes de classification (binaire en particulier), pour aborder un aspect mentionné très rapidement dans le chapitre précédent : les erreurs de classification asymétrique. Pour illustration tout au long des sections, nous nous baserons sur un exemple fil rouge de classification binaire décrit ci-dessous.

Exemple (Classification de champignons). On dispose d'un ensemble de données sur les champignons. Il est constitué de 8124 observations pour lesquelles diverses descriptions sont disponibles comme la surface, l'odeur, la couleur, etc, ainsi que l'information : comestible ("e", pour *eatable*) ou poison ("p"). On cherche à construire un classifieur capable de différencier les champignons comestibles des non-comestibles à partir uniquement de leurs caractéristiques. Voici par exemple l'arbre CART construit sur un échantillon d'apprentissage de taille 6093, avec les paramètres par défaut.



Sur l'échantillon de test, cet arbre commet une erreur de généralisation autour de 0.6%, ce qui est très faible, et l'on pourrait à première vue s'en contenter. Cependant, quand on regarde de plus près les prédictions, on se rend compte qu'un certain nombre de champignons sont prédits comme étant comestibles, alors qu'ils ne le sont pas. Si un tel classifieur était utilisé dans la réalité pour informer des gens sur la dangerosité de champignons, un tel risque ne serait pas acceptable !

Nous avons discuté au chapitre précédent de comment il est possible d'adapter l'algorithme CART à l'aide de coûts différenciés, nous y reviendrons plus tard dans ce chapitre.

L'exemple précédent montre qu'en classification, au-delà d'évaluer l'erreur de généralisation globale, il convient également de faire une analyse plus fine de la concordance entre les prédictions du classifieur et les vraies valeurs cible. L'outil central pour cette étape est la **matrice de confusion**, que nous détaillons dans la section suivante. La matrice de confusion permet d'analyser les résultats pour un classifieur donné, mais ne donne pas le cadre pour le modifier si l'on souhaite se prémunir contre certaines erreurs de prédiction plus graves que d'autres (comme dans l'exemple des champignons). Pour les arbres CART, cela est possible grâce à l'introduction de coûts différenciés, sur lesquels nous nous attarderons plus longuement dans ce chapitre. Enfin, il existe un cadre générique pour tous les classifieurs qui permet d'opérer une modification des prédictions de manière continue et évolutive, qui aboutit à un concept clef en classification, la **courbe ROC**. La courbe ROC permet d'une part d'évaluer un seul classifieur quand on le modifie, mais surtout et également de comparer des classifieurs entre eux. Elle est introduite dans la

dernière section de ce chapitre.

4.1 Définition d'une matrice de confusion

La **matrice de confusion** est définie pour un classifieur fixé, et permet d'analyser plus finement ses performances sur **échantillon de test**. Nous allons la détailler dans le cas de la classification binaire uniquement. Les deux classes possibles pour la sortie sont, dans le cas de la matrice de confusion, d'abord renommées en "Positif" et "Négatif", où en général la classe des "Positif" correspond à une catégorie d'intérêt particulier pour le problème, et à qui très souvent on aura déjà attribué la classe 1 (versus 0 pour l'autre, qui est moins critique dans l'étude). Sur l'exemple précédent des champignons, cela sera les champignons non-comestibles. Une fois ce renommage effectué, la matrice de confusion ne fait que comparer sur l'échantillon test, les prédictions aux vraies valeurs observées pour les deux classes "Positif" et "Négatif". Ce comptage est représenté sous la forme d'une matrice, représentée ci-dessous. Il est important de retenir la dénomination de chacune des cases de cette matrice.

	Positifs Prédits	Négatifs Prédits	Total
Positifs Observés	Vrais positifs <i>True Positive (TP)</i>	Faux Négatifs <i>False Negative (FN)</i>	Total Observés Positifs
Négatifs Observés	Faux Positifs <i>False Positive (FP)</i>	Vrais Négatifs <i>True Negative (TN)</i>	Total Observés Négatifs
Total	Total Prédits Positifs	Total Prédits Négatifs	Total Echantillon Test

On parle de "confusion" car cette représentation met en évidence immédiate les deux types d'erreur différents que le classifieur peut commettre : les **Faux Positifs**, c'est-à-dire les individus prédits comme étant de la classe "Positif" alors qu'ils sont réellement "Négatif", et les **Faux Négatifs**, qui correspondent aux individus prédits de classe "Négatif" alors qu'ils sont en réalité "Positif". En pratique, la matrice de confusion est construite en évaluant les effectifs de toutes les cases, comme ci-dessous.

	Positifs Prédits	Négatifs Prédits	Total
Positifs Observés	n_{11} <i>True Positive (TP)</i>	n_{10} <i>False Negative (FN)</i>	$n_{11} + n_{10}$
Négatifs Observés	n_{01} <i>False Positive (FP)</i>	n_{00} <i>True Negative (TN)</i>	$n_{01} + n_{00}$
Total	$n_{11} + n_{01}$	$n_{10} + n_{00}$	n

Remarque ([⚠](#) Différence selon les packages). Faites bien attention lorsque vous utilisez des packages R ou Python pour calculer et visualiser une matrice de confusion, il arrive que les lignes et les colonnes soient inversées (Prédictions en ligne et Observations en colonne, à la place de la représentation ci-dessus). Mais cela sera toujours précisé, il vous suffira d'être attentifs. De la même façon, quand vous tracerez une matrice de confusion,

il faudra toujours préciser quelle est la classe qui est considérée comme "Positif".

A l'aide de la matrice de confusion, on définit alors les grandeurs d'intérêt suivantes :

- Le nombre de Vrais Positifs VP (*True Positive (TP)*) : n_{11}
- Le nombre de Vrais Négatifs VN (*True Negative (TN)*) : n_{00}
- Le nombre de Faux Positifs FN (*False Positive (FP)*) : n_{01}
- Le nombre de Faux Négatifs FN (*False Negative (FN)*) : n_{10}

A partir de ces grandeurs, on peut construire des **indicateurs de qualité de classification** :

- **Taux de bien classés.** C'est l'estimateur de $\mathbb{P}(\hat{Y} = Y)$, obtenu par la formule $(n_{11} + n_{00})/n$.
- **Taux d'erreur.** C'est l'estimateur de $\mathbb{P}(\hat{Y} \neq Y)$, c'est-à-dire le risque empirique, obtenu par la formule $(n_{01} + n_{10})/n$.
- **Sensibilité ou Taux de Vrais Positifs (TVP).** C'est l'estimateur de la **sensibilité** $\mathbb{P}(\hat{Y} = 1|Y = 1)$, obtenu par la formule $n_{11}/(n_{11} + n_{10})$. En anglais, c'est le *True Positive Rate* (TPR).
- **Taux de Faux Positifs (TFP).** C'est l'estimateur de $\mathbb{P}(\hat{Y} = 1|Y = 0)$, obtenu par la formule $n_{01}/(n_{01} + n_{00})$. En anglais, c'est le *False Positive Rate* (FPR).
- **Spécificité ou 1-TFP.** C'est l'estimateur de la **spécificité** $\mathbb{P}(\hat{Y} = 0|Y = 0) = 1 - \mathbb{P}(\hat{Y} = 1|Y = 0)$, obtenu par la formule $n_{00}/(n_{01} + n_{00})$.

La sensibilité, qui estime la probabilité de bien détecter un positif, et la spécificité, qui estime celle de bien détecter un négatif, sont des indicateurs majeurs en classification. Bien évidemment vous voudriez qu'ils soient tous les deux les plus proches de 100%, et en particulier pouvoir modifier un classifieur de manière à les améliorer tous les deux. Mais malheureusement, pour un classifieur donné, vous ne pourrez atteindre ces deux objectifs car ils sont contradictoires (*no free lunch*). Il faudra donc faire un compromis, qui sera par exemple guidé par le problème (penser par exemple au jeu de données des champignons, où si la classe 1 est "poison" on préférera bien entendu maximiser le plus possible la sensibilité au détriment de la spécificité). La courbe ROC que nous aborderons à la fin n'est en réalité qu'une visualisation de tous les compromis possibles.

Exemple (Classification de champignons - suite). Sur le jeu de données précédent et pour l'arbre CART construit au-dessus, voici la matrice de confusion en considérant que la classe "p" représente les positifs. Elle permet de vérifier que 12 champignons sont prédits comme étant comestibles, alors qu'ils ne le sont pas (Faux Négatifs FN).



A l'aide du package `caret` et de la fonction "confusionMatrix", il est également possible de calculer tous les indicateurs introduits précédemment. En particulier, cet arbre CART a une sensibilité égale à 0.9877 et une spécificité de 1, valeurs que vous pouvez vérifier par simple calcul sur la matrice de confusion.

4.2 Coût différencié pour la construction d'un arbre

Nous avons vu au chapitre précédent qu'il était possible de changer l'affectation d'un nœud terminal d'un arbre pour prendre en compte une asymétrie dans les erreurs de prédiction. Nous avons discuté du coût différentiel de mauvais classement, défini via un coût d'erreur de classement $c(\cdot|\cdot)$ défini a priori :

- $c(l|k)$ est le coût de l'affectation d'un individu à la classe C_l alors qu'il est de classe C_k .
- $c(l|l) = 0$.

A l'aide de ce coût d'erreur $c(\cdot|\cdot)$, pour chaque feuille t on peut calculer le coût d'affectation

$$CM_t(t) = \sum_{k=1}^M c(l|k) \hat{\mathbb{P}}(C_k|t)$$

où $\hat{\mathbb{P}}(C_k|t)$ est l'estimation de la probabilité de la classe C_k dans le nœud t . Au final, on affecte à t la classe qui a le coût d'affectation le plus faible, au lieu de

$\hat{\mathbb{P}}(C_k|t)$ la plus grande quand les coûts sont symétriques.

Cependant, utiliser uniquement un coût différentiel pour l'affectation finale **ne change pas la procédure de construction de l'arbre**, en particulier les coupures. Or, si l'on souhaite profiter au maximum de l'information d'asymétrie pour améliorer le plus possible le classifieur sur les cas problématiques, il serait bénéfique d'en tenir compte dans les critères de coupure aussi. Il y a plusieurs approches pour le faire, mais nous discutons ici de celle qui est utilisée dans la package `rpart`, et qui est assez intuitive. L'idée est de revenir à l'approche Bayésienne d'estimation des probabilités $\hat{\mathbb{P}}(C_l|t)$ détaillée dans le chapitre précédent, mais que nous rappelons ici :

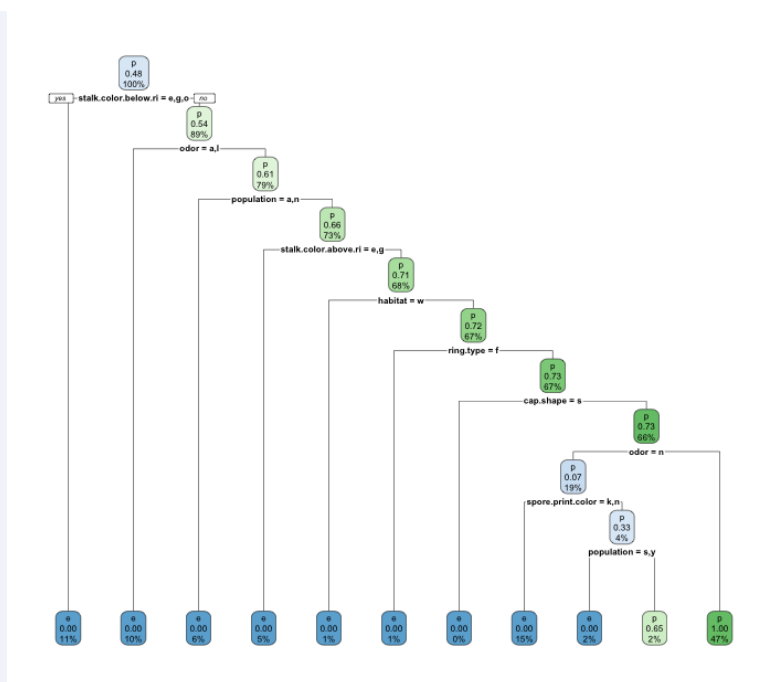
$$\begin{aligned}\hat{\mathbb{P}}(C_l|t) &= \frac{\hat{\mathbb{P}}(t|C_l)\hat{\pi}_l}{\hat{\mathbb{P}}(t)} \\ &= \frac{\frac{n_l(t)}{n_l}\hat{\pi}_l}{\sum_{k=1}^M \frac{n_k(t)}{n_l}\hat{\pi}_k}\end{aligned}$$

où $\hat{\pi}_l$ est interprété comme une probabilité a priori d'être dans la classe C_l . Comme cette estimation de probabilité intervient dans le calcul des impuretés, et donc dans les critères de coupure, il suffit simplement de la modifier pour prendre en compte le coût différencié. Cela se fait à travers la modification des probabilités a priori $\hat{\pi}_l$, dite d'**altération de l'a priori**, qui aboutit à considérer à la place de $\hat{\pi}_l$ des probabilités a priori altérées $\tilde{\pi}_l$ définies par

$$\tilde{\pi}_l = \frac{\hat{\pi}_l c_l}{\sum_{k=1}^M \hat{\pi}_k c_k}$$

où $c_l = \sum_k c(l|k)$. Intuitivement, si une classe a un coût d'erreur de prédiction plus élevé en moyenne par rapport aux autres (mesuré par c_l), la probabilité a priori pour cette classe est **augmentée artificiellement** pour qu'elle ait plus de poids dans l'estimation finale de la probabilité, et donc dans les critères de coupure. Le dénominateur ne sert ici qu'à normaliser toutes les classes pour avoir des a priori qui somment à 1 comme il se doit.

Exemple (Classification de champignons - suite). On reprend le jeu de données des champignons en construisant cette fois-ci un arbre CART avec une fonction de coût différenciée, où le coût de prédire un champignon comestible à tort est 1000 fois plus élevé que l'inverse. Ainsi on essaie de se prémunir contre les FN.



Vous pouvez observer que la structure de l'arbre a fortement changé, conséquence de l'utilisation des probabilités a priori altérées très fortement. En particulier, on remarque une multitude de nouveaux nœuds terminaux "e" complètement purs, ce qui veut dire que l'arbre essaie de ne commettre aucune erreur quand il prédit qu'un champignon est comestible. On peut vérifier que c'est bien le cas avec la nouvelle matrice de confusion :



Vous pouvez observer que cette fois-ci l'arbre CART ne génère plus de FN contrairement au précédent, mais maintenant il prédit des FP. Plus précisément, il a un taux d'erreur de 0.9%, une sensibilité égale à 1 et une spécificité de 0.982. Autrement dit, par rapport au premier arbre construit, celui-ci commet plus d'erreurs en moyenne, mais cela est compensé par une sensibilité parfaite (compromis).

La possibilité d'agir à l'intérieur de la construction même de l'arbre CART pour prendre en compte l'asymétrie de façon aussi simple est assez unique, et facile à mettre en œuvre en pratique car les packages permettent de modifier les coûts en argument. Ce n'est malheureusement pas le cas pour d'autres modèles, en tout cas pas de manière aussi directe.

En revanche, sans altérer la construction d'un classifieur, il est possible de modifier les prédictions finales pour n'importe quel classifieur de façon naturelle, d'une manière similaire aux arbres quand on change le critère d'affectation finale avec un coût (mais la procédure n'est pas du tout la même). Pour cela il faut revenir à la fonction de décision cachée à l'intérieur des classifieurs, dont nous avons parlé dans le deuxième chapitre, et qui dans la suite est assimilée à une *fonction de score* qui est à la base de la fameuse courbe ROC détaillée dans la section suivante.

4.3 Fonction de score et courbes ROC

Définition et construction

Une grande majorité des classifieurs binaires (que ce soit ceux que nous avons étudié dans ce cours, ou ceux que vous étudierez l'année prochaine), s'écrivent de manière générale sous la forme :

$$\hat{h}(\mathbf{x}) = \begin{cases} 1 & \text{si } \hat{\mathbb{P}}(Y = 1 | \mathbf{X} = \mathbf{x}) \geq \frac{1}{2} \\ 0 & \text{sinon} \end{cases}$$

où $\hat{\mathbb{P}}(Y = 1 | \mathbf{X} = \mathbf{x})$ est l'estimation spécifique à chaque méthode de la probabilité conditionnelle que la cible soit de classe 1. Pensez par exemple aux hypothèses faites par le Bayésien naïf ou la LDA/QDA pour construire cette approximation, ou à la fréquence empirique calculée par les plus proches voisins ou les arbres au sein d'un nœud. On peut ré-écrire cette équation sous la forme

$$\hat{h}(\mathbf{x}) = 2\mathbb{1} \left\{ \hat{\mathbb{P}}(Y = 1 | \mathbf{X} = \mathbf{x}) \geq \frac{1}{2} \right\} - 1,$$

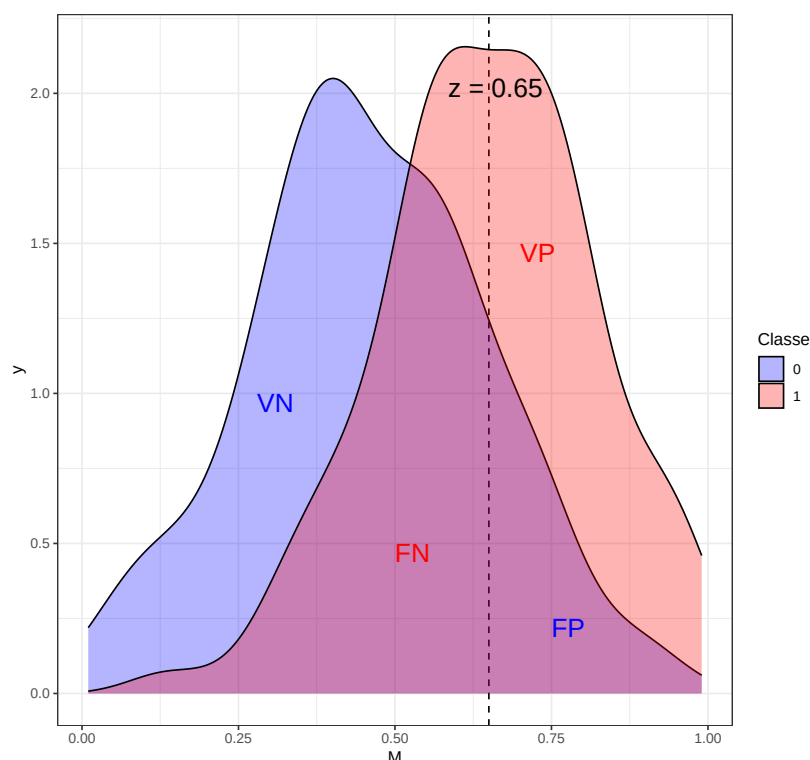
ou encore plus généralement

$$\hat{h}(\mathbf{x}) = 2\mathbb{1} \{s(\mathbf{x}) \geq z\} - 1$$

où l'on a noté $s(\mathbf{x}) = \hat{\mathbb{P}}(Y = 1 | \mathbf{X} = \mathbf{x})$ et $z = \frac{1}{2}$. Cette notation plus générale est d'importance toute particulière, car elle permet de voir la construction de la prédiction finale d'un classifieur avec un peu plus de recul :

1. Le plus important, pour ce cours, est que z peut être interprété comme un **seuil**, et qu'il est alors possible de le faire varier pour changer les prédictions finales du classifieur. Plus ce seuil sera élevé, plus la probabilité conditionnelle de la cible devra être élevée pour que l'on prédise la classe 1 (des positifs en général). Autrement dit quand z augmente, de moins en moins d'individus sont prédits comme étant de classe 1, jusqu'à atteindre $z = 1$ où plus aucun d'entre eux ne l'est : on converge alors vers la situation où TP=0, et donc vers une sensibilité TVP=0. Mais a contrario, si aucun individu n'est prédit comme positif, ils sont tous prédits comme négatifs, ce qui entraîne qu'on a une spécificité 1-TFP = 0 aussi. Cette remarque est essentielle pour la construction de la courbe ROC décrite ci-dessous.
2. D'autre part, dans la dernière expression, la fonction $s(\mathbf{x})$ peut ne pas être intrinsèquement une probabilité conditionnelle, mais juste une fonction qui témoigne de la propension d'un individu $\mathbf{x}$ à être de classe 1. C'est ce que l'on appelle en toute généralité une **fonction de score**, et il existe des classifieurs qui ne se basent pas sur une estimation explicite de la probabilité conditionnelle mais sur d'autres critères (basés sur la géométrie par exemple, vous verrez cela en 3A avec les *Support Vector Machines*). Mais le principe reste le même : plus la valeur du score est élevé, plus on va prédire l'individu comme étant de classe 1, et en faisant varier le seuil z on change la quantité d'individus prédits dans la classe 1 (mais plus forcément entre 0 et 1 maintenant, vu que $s(\mathbf{x})$ n'est plus forcément une probabilité). Mais dans tous les classifieurs de ce cours, $s(\mathbf{x})$ fait toujours référence à la probabilité conditionnelle.

On peut illustrer graphiquement ces principes, en représentant les densités de probabilités f_0 et f_1 des scores prédits pour les individus de l'échantillon de test qui sont réellement de classe 0 et 1, respectivement. Sur le graphique on rajoute une séparation verticale correspondant au seuil (ici pour illustration $z = 0.65$), ce qui permet de visualiser les VP, FP, VN et FN facilement.



Remarque (A propos de f_0 et f_1). Nous utiliserons de nouveau ces notations plus tard dans le chapitre, donc il est judicieux de les définir formellement : f_0 est la densité de probabilité du score des individus de l'échantillon test qui sont de classe 0, c'est-à-dire que c'est la densité de probabilité de la variable aléatoire $s(\mathbf{X})|Y = 0$, et symétriquement f_1 est la densité de probabilité de $s(\mathbf{X})|Y = 1$. De même on notera F_0 et F_1 les fonctions de répartition correspondantes.

Grâce à ces remarques, on peut en déduire que les prédictions finales dépendent d'un **couple** (s, z) , où s est construite par le classifieur sur l'échantillon d'apprentissage, et z est un seuil sur lequel on peut jouer pour ajuster les prédictions a posteriori. Mais comme ces prédictions finales peuvent être utilisées pour assembler une matrice de confusion, et en particulier calculer la sensibilité TVP et 1-spécificité = TFP, on peut symboliquement définir une fonction $R : (s, z) \mapsto (\text{TFP}, \text{TVP})$ qui à tout couple fonction de score / seuil associe une valeur de sensibilité / 1-spécificité, ou encore écrire explicitement la dépendance des indicateurs de qualité sous la forme $\text{TVP}_s(z)$ et $\text{TFP}_s(z)$. Cela correspond

exactement à la courbe ROC.

Définition 14 (Courbe ROC). La **courbe ROC** associée à un classifieur avec fonction de score s est la courbe paramétrique définie implicitement par les points de coordonnées $(TFP_s(z), TVP_s(z))$ quand z varie entre 0 et 1.

Remarque (Points fixes pour toutes les courbes ROC). Par définition, quelque soit la fonction de score s (dans le cas où elle est une probabilité conditionnelle, comme dans ce cours), alors il est évident que $TVP_s(0) = 1$ et $TFP_s(0) = 1$, de même que $TVP_s(1) = 0$ et $TFP_s(1) = 0$. Cela implique que peu importe le classifieur considéré, sa courbe ROC passera forcément par les points de coordonnées $(0, 0)$ et $(1, 1)$.

Remarque (Une subtilité ...). Le raisonnement précédent ne s'applique que si aucun individu n'est prédit avec un score égal à 1. Si c'est n'est pas le cas, alors les individus avec $s(\mathbf{x}) = 1$ sont comptabilisés dans les VP, ce qui impliquera que $TVP_s(1) > 0$. Si l'on définit un classifieur avec une inégalité stricte

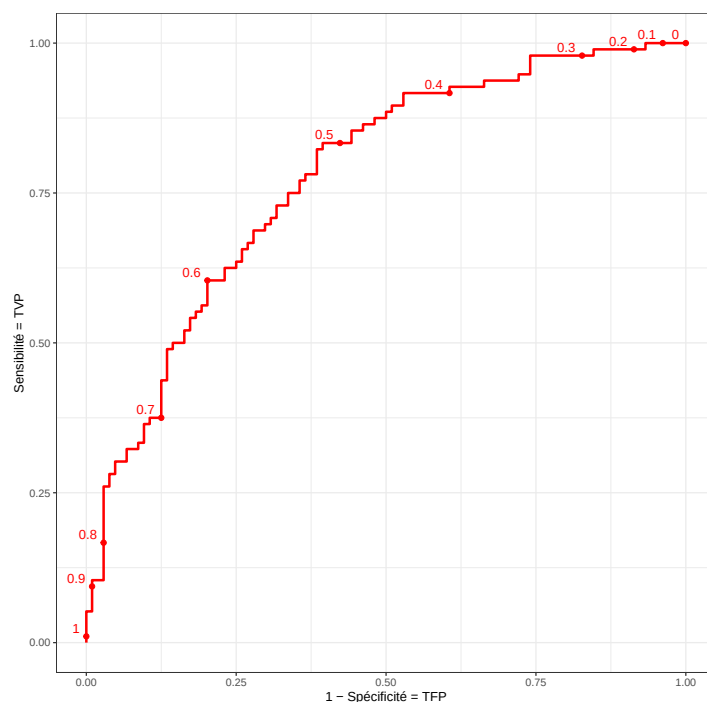
$$\hat{h}(\mathbf{x}) = 2\mathbb{1}\{s(\mathbf{x}) > z\} - 1,$$

alors ce problème ne se présente plus, mais un autre apparaît ... En effet cette fois-ci ce sont les individus prédits avec un score égal à 0 qui sont comptabilisés et impliquent alors $TFP_s(0) < 1$. Dans les librairies c'est la définition avec l'inégalité non stricte qui est implémentée, et donc dans les situations où le score est égal à 1 pour un ou plusieurs individus, vous verrez un léger décalage du point en bas à gauche sur la courbe ROC. Mais sur les tracés que vous ferez vous-mêmes, pensez toujours à démarrer malgré tout du point $(0, 0)$. Vous pourrez remarquer ce phénomène sur les courbes ROC des figures ci-dessous.

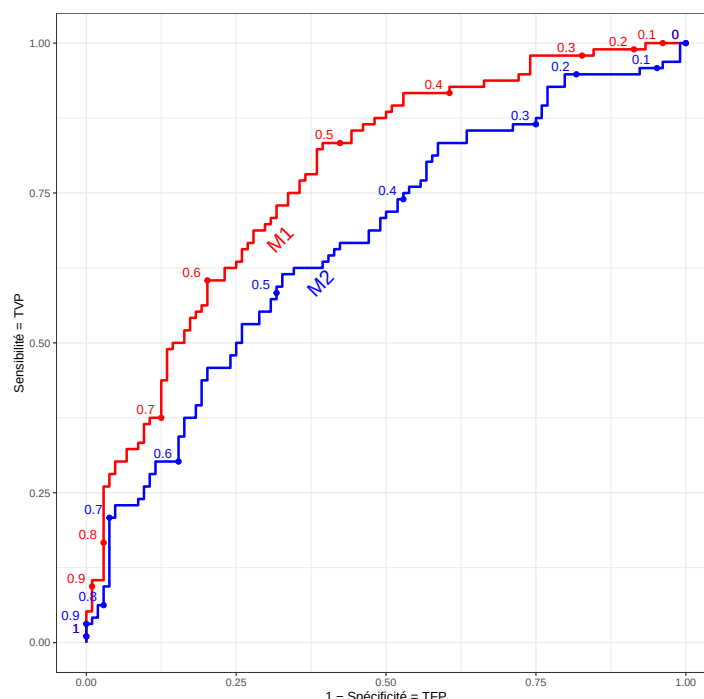
Remarque (A propos de f_0 et f_1 - suite). Grâce à nos nouvelles notations, on peut écrire que

$$\begin{aligned} TFP_s(z) &= \mathbb{P}(s(\mathbf{X}) \geq z | Y = 0) = 1 - F_0(z) \\ TVP_s(z) &= \mathbb{P}(s(\mathbf{X}) \geq z | Y = 1) = 1 - F_1(z). \end{aligned}$$

Une illustration de la courbe ROC est donnée ci-dessous : on considère un classifieur avec une fonction de score fixée, et on trace la courbe pour plusieurs valeurs du seuil z . Pour vous repérez sur la courbe, on annote également les valeurs de $z = 0, 0.1, \dots, 0.9, 1$.



Pour ce classifieur, la courbe ROC permet de visualiser facilement tous les compromis possibles quand le seuil z varie, ce qui en pratique peut vous guider vers les prédictions finales à choisir par rapport aux objectifs de l'étude. Mais il est aussi particulièrement intéressant d'utiliser cette visualisation pour **comparer des classifieurs entre eux**, comme ci-dessous avec deux classifieurs notés M1 et M2.

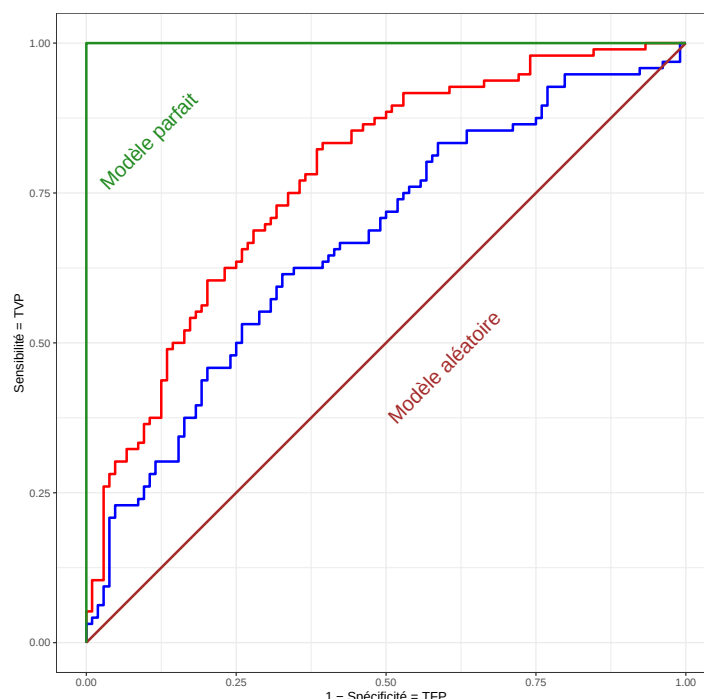


Grâce à cette comparaison graphique, vous pouvez voir très facilement que la courbe ROC du modèle M1 en rouge est systématiquement au-dessus de la courbe du modèle M2 en bleu : cela implique que **le modèle M1 est meilleur que M2** (pour toutes les valeurs de seuil z possibles, la sensibilité et la spécificité de M1 sont supérieures).

Si l'on poursuit le raisonnement, les courbes ROC de deux classifieurs universels (dans le sens où ils sont définis pour tous les problèmes de classification) peuvent être rajoutées sur ce graphique :

1. Le **modèle parfait**, qui prédit exactement la bonne classe pour tous les individus. Bien évidemment ce classifieur n'existe pas, mais il sert de comparaison à tous les classifieurs que vous pouvez construire. Sa courbe ROC, qui passe forcément par les points (0,0) et (1,1) (voir remarque précédente), consiste en un unique point supplémentaire (0,1) (car il ne commet pas d'erreur), appelé le **point idéal**.
2. Le **modèle aléatoire**, qui prédit une classe au hasard. On peut représenter la courbe ROC moyenne d'un tel classifieur, qui est en fait la courbe d'équation $TVP=TFP$.

On arrive donc à la représentation finale ci-dessous.



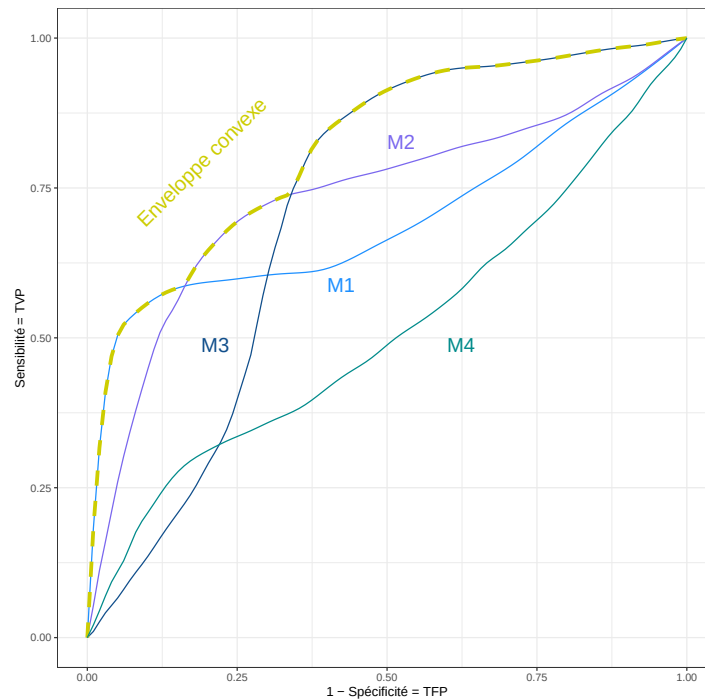
Remarque (Construction de la courbe ROC en pratique). Pour tracer la courbe ROC d'un classifieur donné, les seuls ingrédients nécessaires sont 1) un échantillon de test, avec les valeurs de Y observées et 2) le score fourni par le classifieur (entraîné sur l'échantillon d'apprentissage) pour chaque individu de l'échantillon de test. La procédure consiste simplement en du comptage :

1. On arrange d'abord les individus de l'échantillon de test par score décroissant ;
2. On fait progressivement diminuer le seuil z à partir de 1, et pour chaque valeur de seuil on compte les VP et les FN, ce qui nous permet d'en déduire TVP et TFP, donc les coordonnées d'un point de la courbe à tracer ;
3. Il n'est pas nécessaire de tester toutes les valeurs de seuil, il suffit de ne considérer que des valeurs parmi la liste des valeurs de score. En effet pour une valeur de seuil continue comprise strictement entre deux scores, le comptage ne change pas (puisque'il n'existe aucun individu de l'échantillon de test qui a un score égal à ce seuil).

Vous aurez l'occasion de pratiquer cette procédure vous-mêmes lors du TP3 (n'oubliez pas de rajouter manuellement les points (0,0) et (1,1), si besoin).

Comparaison de classifieurs

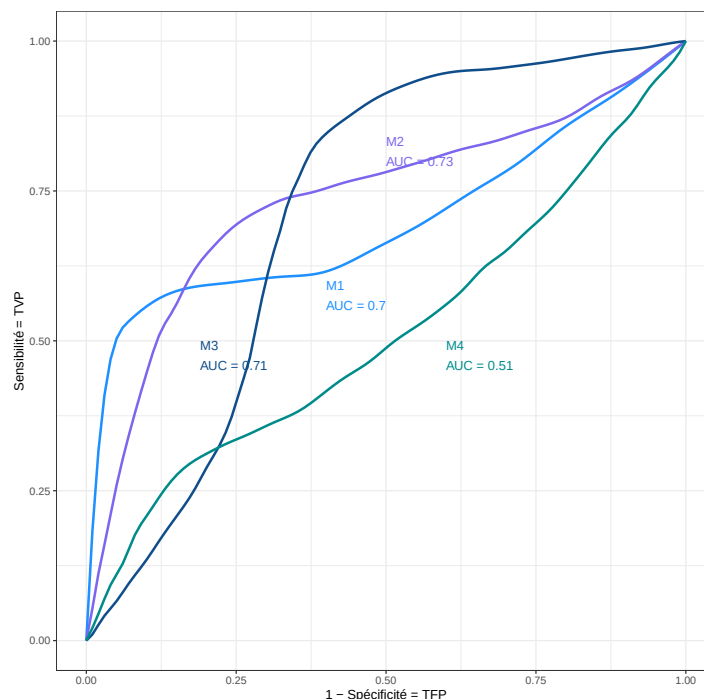
Nous avons vu précédemment que l'on peut se servir de la superposition de courbes ROC pour comparer des classifieurs entre eux, et en particulier détecter qu'un classifieur est meilleur qu'un autre si sa courbe ROC est au-dessus (comme le modèle M1 en rouge meilleur que le modèle M2 en bleu). Mais en pratique la comparaison peut s'avérer plus complexe, comme dans l'exemple ci-dessous où l'on trace les courbes ROC pour 4 classifieurs M1, M2, M3 et M4. Aucun d'entre eux n'est meilleur que tous les autres systématiquement. Cependant, on peut calculer l'enveloppe convexe de toutes les courbes ROC, représentée en jaune ci-dessous, et identifier quels modèles font partie de cette enveloppe. On peut alors **éliminer d'office les modèles ne participant pas à l'enveloppe convexe**, car ils sont obligatoirement moins bons qu'un autre modèle au moins. Ici on éliminera donc M4, car même si dans certaines zones il est meilleur que M3, il est quand même moins bon que M1 et M2. L'enveloppe convexe est composée de M1 sur la gauche, M2 au centre et M3 à droite.



Pour les modèles restants, on ne peut pas conclure, et une fois de plus c'est le cas d'usage qui orientera potentiellement vers l'un d'entre eux par rapport à un compromis opérationnel, s'il y en a un. Une autre approche consiste à comparer un indicateur de performance globale de la courbe qui résume les performances d'un classifieur pour tous les seuils possibles. L'indicateur le plus utilisé en apprentissage statistique moderne (i.e. machine learning) est l'aire sous la courbe.

L'aire sous la courbe AUC (*Area Under the Curve*)

L'aire sous la courbe, ou **AUC**, est tout simplement définie pour un classifieur donné comme étant l'aire sous sa courbe ROC. Sur l'exemple précédent, on obtient les AUC suivants.



Par définition l'AUC est compris entre 0 et 1, et l'on a comme cas particulier que le modèle parfait a un AUC=1, alors que le modèle aléatoire a un AUC=0.5. Plus un classifieur a un AUC grand, plus il est performant, alors qu'un AUC proche de 0.5 indique un modèle pauvre proche de la prédiction au hasard. Enfin, il existe une interprétation probabiliste intéressante de l'AUC. En effet, on peut écrire par exemple que par définition

$$\text{AUC} = \int_0^1 \text{TVP}(\text{TFP}^{-1}(u)) du$$

où u représente toutes les valeurs possibles prises par TFP quand le seuil z varie.

On fait alors le changement de variable $\text{TFP}^{-1}(u) = z$:

$$\begin{aligned}
 \text{AUC} &= \int_1^0 \text{TVP}(z) \frac{d}{dz} \text{TFP}(z) dz \\
 &= \int_1^0 (1 - F_1(z)) (-f_0(z)) dz \\
 &= \int_0^1 (1 - F_1(z)) f_0(z) dz \\
 &= \int_0^1 \left(\int_0^1 \mathbb{1}_{z' \geq z} f_1(z') dz' \right) f_0(z) dz \\
 &= \int_0^1 \int_0^1 \mathbb{1}_{z' \geq z} f_1(z') f_0(z) dz' dz \\
 &= \mathbb{P}(s(\mathbf{X}_1) \geq s(\mathbf{X}_0))
 \end{aligned}$$

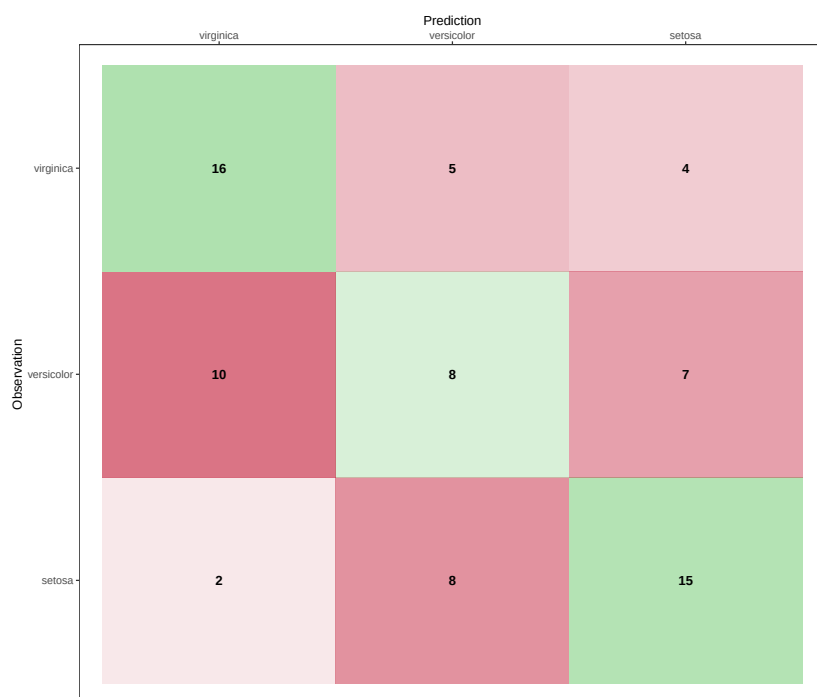
où $\mathbf{X}_0 := \mathbf{X}|Y = 0$ et $\mathbf{X}_1 := \mathbf{X}|Y = 1$. Autrement dit, l'AUC est la probabilité que le classifieur prédise un score plus élevé pour n'importe quel individu de test de classe 1 que pour n'importe quel autre individu de classe 0.

La courbe LIFT

Nous terminons ce chapitre par une mention rapide d'une variante de la courbe ROC, appelée la **courbe LIFT**, qui est plutôt utilisée dans le domaine du marketing. Elle contient les mêmes informations que la courbe ROC, mais propose une visualisation un peu différente, avec une courbe composée des points de coordonnées $(\text{TP}_s(z), \text{TVP}_s(z))$ quand z varie entre 0 et 1. L'ordonnée est donc la même, mais l'abscisse diffère.

4.4 Extension à la classification multi-classes

Dans ce chapitre nous nous sommes concentrés sur la définition de la matrice de confusion, des indicateurs de qualité, de la courbe ROC et de l'AUC pour la classification binaire avec deux classes seulement. Pour les problèmes avec plus de deux classes, la généralisation de la matrice de confusion est simple, comme illustré ci-dessous avec un problème de classification à 3 classes (jeu de données iris).



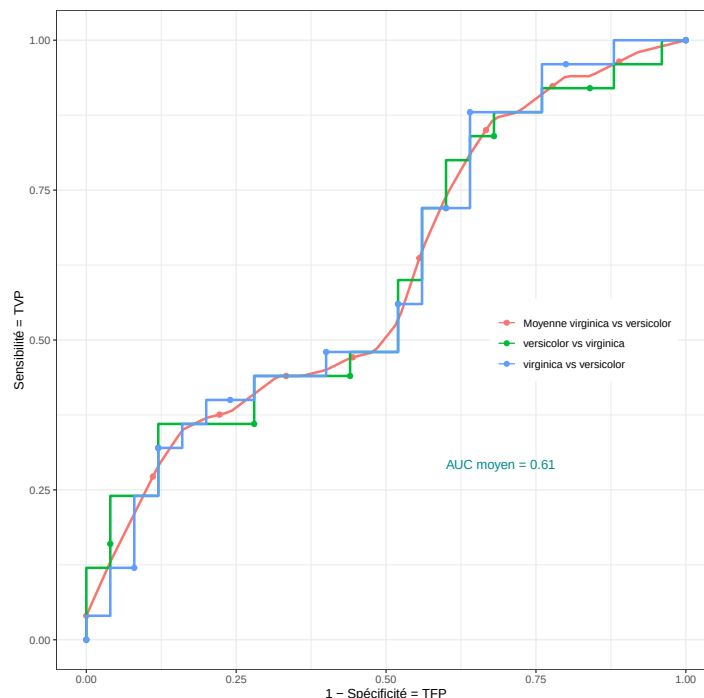
En revanche, les autres mesures ne se généralisent pas aussi directement. En effet, elles reposent sur le concept de classe des positifs et des négatifs, qui est intrinsèquement un concept binaire. En pratique, il faut donc se ramener à cette situation, et il y a deux voies possibles, qui ont le point commun de transformer le problème multi-classes en plusieurs problèmes binaires et à agréger les résultats.

Remarque ($\triangle$ Ne pas confondre évaluation du classifieur et construction du classifieur). Ici, quand on dit que l'on transforme le problème multi-classes en plusieurs problèmes binaires, c'est **uniquement pour évaluer la performance** d'un classifieur donné. Cela ne veut pas dire que le classifieur multi-classes construit initialement l'a été avec une astuce de ce genre. En particulier, la fonction de score du classifieur multi-classes que l'on considère est de la forme $s(\mathbf{x}) = [s_1(\mathbf{x}), \dots, s_M(\mathbf{x})] \in [0, 1]^M$ où $s_l(\mathbf{x})$ est le score de la classe C_l , et $\sum_{l=1}^M s_l(\mathbf{x}) = 1$ (on rappelle que dans ce chapitre le score est la probabilité conditionnelle d'être dans une classe, et par conséquent les scores somment à 1).

L'approche une-contre-une (*one-versus-one* ou *ovo* en anglais)

Le principe de cette première approche est de répéter plusieurs problèmes de classification binaire C_l contre $C_{l'}$ pour tous les couples $l, l' \in \{1, \dots, M\}$, en considérant pour chaque couple d'abord C_l comme classe des positifs puis $C_{l'}$, où l'on calcule les différents indicateurs de qualité (TVP et TFP en particulier). Pour un couple (l, l') donné, on fait la moyenne des résultats obtenus avec C_l positif et $C_{l'}$ positif, et on peut en déduire une courbe ROC moyenne et un AUC moyen associé, voir l'exemple ci-dessous sur le jeu de données iris pour virginica

contre versicolor.



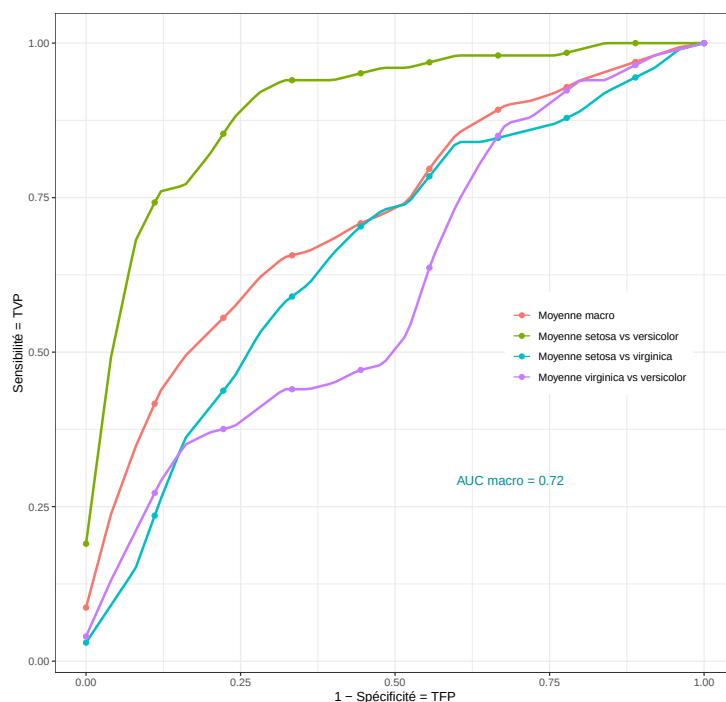
On peut tracer ces courbes ROC pour tous les couples (l, l') , et calculer l'AUC moyen final comme étant la moyenne de tous les AUC moyen. Dans ce cas on dit que l'on a fait une *moyenne macro*, qui s'écrit sous la forme

$$\text{AUC}_{\text{macro}} = \frac{1}{M(M-1)} \sum_{l \neq l'} \text{AUC}(l, l')$$

où l'on note $\text{AUC}(l, l')$ l'AUC pour la classification binaire C_l contre $C_{l'}$ quand l est considéré comme la classe des positifs. Il existe également une version *pondérée*, où chaque $\text{AUC}(l, l')$ a un poids égal à la fréquence empirique de $C_l \cup C_{l'}$ dans l'échantillon de test.

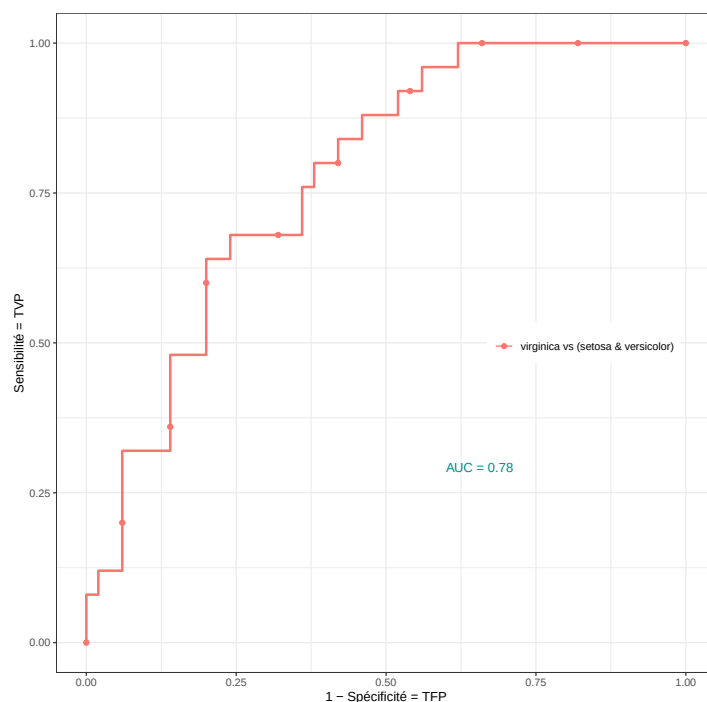
Remarque (Calcul en pratique). Il n'est pas nécessaire pour construire toutes ces classifications binaires de reconstruire un classifieur, il suffit en pratique d'utiliser la fonction de score multi-classes définie dans la remarque précédente, en ne gardant tout à tour que deux colonnes correspondant aux classes que l'on met l'une contre l'autre.

Voici le résultat final, avec toutes les courbes ROC moyennes, la moyenne finale des courbes ROC moyennes, et l'AUC macro sur iris.



L'approche une-contre-les autres (*one-versus-rest* ou *ovr* en anglais, plus rarement *one-versus-all* ou *ova*)

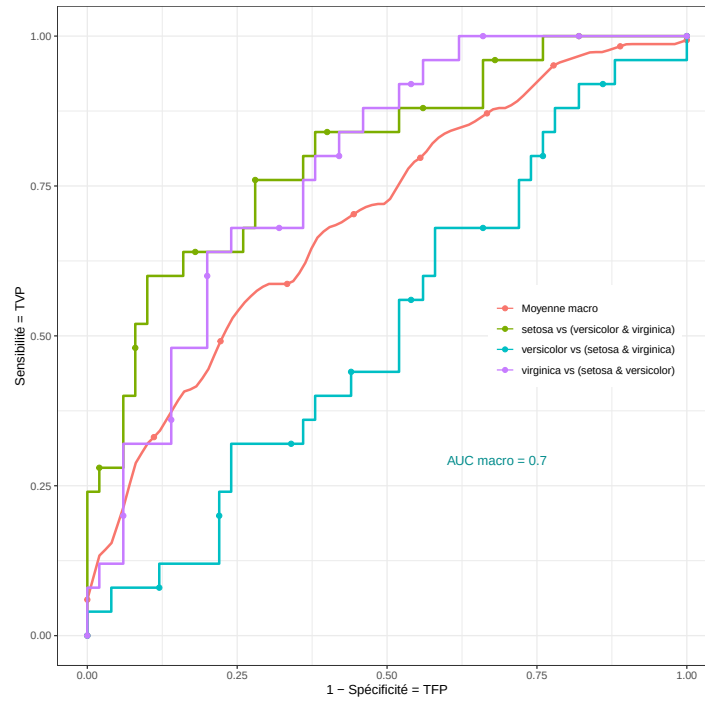
La seconde approche est similaire dans l'esprit, mais au lieu de considérer tous les couples possibles de classification binaire C_l contre $C_{l'}$, l'approche une-contre-les autres étudie seulement les classifications binaires C_l contre $\{C_{l'}\}_{l' \neq l}$. Cela implique en particulier un coût calculatoire réduit, avec seulement M classifications binaires au lieu de $M(M - 1)$ pour l'approche une-contre-une. Sur l'exemple précédent, voici ce que l'on obtient pour virginica contre (setosa & versicolor).



On répète le processus pour toutes les classes, et on peut calculer comme précédemment la courbe ROC moyenne macro et l'AUC moyen macro, par exemple

$$AUC_{\text{macro}} = \frac{1}{M} \sum_l AUC(l)$$

où l'on note $AUC(l)$ l'AUC pour la classification binaire C_l contre $\{C_{l'}\}_{l' \neq l}$. Comme pour ovo, il est également possible de pondérer par la prévalence de chaque classe. Voici le résultat par moyenne macro pour ovr sur iris.

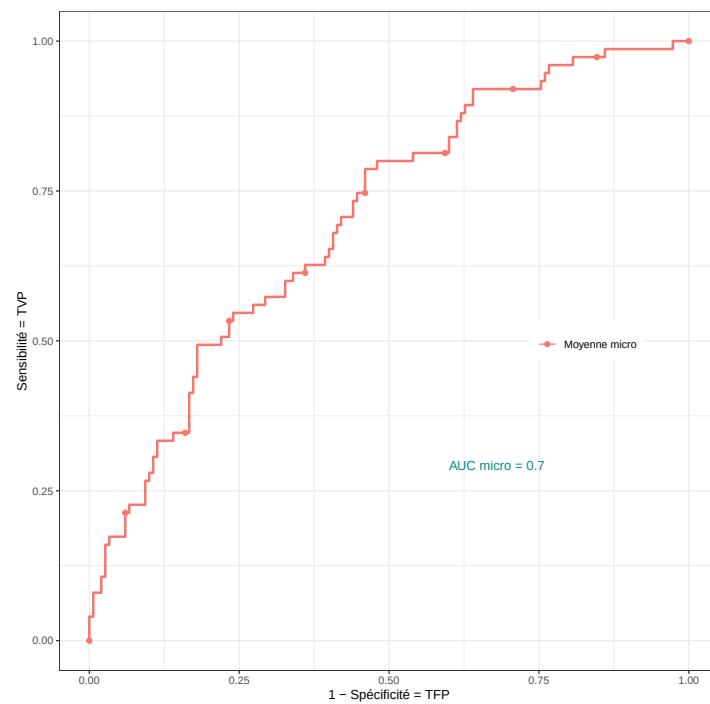


Enfin, pour l'approche une-contre-les autres, il est possible de moyenner les résultats différemment. Si l'on note VP_l , FP_l , VN_l et FN_l le nombre de vrais / faux positifs / négatifs lors de la classification binaire C_l contre $\{C_{l'}\}_{l' \neq l}$, on peut définir la *moyenne micro* de TVP et TFP :

$$\text{TVP}_{\text{micro}} = \frac{\sum_{l=1}^M VP_l}{\sum_{l=1}^M (VP_l + FN_l)}$$

$$\text{TFP}_{\text{micro}} = \frac{\sum_{l=1}^M FP_l}{\sum_{l=1}^M (FP_l + FN_l)},$$

et à l'aide de ces indicateurs moyennés on calcule la courbe ROC micro et l'AUC moyen micro. Sur iris on obtient le résultat suivant.



Séance 6 : Apprentissage supervisé pratique

24 Octobre, 2023

- Considérations sur les variables explicatives catégorielles
- Le problème de la grande dimension
 - Sélection de variables
 - Transformation de variables et réduction de dimension
- Les données manquantes
- Optimisation des hyperparamètres d'un modèle
- Visualisation

Chapitre 5

Apprentissage supervisé pratique

Pour terminer ce cours, nous allons discuter d'un certain nombre de spécificités que vous rencontrerez très certainement lors de l'utilisation des méthodes d'apprentissage supervisé sur des problèmes pratiques. Le tableau ci-dessous résume un certain nombre de caractéristiques des méthodes introduites dans ce cours, mais également des méthodes vues dans d'autres cours de 2A ou de 3A l'année prochaine : la gestion des variables explicatives catégorielles, la complexité pour la phase d'entraînement du modèle (on a supposé $d < n$ ci-dessous), la robustesse du modèle par rapport à la normalisation des variables, la gestion des valeurs manquantes (NA), et la capacité prédictive.

Méthodes			Var. Cat.	Complexité entraînement	Normalisation	NA	Capacité prédictive
Classif. & Régr.	Ce cours	KNN	--	$O(dn)$	--	--	+ (choix dist.)
		Arbre CART	++	$O(dn \log(n))$	++	++	-
		Forêts	++	$O(B \text{ mtry } n \log(n))$	++	++	++ (n grand)
	3A	Bag./Boost.	++	-	++	++	++ (n grand)
		Rés. Neurones	--	-	-	--	++ (n grand)
Classification	Ce cours	Bayes naïf	++	$O(dn)$	+	+	-
		LDA / QDA	×	$O(d^2 n)$	+	+	-
	2A	Régr. Log.	+	$O(dn)$	-	--	-
	3A	SVM	--	$O(d^2 n)$	--	--	++ (n petit)
Régression	2A	Régr. Lin.	+	$O(d^2 n)$	-	--	-
	3A	Kernel Regr.	--	$O(dn^2 + n^3)$	--	--	++ (n petit)

Chacun de ces points, ainsi que d'autres aspects, sont détaillés dans la suite du chapitre.

5.1 Gérer les variables explicatives catégorielles

5.1.1 Impact sur les modèles vus en cours

Comme résumé dans le tableau récapitulatif ci-dessus, les modèles vus dans ce cours sont plus ou moins sensibles à la présence de variables catégorielles, dans le sens où certains d'entre eux les gèrent de manière naturelle, alors que pour les autres il est nécessaire de les traiter de manière particulière. Par exemple :

- Les arbres CART, les forêts aléatoires et le Bayésien naïf intègrent les variables catégorielles sans modification.
- La LDA et la QDA ne peuvent pas être appliquées (même si la stratégie d'encodage décrite dans la suite pourrait être envisagée, elle est en générale déconseillée pour ces méthodes).
- Les plus proches voisins nécessitent soit une définition spécifique de la distance (voir le chapitre correspondant), soit une stratégie d'encodage.

Il est fréquent en apprentissage statistique moderne d'utiliser une stratégie générale basée sur une transformation des variables catégorielles en variables continues, une telle transformation est connue sous le nom d'**encodage**. En particulier, les méthodes listées dans le tableau qui ne font pas partie de ce cours font appel à ce type de stratégie.

5.1.2 Stratégie générale : l'encodage

Plusieurs méthodes d'encodage des variables catégorielles sont couramment utilisées, chacune ayant ses propres avantages et inconvénients.

1. **Encodage one-hot.** Cette méthode est largement utilisée pour les variables catégorielles. Elle crée un ensemble de variables indicatrices (*dummy variables* en anglais) pour chaque catégorie. Prenons l'exemple de la variable "Couleur" avec les catégories "Rouge", "Vert", "Bleu". L'encodage one-hot créera trois nouvelles variables, "Rouge", "Vert", "Bleu". Chaque observation recevra un 1 dans la colonne correspondant à sa couleur et des zéros dans les autres. Par exemple, si un élément a la couleur "Vert", il sera représenté comme (0, 1, 0).

Couleur		Rouge	Vert	Bleu
Rouge	→	1	0	0
Vert		0	1	0
Bleu		0	0	1
Vert		0	1	0
Rouge		1	0	0

L'encodage one-hot est approprié lorsque les catégories n'ont pas d'ordre intrinsèque et qu'aucune relation d'ordre ne doit être imposée. Cependant, il peut augmenter considérablement la dimension des données. Pour la régression linéaire et la régression logistique, c'est cet encodage qui est utilisé (avec une subtilité pour l'identifiabilité des coefficients, dont on vous a parlé en cours).

2. **Encodage de fréquence.** Cette méthode remplace chaque catégorie par sa fréquence d'occurrence dans le jeu de données. Par exemple, si "Rouge" apparaît 2 fois, "Vert" 2 fois et "Bleu" 1 fois sur un jeu de données de taille $n = 5$, les catégories seraient encodées comme "Rouge" : $2/5$, "Vert" : $2/5$, "Bleu" : $1/5$.

Couleur		Couleur
Rouge	→	0.4
Vert		0.4
Bleu		0.2
Vert		0.4
Rouge		0.4

L'encodage de fréquence peut être utile lorsque la fréquence de chaque catégorie est importante pour la tâche.

3. **Encodage de cible (Target Encoding).** Cette méthode, réservée aux problèmes de régression, utilise les valeurs cibles de la sortie pour encoder les catégories. Chaque catégorie est encodée en fonction de la moyenne de la variable cible pour cette catégorie.

Couleur	Y		Couleur
Rouge	2.3	→	1.7
Vert	6.7		5.45
Bleu	-2.67		-2.67
Vert	4.2		5.45
Rouge	1.1		1.7

Cette transformation peut aider le modèle à capturer les relations entre les catégories et la variable cible.

Remarque (⚠ L'encodage se fait sur l'échantillon d'apprentissage). Toute transformation préliminaire des données, lorsqu'elle est guidée par les valeurs du jeu de données, doit s'estimer sur l'échantillon d'apprentissage : c'est le cas par exemple pour l'encodage de fréquence ou l'encodage de cible. Une fois l'encodage effectué et le modèle entraîné, pour faire les prédictions sur l'échantillon de test il faudra remplacer les variables catégorielles par la valeur encodée estimée sur l'échantillon d'entraînement. Cela implique qu'il faudra être vigilant au fait que les catégories présentes dans l'échantillon d'entraînement et de test soient les mêmes suite à leur séparation, sinon l'encodage sera impossible pour l'échantillon de test (ce commentaire s'applique également pour l'encodage one-hot).

Remarque (⚠ Le type d'encodage est un hyperparamètre!). Si dans la pratique vous souhaitez tester différents types d'encodage et choisir le plus adapté à votre problème, l'encodage devient de facto un hyperparamètre,

et vous devrez utiliser de la validation croisée pour identifier le meilleur.

Vous pourrez rencontrer également deux autres types d'encodage : l'encodage de label (*label encoding*), qui remplace chaque catégorie par un numéro entre 1 et le nombre total de catégories, et l'encodage binaire (*binary encoding*), qui représente chaque catégorie par un nombre binaire obtenu en concaténant les valeurs de l'encodage one-hot (par exemple 01000). Mais ces deux types d'encodage conduisent à ajouter artificiellement un ordre et des distances entre les catégories, et sont donc à éviter en pratique.

5.1.3 Gestion des catégories rares

La gestion des catégories rares est un aspect essentiel du prétraitement des variables catégorielles. Les catégories rares se produisent lorsque certaines catégories ont un nombre très faible d'occurrences dans le jeu de données, ce qui peut être problématique pour les modèles d'apprentissage supervisé. Dans de tels cas, il est souvent nécessaire de consolider ces catégories rares pour éviter le surajustement. Pour aborder ce problème, on peut définir un seuil de fréquence minimale en deçà duquel les catégories sont considérées comme rares. Les catégories qui ne satisfont pas ce critère peuvent être regroupées en une catégorie commune, par exemple, "Autre". Cette consolidation réduit la dimension des données et permet d'éviter le biais du modèle vers les catégories majoritaires.

5.2 Jeux de données de grande taille

A l'heure actuelle et dans de nombreux domaines, l'augmentation de la collecte de données tout azimuth conduit à devoir traiter de plus en plus fréquemment des jeux de données avec beaucoup d'individus, c'est-à-dire n très grand. Dans ces situations, il conviendra donc de grader à l'esprit que tous les algorithmes d'apprentissage supervisé ont des complexités potentiellement très différentes, à la fois en fonction de n mais aussi de d le nombre de variables explicatives (certaines complexités sont données dans le tableau récapitulatif au début du chapitre). Cela veut donc dire que vous ne pourrez pas utiliser certains d'entre eux en pratique, ou en tout cas pas dans leur version initiale (il existe des adaptations possibles, que vous apprendrez en partie en 3A).

5.3 Gérer la grande dimension des variables explicatives

5.3.1 Motivation et Contexte

Dans de nombreux cas, vous travaillerez avec des ensembles de données qui présentent un grand nombre de caractéristiques, parfois bien au-delà de ce qui est nécessaire pour effectuer une tâche d'apprentissage supervisé. La réduction de dimension intervient alors pour aider à gérer cette complexité :

- L'une des principales motivations de la réduction de dimension est la simplification des modèles. Les modèles d'apprentissage supervisé, qu'il

s'agisse de réseaux de neurones, d'arbres de décision ou d'autres méthodes, peuvent rapidement devenir trop complexes et surajustés lorsque le nombre de caractéristiques est élevé.

- Un autre aspect essentiel de la réduction de dimension est son rôle dans l'amélioration des performances des modèles. En éliminant les caractéristiques redondantes ou inutiles, cette technique permet de mettre en évidence les caractéristiques discriminantes qui contribuent le plus à la tâche d'apprentissage supervisé. Cela peut avoir un impact significatif sur la précision et l'efficacité des modèles.
- De plus, un point complémentaire est que la réduction de dimension permet de visualiser les données de manière plus aisée. En réduisant la dimension à deux ou trois dimensions, il devient possible de représenter les données dans un espace graphique, ce qui facilite grandement la compréhension des relations entre les points de données.

En pratique, il existe deux grandes familles pour la réduction de dimension :

1. **La sélection de variables (*Feature selection*)**. C'est une méthode spécifique à l'apprentissage supervisé, qui vise à sélectionner, parmi les variables explicatives initiales, un sous-ensemble de taille plus petite composé uniquement des variables explicatives qui expliquent le mieux la variable cible. Au final, le modèle est entraîné avec les variables explicatives originelles, mais seulement avec certaines d'entre elles.
2. **La projection ou plongement (*Embedding*)**. Les méthodes de plongement ne sont pas spécifiques à l'apprentissage supervisé, et ont par ailleurs émergé d'abord en apprentissage non-supervisé. Ici, et contrairement à la sélection de variables, l'idée est de construire de nouvelles variables explicatives obtenues par transformation des variables originales, et en petit nombre. Intuitivement, on cherche à identifier une transformation qui "compresse" l'information des variables explicatives dans une nouvelle représentation de petite taille, cette opération s'appelant alors le plongement (on "plonge" les variables explicatives dans un espace de dimension réduit).

Dans chacune de ces familles, on va retrouver une séparation nette entre deux classes de méthodes : celles qui se basent sur des opérations linéaires, et celles sur des opérations non-linéaires.

5.3.2 Sélection de variables

Comme dit précédemment, la sélection de variables vise à identifier, parmi toutes les variables explicatives, celles qui ont le plus de pouvoir prédictif pour prédire la cible. Les méthodes de sélection de variables sont de deux natures :

1. **Méthodes agnostiques**. Ce sont des méthodes, aussi appelées d'influence de variables préliminaire, qui utilisent uniquement le jeu de données pour estimer une mesure de la relation entre chaque variable explicative et la variable cible, et à ne sélectionner que celles qui ont une intensité grande. Le modèle d'apprentissage supervisé est ensuite entraîné dans une

seconde phase sur celles restantes. Dans cette classe de méthodes, on trouvera par exemple la corrélation, que vous avez étudiée l'année dernière, pour mesurer l'intensité de la relation entre une variable explicative continue et une variable cible continue elle aussi (si une variable explicative est catégorielle mais que la variable cible est continue, vous pouvez utiliser un test du chi-deux, et enfin, si elles sont toutes les deux catégorielles, utiliser une table de contingence). L'inconvénient de la corrélation est qu'elle ne considère que de potentielles relations linéaires entre les variables, et plus récemment des méthodes non-linéaires efficaces ont été proposées à la place : l'information mutuelle, la distance correlation ou les HSIC.

2. **Méthodes reposant sur un modèle d'apprentissage.** Contrairement aux méthodes agnostiques, d'autres méthodes entremêlent la construction d'un modèle d'apprentissage supervisé avec la sélection de variables, avec une procédure de couplage (forte ou faible). Vous avez vu par exemple la sélection forward/backward de variables explicatives dans un modèle de régression linéaire : cette sélection est complètement spécifique au modèle choisi. Une alternative similaire pour le modèle linéaire, plus récente et très populaire, s'appelle le Lasso (vous l'apprendrez en 3A). Mais attention, cela ne veut pas dire qu'une variable non sélectionnée n'est pas influente : cela veut juste dire que **pour le modèle considéré uniquement**, elle n'est pas utile. Par exemple une variable centrée avec une influence quadratique ne sera jamais détectée avec un modèle linéaire (comme d'ailleurs elle ne l'aurait pas été avec une méthode agnostique basée sur la corrélation), voir également la remarque ci-dessous. Il est à noter que l'idée de sélection se généralise pour d'autres types de modèles que vous apprendrez plus tard, mais elle est moins populaire que dans le cadre de la régression linéaire ou logistique.

Une autre approche, qui s'utilise pour les arbres et les forêts, est l'élimination récursive de caractéristiques, qui se base sur la capacité des arbres et des forêts d'estimer après leur construction l'importance des variables explicatives (avec des mesures comme le MDI ou le MDA, voir le chapitre correspondant). L'idée est de commencer par construire un modèle avec toutes les variables, de calculer avec ce modèle l'importance de chacune d'entre elles, puis de retirer celle qui a le moins d'influence et de répéter le processus.

Remarque (⚠ Ne pas sauter trop vite sur des conclusions). Lorsqu'une variable explicative n'est pas sélectionnée lors d'une sélection couplée avec la construction d'un modèle, il convient de ré-insister que cela ne veut pas dire qu'elle n'a pas d'influence sur la variable cible. Au-delà de l'exemple d'une variable qui aurait un effet quadratique et ne serait pas détectée par un modèle linéaire, il y a un cas plus subtil mais très fréquent : celui d'une variable dont on serait capable de détecter l'influence, mais qui n'a aucune utilité pour améliorer le modèle car il a déjà sélectionné à une itération précédente une variable explicative très influente sur la cible, mais très fortement dépendante de cette variable. Cette nouvelle variable apporte donc une information redondante pour expliquer la cible, et le modèle n'a pas

besoin de s'en servir pour faire des bonnes prédictions. Cet exemple est un piège fréquent, à retenir.

5.3.3 Projection / Plongement / Embedding

L'autre approche pour réduire la dimension, le plongement, vise à construire des nouvelles variables explicatives en petit nombre. Il existe trois grandes classes de méthodes :

1. **Méthodes non-supervisées.** Ces méthodes utilisent uniquement les variables explicatives pour identifier une transformation, qui en générale prend la forme d'une compression qui cherche à conserver la variabilité observée des variables explicatives, mais à l'aide d'un nombre réduit de nouvelles variables. Dans cette famille vous connaissez particulièrement bien l'analyse en composant principale (ACP), qui cherche les nouvelles variables sous la forme de combinaisons linéaires des variables originales. Une généralisation non-linéaire directe basée sur les méthodes à noyaux, appelée l'ACP à noyau, utilise des concepts que vous apprendrez en 3A. Plus récemment, deux méthodes ont émergé et sont extrêmement populaires en data science ces dernières années : l'algorithme **t-SNE** et l'algorithme **UMAP** (dont vous avez eu une illustration plus tôt dans ce cours). Ces deux algorithmes sont basées sur l'idée commune de trouver une transformation pour laquelle les voisins d'un individu une fois plongé dans le nouvel espace sont proches de ces voisins dans l'espace initial. Initialement développés pour faire de la réduction de dimension à but de visualisation principalement (donc plongement en dimension 2 ou 3 maximum), ils sont également utilisés de manière plus générale en amont de modèles d'apprentissage supervisés.
Pour des variables explicatives plus exotiques (voir la section Bonus 1 à la toute fin), des méthodes particulières basées sur des réseaux de neurones ont été proposées, en particulier pour des images, telles que les auto-encodeurs.
2. **Méthodes supervisées agnostiques.** Pour quelques-unes des méthodes précédentes, des extensions ont été proposées pour guider la transformation avec les valeurs de la variable cible (typiquement en généralisant le concept des voisins de l'espace des variables explicatives, en rajoutant la valeur de la cible). Un cas fameux est celui de la méthode UMAP (disponible uniquement dans la librairie Python, pas encore en R).
3. **Méthodes supervisées reposant sur un modèle d'apprentissage.** Enfin, certains modèles d'apprentissage supervisé sont définis avec un plongement explicite dans leur formulation, qui est estimé en même temps que le modèle est construit (de manière itérative ou non). L'exemple historique, qui mélange ACP et modèle linéaire, est la régression des moindres carrés partiels (*Partial Least Squares* en anglais, ou PLS). Elle cherche à identifier les combinaisons linéaires des variables explicatives qui expliquent linéairement le mieux la cible. Ce principe a été généralisé pour des modèles non-linéaires, mais toujours avec un plongement linéaire à l'intérieur.
Enfin, une fois de plus pour des variables explicatives plus exotiques (voir

la section Bonus 1 à la toute fin), des architectures spécifiques de réseaux de neurones (U-net par exemple) consistent en une première partie de plongement, dont les paramètres de construction sont estimés lors de la phase d'entraînement.

Remarque ( Pas de plongement utile si les variables explicatives sont indépendantes avec une méthode non-supervisée). Si vous utilisez une méthode de plongement non-supervisée, gardez à l'esprit que la dimension du nouvel espace sera réduite uniquement si les variables explicatives originelles sont dépendantes, c'est-à-dire qu'elles vivent en réalité dans une variété de dimension plus petite que d . Si ce n'est pas le cas, il est évident qu'il est impossible de compresser l'information contenue dans le jeu de données !

Remarque ( Le plongement est utile même sans réduire la dimension). À l'origine un plongement est par définition utile pour trouver un espace de plus petite dimension pour les variables explicatives. Mais si vous utilisez une méthode non-linéaire, en gardant la même dimension vous pourrez potentiellement trouver des nouvelles variables qui rendent le problème d'apprentissage supervisé plus simple. Vous pouvez vous servir du dernier exemple de la section Bonus 2 pour vous faire une opinion sur la question (dans le cas où tous les individus auraient été labellisés dans cet exemple).

Remarque ( Le plongement est utile en apprentissage non-supervisé aussi). Nous avons discuté de l'utilité du plongement dans le cadre de l'apprentissage supervisé, cependant les méthodes non-supervisées sont également populaires comme pré-traitement pour de l'apprentissage non-supervisé tel que le clustering ou l'estimation de densité. Par exemple en employant une méthode non-linéaire, il est possible que dans l'espace transformé un algorithme des k-means se comportera mieux (une fois de plus vous pouvez regarder le dernier exemple de la section Bonus 2, en imaginant que le problème aurait été du clustering non-supervisé). D'autre part, réduire la dimension simplifie potentiellement le problème d'estimation de densité. Un exemple populaire est récent est celui des réseaux de neurones génératifs (les GAN) !

5.4 Normalisation et transformations préliminaires de variables

La normalisation des variables est une étape cruciale dans le prétraitement des données pour l'apprentissage supervisé. Les variables dans un jeu de données peuvent avoir des échelles très différentes, ce qui peut avoir un impact significatif sur certains algorithmes d'apprentissage supervisé. Lorsque les variables ne

sont pas normalisées, les modèles d'apprentissage supervisé peuvent attribuer involontairement plus de poids aux caractéristiques avec des échelles plus larges, ce qui peut biaiser les résultats. Par exemple, un algorithme comme la régression linéaire est sensible à l'échelle des variables, de sorte que des variables avec des échelles différentes peuvent biaiser les coefficients du modèle. La normalisation vise à mettre toutes les variables sur la même échelle (généralement entre 0 et 1, ou alors de moyenne nulle et de variance 1), afin que chaque variable contribue de manière équitable à la construction du modèle. Cela facilite la comparaison de l'importance des différentes variables et améliore la convergence de nombreux algorithmes d'optimisation. En outre, elle peut aider à éviter la domination d'une variable par rapport aux autres dans les méthodes basées sur la distance, telles que les plus proches voisins.

5.4.1 Impact sur les modèles vus en cours

Pour les algorithmes vus dans ce cours, la normalisation est particulièrement critique pour les plus proches voisins, à cause de l'utilisation de la distance entre variables (plus généralement cela sera le cas pour toutes les méthodes utilisant une distance). Il est donc conseillé de systématiquement normaliser les données pour les KNN. La normalisation peut également avoir un impact, léger, sur la LDA / QDA et le Bayésien naïf, mais d'un point de vue stabilité numérique uniquement (pour l'estimation de matrices de covariance et l'estimation de densité, respectivement). Enfin, les arbres CART et les forêts aléatoires sont insensibles à toute transformation monotone des variables explicatives, et ne requièrent donc pas théoriquement de normalisation. Cependant, le conseil général est de systématiquement normaliser les variables au début de l'étude, car en pratique de toute façon il sera fréquent d'entraîner et de comparer plusieurs modèles, dont certains seront sensibles aux échelles.

5.4.2 Transformations

Avant de normaliser les variables, il est important de déterminer si une transformation préliminaire est nécessaire. La nécessité de la transformation peut être détectée de plusieurs manières. Tout d'abord, un simple graphique des valeurs des variables peut révéler des distributions asymétriques ou hétéroscédastiques. Les données qui ne suivent pas une distribution normale ou qui ont des valeurs extrêmes peuvent bénéficier de transformations pour les rendre plus proches d'une distribution normale. Un histogramme, un diagramme quantile-quantile (Q-Q plot) ou un test statistique de normalité comme le test de Shapiro-Wilk peuvent être utilisés à cette fin. Ensuite, l'examen des relations entre les variables et la variable cible est essentiel. Des transformations peuvent être nécessaires si la relation entre une variable explicative et la variable cible est non linéaire. Par exemple, si une relation quadratique ou logarithmique est présente, une transformation peut être utilisée pour linéariser la relation. Les transformations couramment utilisées comprennent la transformation logarithmique, la racine carrée, la réciproque, ou même des transformations plus complexes comme celles de Box-Cox ou Yeo-Johnson, qui adaptent la transformation en fonction de la distribution des données. Cependant, il est essentiel de noter que les transformations préliminaires doivent être appliquées avec prudence, car elles peuvent influencer les résultats et l'interprétation du modèle. Il est donc recommandé

de tester différentes transformations et de choisir celle qui améliore le mieux la qualité du modèle sans introduire de distorsions indésirables.

Remarque ( Les transformations se font sur l'échantillon d'apprentissage). Comme pour l'encodage, l'identification d'une possible transformation se fait exclusivement sur l'échantillon d'apprentissage.

5.5 Gérer les données manquantes

5.5.1 Généralités

Définition des valeurs manquantes. Les valeurs manquantes sont un élément omniprésent dans de nombreux jeux de données. Elles désignent simplement l'absence d'informations pour certaines observations ou variables. Cela peut survenir pour diverses raisons, telles que des erreurs de collecte de données, des non-réponses des répondants ou des problèmes de stockage. La gestion efficace des valeurs manquantes est essentielle, car elles peuvent fausser considérablement les résultats de l'apprentissage supervisé si elles ne sont pas traitées correctement.

Les valeurs manquantes ne sont pas uniformes et peuvent se manifester de différentes manières. Une compréhension approfondie de leur nature est cruciale pour choisir la meilleure stratégie de gestion. Dans ce contexte, nous distinguons trois types principaux de valeurs manquantes. Le premier est le MCAR (*Missing Completely At Random* en anglais), où les valeurs manquantes surviennent de manière aléatoire et indépendante des autres données. Le deuxième est le MAR (*Missing At Random* en anglais), où les valeurs manquantes dépendent d'autres variables observées. Enfin, le MNAR (*Missing Not At Random* en anglais) concerne les valeurs manquantes dépendantes d'une information non observée. Cette distinction est fondamentale pour déterminer les méthodes appropriées et évaluer l'impact sur les résultats.

Impact sur les modèles vus en cours. Pour les algorithmes vus dans ce cours, à l'exception des KNN qui y sont sensibles, les valeurs manquantes ont un impact assez limité (à condition qu'elle ne soient pas trop nombreuses évidemment) :

- Pour les arbres et les forêts, grâce au principe des coupures équi-divisantes
- Pour la LDA / QDA et le Bayésien naïf, les estimations de densité ne sont que peu perturbées

Pour les KNN, le calcul des distances est impacté et l'algorithme ne peut donc pas être utilisé sans modification, et la majorité des autres modèles de 2A et 3A sont dans le même cas. C'est la raison pour laquelle nous discutons de stratégies usuelles de contournement ci-dessous.

5.5.2 Traitement des valeurs manquantes

Suppression des lignes ou des colonnes. Lorsqu'on est confronté à des valeurs manquantes dans un jeu de données, l'une des méthodes les plus simples pour les gérer est de les supprimer. Il existe deux approches principales à cette méthode : la suppression de lignes et la suppression de colonnes. La suppression de lignes consiste à retirer les individus qui contiennent des valeurs manquantes. Cette approche peut être efficace si les valeurs manquantes sont peu nombreuses par rapport à la taille totale du jeu de données. Cependant, elle peut entraîner une perte significative d'informations. La suppression de colonnes, en revanche, consiste à éliminer les variables (ou caractéristiques) qui comportent des valeurs manquantes. Cela peut être approprié lorsque certaines caractéristiques ne sont pas essentielles pour la tâche d'apprentissage supervisé en cours. Néanmoins, cette approche est potentiellement dangereuse et peut avoir des conséquences sur la qualité des prédictions. Voici par exemple ce que l'on obtient avec une suppression des lignes :

X^1	X^2	X^3		X^1	X^2	X^3
A	3.42		→	B	2.4	-11.02
A	5.24			C	-1.1	8.57
	-6.57	3.29				
B	2.4	-11.02				
C	-1.1	8.57				
	2.5	-4.02				

et avec une suppression des colonnes :

X^1	X^2	X^3		X^2
A	3.42		→	3.42
A	5.24			5.24
	-6.57	3.29		-6.57
B	2.4	-11.02		2.4
C	-1.1	8.57		-1.1
	2.5	-4.02		2.5

Vous pourrez constater que ce sont des stratégies risquées, qui font potentiellement perdre beaucoup d'information.

Imputation des valeurs manquantes. L'imputation des valeurs manquantes est une approche plus sophistiquée qui vise à remplacer les valeurs manquantes par des valeurs estimées. Plusieurs méthodes d'imputation existent, dont certaines sont simples, telles que l'imputation par la moyenne, la médiane ou la mode (de la colonne dans laquelle il y a une valeur manquante). Ces méthodes conviennent lorsque les données manquantes sont MCAR ou MAR. Cependant, pour les données MNAR, des techniques plus avancées telles que l'imputation par régression ou classification peuvent être nécessaires (on entraîne un modèle de régression pour prédire une variable explicative en fonction des autres, et on

impute la prédiction de ce modèle), mais évidemment cela ne fonctionne que s'il y a une dépendance entre les variables explicative. Une version plus moderne de ce principe repose sur la factorisation de matrice. Enfin, l'imputation multiple par la méthode MICE (*Multiple Imputation by Chained Equations*) est également largement utilisée pour gérer les valeurs manquantes de manière plus robuste et réaliste. Mais le choix de la méthode d'imputation dépend de la nature des données, de la taille de l'échantillon et de l'objectif de l'analyse. Sur l'exemple précédent, voilà à quoi ressemble une imputation par la moyenne (pour les variables explicatives continues) et une imputation par la catégorie la plus fréquente (pour les variables explicatives catégorielles).

X^1	X^2	X^3		X^1	X^2	X^3
A	3.42		→	A	3.42	-0.795
A	5.24			A	5.24	-0.795
	-6.57	3.29		A	-6.57	3.29
B	2.4	-11.02		B	2.4	-11.02
C	-1.1	8.57		C	-1.1	8.57
	2.5	-4.02		A	2.5	-4.02

Remarque (⚠ L'imputation se fait sur l'échantillon d'apprentissage). Comme pour l'encodage et les transformations de variables, l'imputation s'estime exclusivement sur l'échantillon d'apprentissage.

Remarque (⚠ Le type d'imputation est un hyperparamètre!). Si dans la pratique vous souhaitez tester différents types d'imputation et choisir la plus adaptée à votre problème, l'imputation devient de facto un hyperparamètre, et vous devrez utiliser de la validation croisée pour identifier la meilleure.

Encodage des valeurs manquantes. Une autre approche pour gérer les valeurs manquantes, uniquement dans le cas d'une variable explicative catégorielle, consiste à encoder les données manquantes comme une catégorie spéciale distincte. Cela signifie que les valeurs manquantes sont traitées comme une information à part entière, plutôt que d'être remplacées par une estimation. Cette méthode peut être particulièrement utile lorsque la raison des valeurs manquantes est significative ou lorsque l'absence de données est en soi un indicateur d'information. Toutefois, il est important de choisir avec soin cette option, car elle peut influencer la manière dont les modèles d'apprentissage supervisé interprètent les données. L'encodage des valeurs manquantes est couramment utilisé dans des domaines tels que la biologie, où des valeurs manquantes peuvent avoir une signification biologique spécifique. Sur l'exemple précédent encore une fois, on illustre une imputation par la moyenne (pour les variables explicatives continues) et un encodage avec une nouvelle catégorie "M" pour "Manquante" (pour les variables explicatives catégorielles).

X^1	X^2	X^3		X^1	X^2	X^3
A	3.42			A	3.42	-0.795
A	5.24			A	5.24	-0.795
	-6.57	3.29	→	M	-6.57	3.29
B	2.4	-11.02		B	2.4	-11.02
C	-1.1	8.57		C	-1.1	8.57
	2.5	-4.02		M	2.5	-4.02

5.6 Optimisation des hyperparamètres d'un modèle

Vous savez désormais que pour optimiser les hyperparamètres d'un modèle d'apprentissage supervisé, on a fréquemment recours à une procédure de validation croisée. Dans les bibliothèques que vous avez utilisées, en général la procédure est la suivante :

1. Choix des hyperparamètres à faire varier, et des valeurs à tester pour chacun d'entre eux
2. Construction d'une grille qui contient toutes les combinaisons possibles de ces valeurs
3. Pour chaque combinaison, calcul de l'erreur par validation croisée
4. Sélection de la combinaison qui a l'erreur la plus faible

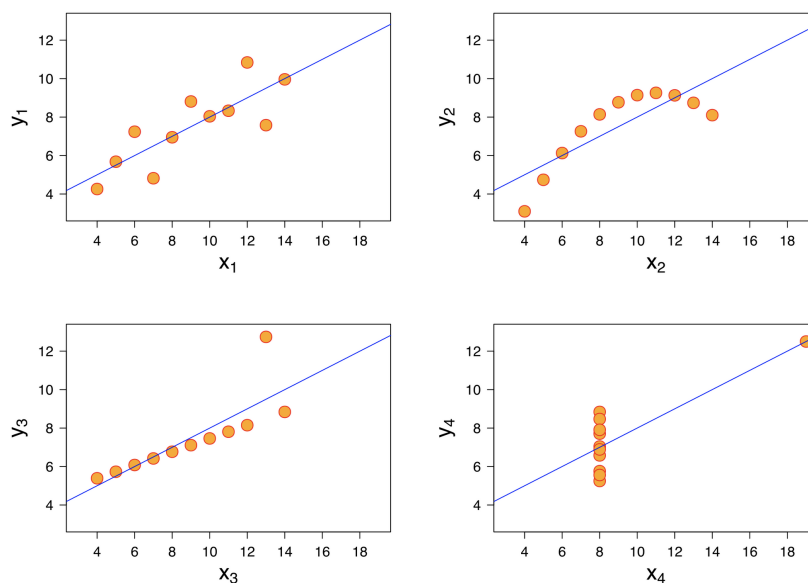
Sur des problèmes réels, possiblement de grande taille, l'étape 3. de calcul de l'erreur peut être particulièrement coûteuse en temps de calcul (vous en avez peut-être même fait l'expérience vous-mêmes avec certaines méthodes). Dans le cas où le nombre de combinaisons à tester est grand, cette procédure peut s'avérer alors très chronophage, et même tellement longue qu'elle en devient inutilisable. Une approche plus moderne, de plus en plus répandue, consiste à remplacer la construction d'une grille de combinaison par un algorithme d'optimisation, qui prend comme variables à optimiser les hyperparamètres et à qui on indique des bornes sur chacun d'entre eux. La fonction objectif qui est minimisée par l'optimiseur est alors l'erreur de validation croisée. En pratique, le nombre de fois où l'optimiseur va demander à calculer la fonction objectif (qui, on le rappelle, peut être coûteux) est souvent plus faible que le nombre total de calculs qu'on aurait fait avec une grille. Enfin, il est également de plus en plus fréquent d'utiliser une stratégie d'optimisation spécifique et adaptée pour les cas de fonctions objectives coûteuses, ce qui permet de réduire encore plus le temps de calcul de la procédure.

5.7 Visualiser, visualiser, visualiser

On ne saurait trop insister sur le fait que les outils d'analyse descriptive que vous avez appris en 1A, et en particulier les visualisations, restent toujours précieux en apprentissage supervisé, et qu'il est très fortement recommandé de les mettre en œuvre systématiquement. Les visualisations sont fréquemment

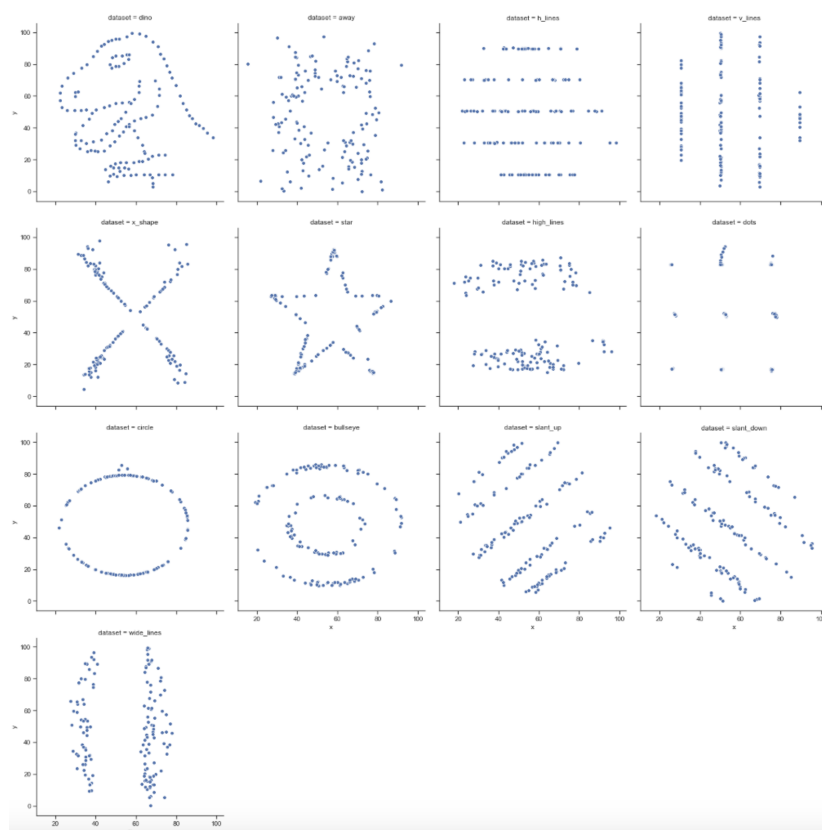
oubliées au profit d'indicateurs quantitatifs (moyennes, variances, corrélations, R^2 , etc), mais elles possèdent de nombreux avantages non négligeables :

1. Pour identifier de potentielles transformations de variables comme évoqué précédemment, tracer l'histogramme de toutes les variables explicatives et de la variable cible s'avère souvent utile. Vous pourrez trouver quelques exemples ici : <https://asq.org/quality-resources/histogram>.
2. Toujours pour identifier des transformations, mais aussi pour explorer si certaines variables explicatives ont plus d'influence que d'autres sur la cible, des scatterplots de la cible (en ordonnée) par rapport à une variable explicative (en abscisse) sont souvent informatifs.
3. **Les indicateurs quantitatifs sont potentiellement très trompeurs**, car par nature ils agrègent l'information d'un jeu de donnée, et résument de manière grossière le phénomène. Un exemple très connu est celui du quartet d'Anscombe. Il s'agit de 4 jeux de données, pour lesquels on a une variable explicative x et une variable cible y , qui ont la particularité d'avoir tous les mêmes indicateurs quantitatifs standard pour les variables : même moyenne, variance, corrélation et R^2 . Si on se limite à cette vision, alors on pourrait conclure que ces 4 jeux de données représentent des phénomènes similaires. Cependant, si l'on affiche tout simplement les scatterplots correspondant, la réalité est toute autre :



Il est évident à la lecture de ces graphiques qu'en réalité les jeux de données correspondent à des situations pratiques très différentes : 1) une relation linéaire avec du bruit sur les observations de la cible y 2) une relation non-linéaire 3) une relation linéaire avec très peu de bruit mais une observation aberrante sur y 4) des mesures bruitées répétées pour une seule valeur de x , avec une observation aberrante sur x et y . Cette simple visualisation vous permet de mieux comprendre le phénomène pour lequel vous avez

des observations, en terme de relation, de présence de données aberrantes, etc. De manière amusante, pour appuyer ce propos, un autre exemple similaire au quartet d'Anscombe a été proposé, avec des variations encore plus surprenantes mais toujours des indicateurs quantitatifs identiques, c'est le Datasaurus Dozen :



5.8 Bonus 1 : gérer des variables explicatives plus "exotiques"

Dans tout le cours, nous nous sommes focalisés sur un cadre "classique" d'apprentissage supervisé, où les variables explicatives vivent dans un espace $\mathcal{X}$ inclus dans $\mathbb{R}^d$, c'est-à-dire qu'elles sont données sous la forme de valeurs scalaires. C'est également dans ce cadre que vous avez travaillé dans le cours de clustering en 1A. Mais en pratique, de plus en plus de tâches d'apprentissage (supervisé ou non) concernent des données qui ne sont pas scalaires : les images sont désormais monnaie courante (classification d'images, clustering d'images, segmentation d'images, segmentation sémantique d'images, super-résolution, complétion, génération, etc), de même que le texte (classification de textes, clustering de textes, génération, etc) mais aussi l'audio, la vidéo, les graphes, les molécules, les séries temporelles, etc.

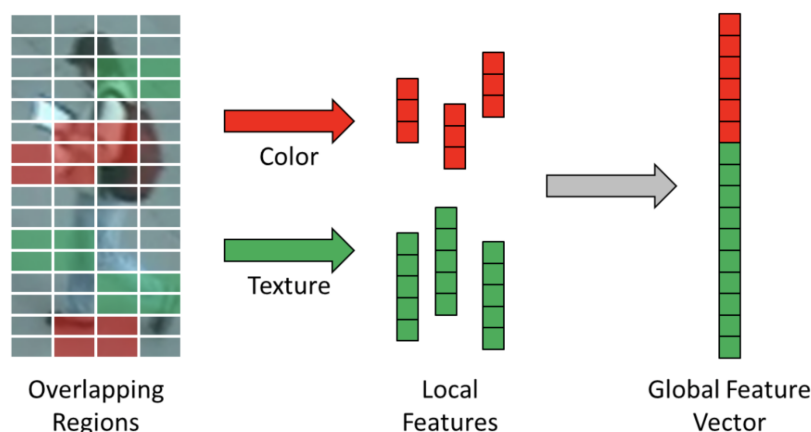
Il faudrait un cours entier pour aborder tous ces aspects, et nous proposons ici de nous focaliser sur quelques-uns d'entre eux : tout d'abord le principe général

de plongement, qui consiste à construire une représentation vectorielle (donc plusieurs variables explicatives scalaires) d'un objet "exotique". Ce plongement transforme toutes les données dans $\mathbb{R}^d$, et donc les algorithmes usuels peuvent être utilisés directement sur cette nouvelle représentation. C'est une généralisation du principe de plongement ou embedding dont nous avons discuté dans la section sur la réduction de dimension.

Comme dit précédemment, le plongement se base donc sur l'identification d'une représentation vectorielle d'un objet, cette représentation étant ensuite utilisée pour la tâche d'apprentissage. Mais bien évidemment la qualité du résultat final de l'apprentissage dépendra très fortement de la qualité de la représentation. Par exemple pour un problème de classification d'images il serait tout à fait possible de représenter chaque image par une valeur, la moyenne de l'intensité de ses pixels. Mais cette représentation n'est pas du tout expressive et ne rend pas compte des propriétés intrinsèques de chacune des images : il est donc à peu près certain que cette variable ne suffira pas du tout à expliquer qu'une image est dans une catégorie plutôt qu'une autre.

Descripteurs (non-supervisé). Historiquement, l'utilisation de descripteurs pour construire un plongement a été très populaire, mais plutôt connue sous le nom d'extraction de caractéristiques (*feature extraction* en anglais). Le principe est simple : chaque objet est représenté par un ensemble de descripteurs spécifiques au type d'objet, ces descripteurs correspondant à des résumés informatifs de l'objet guidés par une connaissance métier. Parmi les exemples connus, on peut citer

- Pour les images, des résumés locaux ou globaux des couleurs, des textures ou des contours, comme dans la figure ci-dessous.

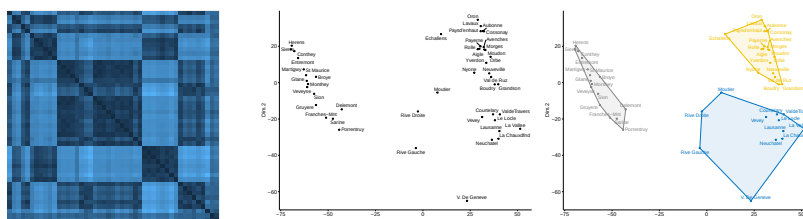


- Pour les textes, des résumés sous forme de fréquence de mots d'un dictionnaire (généralisé avec le fameux "tf-idf"), ou plus généralement de n-grammes.
- Pour les molécules, des comptages d'atomes, de leurs distances, ou encore des représentations sous forme de chaîne de caractères (les transformant alors en du texte).

- Pour les graphes, des résumés de la connectivité, des vecteurs propres issus d'une analyse spectrale.
- ...

Cette approche a pendant longtemps été la plus performante, avant l'arrivée des architectures de réseaux de neurones profonds et en particulier ceux basés sur de la supervision (en effet les descripteurs mentionnés précédemment, bien que justifiés car construits sur des critères métier, sont fixes et non pas estimés pour répondre à une tâche précise d'apprentissage supervisé).

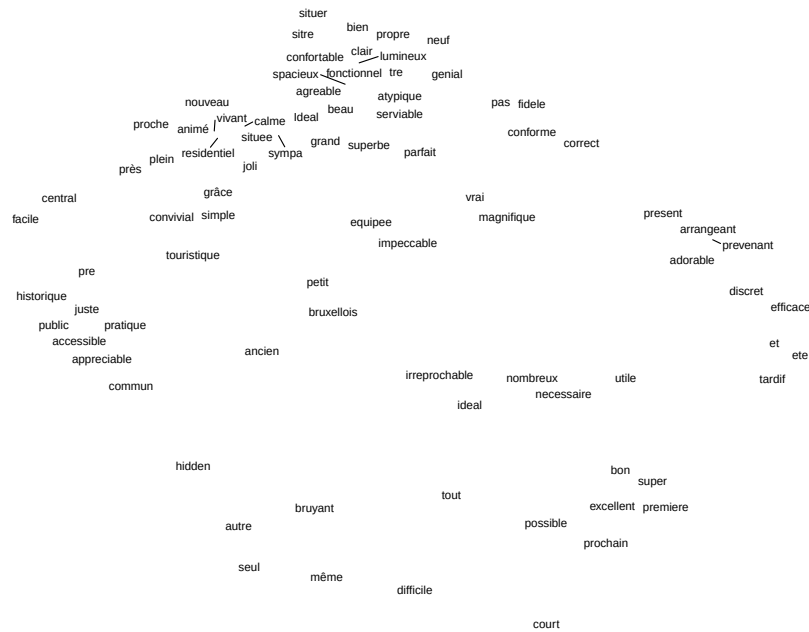
Quand une distance adaptée est disponible (non-supervisé). Pour certains types d'objets, une représentation sous forme de descripteurs n'est pas forcément disponible, pour lesquels des connaissances du domaine d'application ne permettent pas de définir des descripteurs directement, mais permettent en revanche de définir une distance entre les objets. Par exemple, pour des images ayant été au préalable segmentées en zones de certains types, on construit une distance de Hausdorff (i.e. entre ensembles) entre les zones de même type. Ou encore pour des molécules, des distances basées sur des critères physiques entre différents groupes d'atomes. Pour la méthode des KNN (mais aussi pour les k-means ou la CAH pour l'apprentissage non-supervisé), de telles distances peuvent être directement utilisées dans l'algorithme, mais ce n'est pas le cas de la grande majorité des autres algorithmes que vous avez appris. Tout n'est cependant pas perdu, car il existe une méthode qui permet, à partir d'une distance définie entre objets, de trouver une représentation vectorielle dans $\mathbb{R}^d$ de chacun d'entre eux, de telle manière que la distance euclidienne dans cet espace de représentation soit proche de la distance donnée au départ. Cette représentation finale sert donc de plongement, pour transformer tous les objets, et qui est utilisé ensuite dans n'importe quelle tâche d'apprentissage. Cette méthode s'appelle le *Multidimensional Scaling* (MDS), elle est illustrée ci-dessous avec à gauche la matrice de distance entre tous les objets du jeu de données, au centre une représentation vectorielle de dimension 2 où les distances sont préservées, et à droite le résultat d'un clustering par k-means dans ce nouvel espace.



Bien évidemment, si la distance définie au départ n'est pas expressive pour la tâche d'apprentissage, la représentation obtenue par MDS ne sera pas pertinente et ne donnera donc pas de bons résultats.

Réseaux de neurones (non-supervisé ou supervisé). Pour des données de type image ou texte, les architectures de réseaux de neurones profonds proposées récemment ont démontré une efficacité impressionnante pour des tâches d'apprentissage supervisé. En particulier, les architectures performantes

reposent souvent sur une première partie de réseau consistant à construire un plongement des données, au travers de filtres adaptés au type de données (couches convolutionnelles pour les images par exemple). De tels réseaux ont déjà été entraînés sur des tâches de classification ou de régression, sur des bases de données d'images ou de textes très volumineuses, et ils sont disponibles facilement. Il vous est donc aisé de les utiliser pour n'importe quel nouvel tâche sur des images ou du texte, en utilisant uniquement la première partie du réseau qui fait le plongement pour transformer vos propres données, et utiliser cette représentation vectorielle dans n'importe quel algorithme. C'est ce que l'on appelle communément le *transfer learning*. Ci-dessous on donne un exemple d'un plongement obtenu avec un réseau de neurones "word2vec" de données textuelles correspondant à des avis sur des Airbnb à Bruxelles, et on visualise une représentation vectorielle en deux dimensions des adjectifs.



Une ressource particulièrement pratique pour accéder à une multitude de réseaux de neurones pré-entraînés pour des tâches et des types de données variés : <https://huggingface.co/models>. Il existe aussi des réseaux entraînés de manière non-supervisée sur des images, tels que des auto-encodeurs ou auto-encodeurs variationnels.

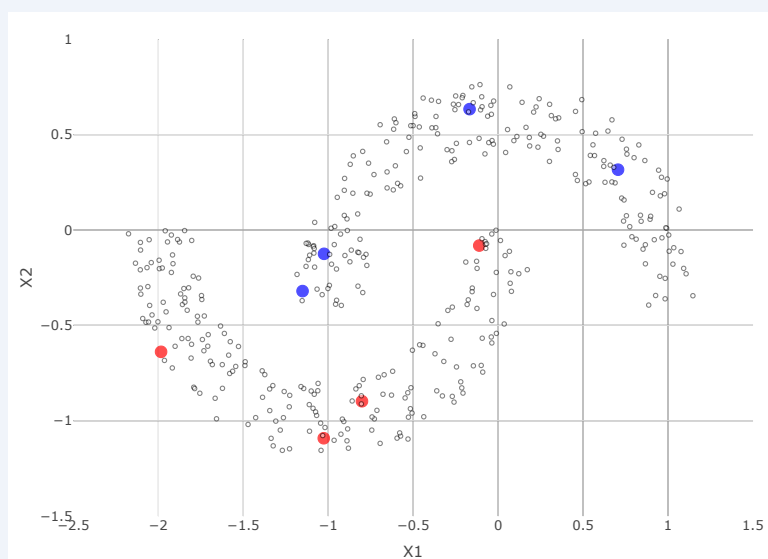
5.9 Bonus 2 : l'apprentissage semi-supervisé

L'apprentissage semi-supervisé est une approche qui se situe entre l'apprentissage supervisé et l'apprentissage non supervisé. Dans ce scénario, vous avez accès à la fois à des données étiquetées (apprentissage supervisé, couples $\{\mathbf{x}_i, y_i\}_{i=1}^n$) et non étiquetées (apprentissage non-supervisé, observations $\{\mathbf{x}_j\}_{j=1}^N$). Les données étiquetées sont généralement rares et coûteuses à obtenir, tandis que les données

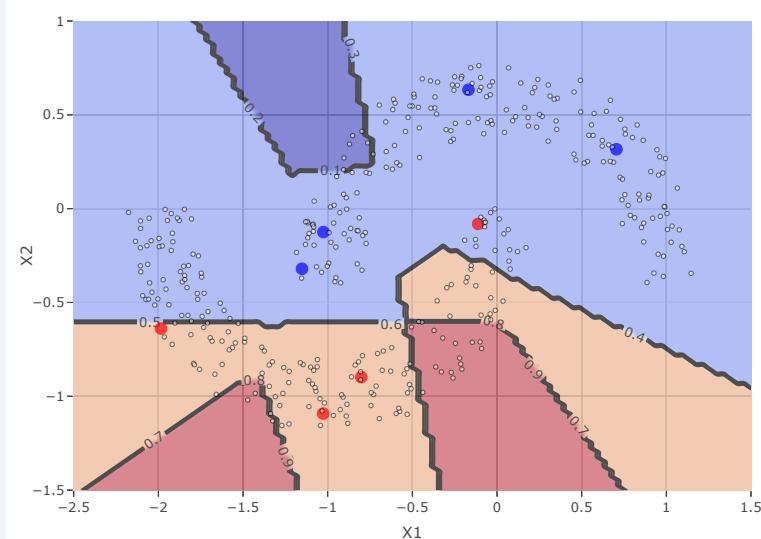
non étiquetées sont plus abondantes ($N \gg n$). Voici un exemple ci-dessous.

Exemple (Apprentissage semi-supervisé sur le jeu de données "Deux Lunes").

Sur le jeu de données Deux Lunes que nous avons vu à plusieurs reprises dans ce cours, imaginons qu'il soit très coûteux (en temps ou en argent) d'obtenir le vrai label pour chaque individu, et que nous n'avons pu le faire que pour un petit nombre d'entre eux, alors qu'il est peu coûteux d'obtenir les caractéristiques de chacun d'entre eux. On se retrouve alors dans la situation suivante.



Les individus en rouge et en bleu sont ceux pour lesquels on a le vrai label (couples $\{\mathbf{x}_i, y_i\}_{i=1}^n$), mais pour tous les autres nous n'avons que leurs caractéristiques ($\{\mathbf{x}_j\}_{j=1}^N$). On peut très bien utiliser un KNN avec $k = 3$ sur les données étiquetées uniquement, et l'on obtient le résultat ci-dessous.

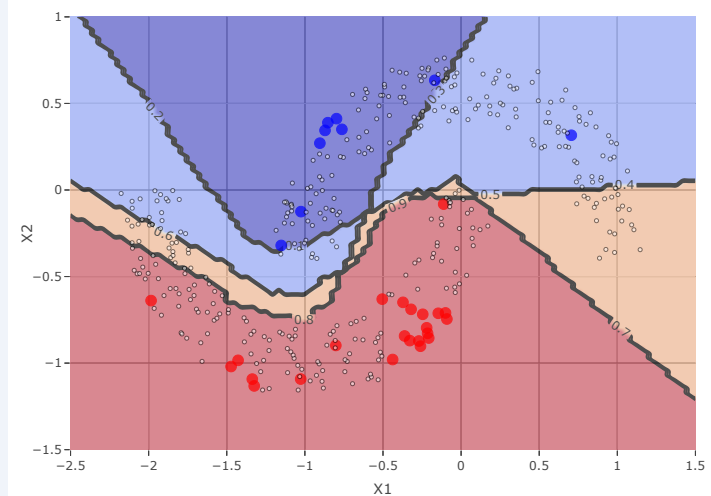


Mais évidemment on souhaiterait utiliser les données non labellisées pour guider la construction du classifieur et améliorer sa qualité.

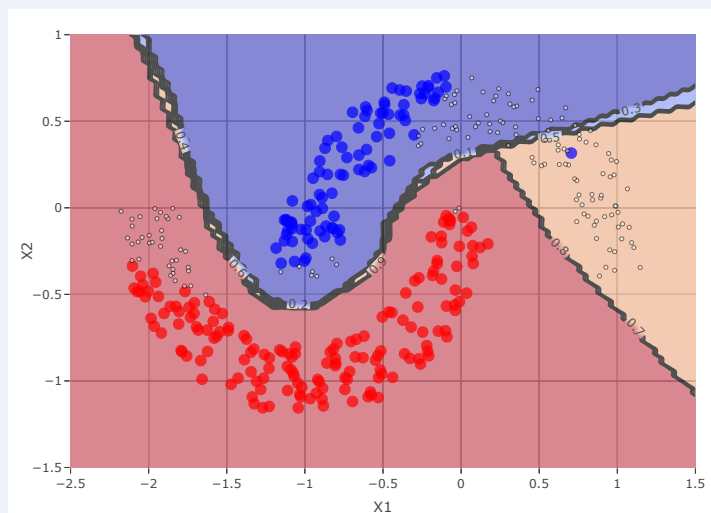
L'objectif de l'apprentissage semi-supervisé est d'exploiter ces données étiquetées et non étiquetées pour construire un modèle de meilleure qualité (que celui qu'on obtiendrait en utilisant uniquement les données labellisées). Plusieurs approches sont possibles, nous ne mentionnons ici que les trois méthodes les plus utilisées en pratique :

1. **L'auto-étiquetage (Self-training).** Cela consiste à utiliser un modèle initial pour attribuer à des données non étiquetées des labels *virtuels* (prédits par le modèle). Ces nouvelles étiquettes virtuelles sont ensuite ajoutées aux données étiquetées comme si elles étaient des vraies observations, et le modèle est ré-entraîné avec ce jeu de données augmenté. L'auto-étiquetage peut être itératif, où le modèle s'améliore continuellement en ajoutant de nouvelles étiquettes aux données non étiquetées. En pratique, il est fréquent de d'ajouter à chaque itération uniquement les individus pour lesquels les labels sont prédits avec la plus grande confiance.

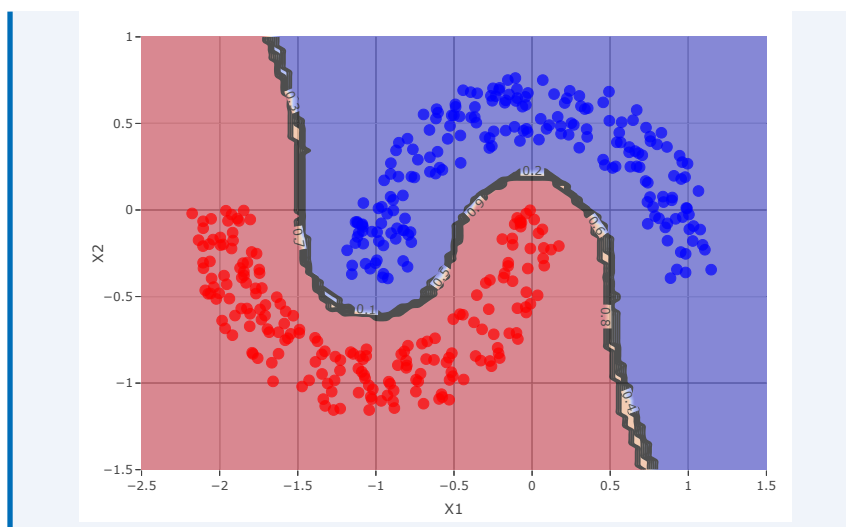
Exemple (Auto-étiquetage sur le jeu de données "Deux Lunes"). Voici par exemple ce que donne cette méthode avec le classifieur KNN avec $k = 3$, en ajoutant à chaque itération les individus non labellisés pour lesquels le KNN a une confiance plus grande que 0.95. Après 1 itération :



Après 2 itérations :



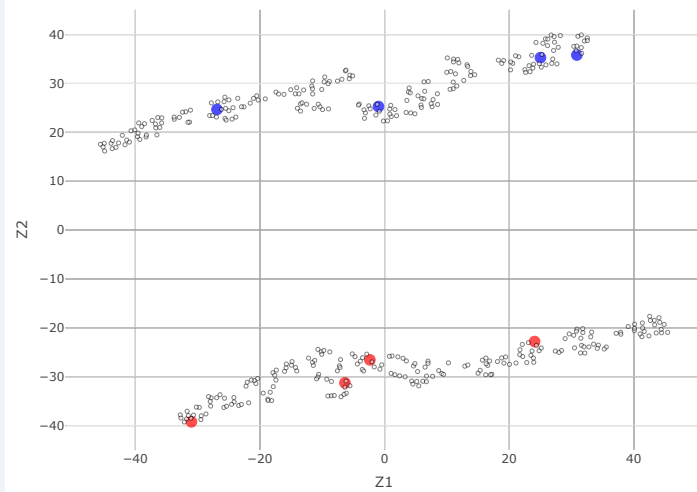
Après 5 itérations :



Même si sur le cas précédent l'auto-étiquetage fonctionne particulièrement bien, il est important de garder à l'esprit que cette approche est potentiellement dangereuse : en effet si le classifieur initial est trop sûr de lui à tort, alors les labels virtuels seront erronés et le modèle s'auto-alimentera de données complètement fausses.

2. **L'apprentissage de représentation.** Une autre approche heuristique mais naturelle consiste à utiliser les variables explicatives de tous les individus labellisés ou non $\{\mathbf{x}_i\}_{i=1}^n \cup \{\mathbf{x}_j\}_{j=1}^N$ pour en apprendre une représentation. On peut par exemple penser à une méthode de réduction de dimension non-supervisée parmi celles abordées dans ce chapitre. Une fois cette transformation $\mathbf{x} \mapsto \mathbf{z}$ construite, un modèle d'apprentissage supervisé est entraîné sur les individus $\{\mathbf{z}_i, y_i\}_{i=1}^n$.

Exemple (Apprentissage de représentation sur le jeu de données "Deux Lunes"). En utilisant l'algorithme t-SNE (évoqué dans ce chapitre) sur les variables explicatives de tous les individus, on obtient la représentation suivante, où l'on a coloré en rouge et bleu les individus pour lesquels on a les labels observés.



L'algorithme a bien appris la structure sous-jacente des données, et dans ce nouvel espace un classifieur simple commettra peu d'erreurs.

3. **La régularisation.** Enfin, une approche plus en lien avec l'apprentissage supervisé repose sur l'idée de formuler le problème comme un problème de minimisation de risque empirique (sur les données étiquetées) avec une pénalisation sur le classifieur obtenu qui le contraint à respecter une certaine structure sur les données non labellisées, par exemple en limitant ses variations. Dans cette classe de méthode on trouve la régularisation de variété (*Manifold regularization*) ou encore la séparation de régions de faible densité. Très souvent ces approches se basent sur des méthodes à noyau que vous étudierez dans le cours de machine learning en 3A.